Diffusion Models for Financial Generation

Synthetic Data, Regime Analysis, Multimodal Sequence Generation, and Financial Applications

A Research Book Wrapper for a Three-Chapter Collection on Diffusion Models in Finance

Alejandro Reynoso

March 11, 2026

Executive Summary

This book gathers three related chapters under a single research umbrella: the use of diffusion models as generative engines for financial data. The three chapters move from synthetic return generation under regime dynamics, to structured market-feature sequence generation, to multimodal sequence-and-text generation. Read separately, each chapter makes a distinct contribution. Read together, they articulate a broader claim. The claim is that diffusion models provide one of the most promising frameworks available for building financial generative systems that are not only statistically expressive, but also structurally interpretable, diagnostically useful, and extensible across modalities.

The first chapter develops the core case for diffusion models in finance by focusing on synthetic financial return generation under regime-dependent dynamics. Its emphasis is on the idea that a financial generator should do more than imitate isolated marginals. It should learn path structure, regime mixtures, and latent market-state organization. The chapter therefore studies a cosine-schedule denoising diffusion probabilistic model with a Transformer denoiser applied to synthetic regime-switching data. The contribution is not merely that such a model can generate realistic-looking samples. The deeper contribution is that intermediate denoising states and latent representations can become useful instruments for regime identification, process diagnosis, stress testing, and model validation.

The second chapter extends this logic into a richer financial setting in which the target of generation is not only a return series but a structured market feature sequence. Here the focus shifts toward OHLCV-style data and engineered technical indicators. The underlying question becomes more operational: how should one build a generative model that can handle realistic structured financial sequences while preserving enough flexibility for practical use in synthetic data generation, imputation, and scenario design? This chapter strengthens the case that diffusion models can move beyond toy one-dimensional return generation and operate in higher-dimensional financial state spaces in ways that remain mathematically coherent and practically useful.

The third chapter goes further by arguing that a truly advanced financial generative model must be multimodal. Financial data do not exist in a purely numerical universe. Price paths, volume structure, technical indicators, sentiment labels, and descriptive technical-analysis language are different manifestations of the same market event. A model that generates only one modality is

therefore incomplete. The third chapter introduces a dual-head diffusion Transformer in which continuous financial-feature sequences and natural-language technical-analysis descriptions are generated jointly through a shared backbone with cross-modal fusion. This creates a multimodal financial generator capable of producing paired synthetic datasets, automated technical commentary, and cross-modal consistency diagnostics.

Taken together, the three chapters define a progression. The first establishes diffusion as a framework for financial path generation under regime uncertainty. The second broadens the scope to more structured feature spaces and practical financial sequence tasks. The third expands the target itself from unimodal numerical generation to multimodal market-sequence-and-text generation. This progression matters because it reflects the actual growth path of the field. Financial generative modeling is unlikely to stop at synthetic returns. It is moving toward richer state representations, richer conditioning schemes, and richer output modalities. These chapters collectively anticipate that movement.

Several themes unify the volume. The first is that **generation in finance is not only about realism, but about usefulness**. A synthetic financial sample is valuable not merely if it looks plausible under a histogram or a stylized-fact table. It is valuable if it supports backtesting, scenario analysis, missing-data reconstruction, regime interpretation, model diagnostics, or downstream fine-tuning workflows. This collection therefore treats generative quality as multidimensional. Marginal fidelity matters, but so do latent structure, path dependence, semantic consistency, and conditional usefulness.

The second theme is that **regime structure matters**. Financial data are not well captured by one stationary law. Volatility changes, drift regimes shift, transitions occur, and market behavior is often better understood as a mixture over latent states than as one homogeneous process. The first chapter addresses this most directly, but the theme persists throughout the volume. Even when the modality changes from returns to OHLCV features or from features to text, the deeper point remains: financial generative systems must be able to organize heterogeneous market states rather than simply average them away.

The third theme is that **Transformers are the natural diffusion backbone for finance**. The original success of diffusion models was rooted in image generation, where U-Nets were natural because images are spatial objects. Financial sequences are different. Their dependencies are often long-range, cross-sectional, regime-conditioned, and semantically structured. Transformers are therefore particularly attractive because self-attention can model non-local temporal and contextual relationships more directly than a purely convolutional architecture. This becomes especially clear in the later chapters, where shared backbones and cross-modal fusion depend on the representational flexibility of attention mechanisms.

The fourth theme is **diagnostic value**. One of the strongest claims running across the collection is that a diffusion model in finance should not be seen only as a black-box sample generator.

Because diffusion models define structured forward and reverse processes, and because their hidden states evolve through denoising, they can serve as analytical tools. One can inspect intermediate representations, compare generated and real distributions, study regime proximity in latent space, or use cross-modal consistency as a quality filter. In that sense, diffusion is not merely generative; it is also diagnostic.

The fifth theme is **modularity and implementation realism**. These chapters are inspired by working Colab notebooks. That matters. The collection is not written as abstract generative theory detached from implementation. It is written in a style that tries to keep the mathematics close to actual research workflows: schedule precomputation, epsilon prediction, DDIM sampling, masked diffusion, feature conditioning, lightweight tokenization, and practical optimization choices are all treated as part of the scientific object rather than mere engineering residue. This makes the book relevant both to readers who care about theory and to readers who want to build working systems.

The sixth theme is **extension toward multimodality**. Financial AI is increasingly moving toward settings where numerical data and language data interact. Traders and analysts do not reason only over prices, and language models trained for finance should not live in total isolation from market sequences. The dual-head multimodal chapter therefore represents more than an isolated architectural experiment. It is a signal of where the field is going: toward joint generation over prices, indicators, annotations, text, and possibly other modalities such as events, reports, or portfolio states. The multimodal chapter is best read not as an endpoint but as a frontier-facing prototype.

This book wrapper exists to make these internal links explicit. Without a wrapper, the three papers could be read as adjacent but separate works. With the wrapper, they become a coherent research trajectory. The book therefore serves two purposes. It organizes the papers into a compact volume, and it provides an interpretive lens through which the collection can be read as one argument about the future of financial generative modeling.

That argument is simple to state. Diffusion models are becoming central to generative modeling because they combine probabilistic rigor, training stability, representational richness, and architectural flexibility. Finance is a domain where those qualities are especially valuable, because uncertainty, heterogeneity, weak signal, and structural change are normal conditions rather than edge cases. When diffusion models are brought into finance thoughtfully – with regime awareness, Transformer backbones, practical sampling schemes, and eventually multimodal fusion – they become more than sample generators. They become a framework for synthetic data creation, structured financial interpretation, and controlled exploration of financial process families.

The three chapters collected here should therefore be read as stages in that larger development. The first answers why diffusion matters for synthetic financial time series under regime uncertainty. The second expands the answer into higher-dimensional structured financial features and applied sequence generation. The third shows that the financial generative problem is ultimately larger than sequences alone and points toward multimodal financial engines that couple prices with language.

Together, they make the case that diffusion models deserve a serious place in the future of financial machine learning.

Preface

This book was assembled because the three papers it contains belong together more deeply than a simple list of related diffusion titles might suggest. Each paper stands independently. Each has its own technical focus, its own target problem, and its own direct contribution. Yet when they are read together, a broader research structure comes into view. That structure is the progressive extension of diffusion modeling from synthetic financial time series, to regime-sensitive diagnostic generation, to multimodal sequence-and-text financial generation. This wrapper exists to make that structure visible.

The timing for such a wrapper is appropriate because diffusion models now occupy a central position in the broader generative modeling landscape. Their rise was first most visible in image synthesis, but their deeper significance lies elsewhere. Diffusion models are attractive because they reframe generation as a structured inverse problem. Instead of asking a model to emit a finished sample in one fragile step, they define a principled forward process that gradually destroys structure and a learned reverse process that reconstructs it. This creates a generative framework that is both probabilistically grounded and empirically stable. For finance, that combination is unusually important. Financial data are noisy, state dependent, structurally heterogeneous, and often weakly identified. A framework built explicitly around corruption, uncertainty, and structured recovery is therefore naturally aligned with the domain.

That alignment is the starting point of this book.

The first paper included in the collection studies diffusion models as generative engines for synthetic financial return data under regime-switching dynamics. At one level, this is a paper about architecture and probability: cosine schedules, Transformer denoisers, reverse processes, and masked diffusion for regime tokens. But at a deeper level, it is a paper about what it means to generate financial data honestly. A synthetic financial path is not useful merely because it resembles historical returns under a few low-order statistics. It becomes useful when it captures regime mixtures, transition structure, path dependence, and latent-state organization. The first paper therefore asks a question that is more ambitious than ordinary simulation: can a diffusion model learn the mixture geometry induced by financial regimes, and can its latent states be used not only for generation but also for diagnosis?

The second paper grows from that question. Once one accepts that diffusion can act as a financial generator under regime dependence, the next natural step is to ask how far that idea can be extended into richer state spaces. Financial reality is rarely summarized by one return dimension. A realistic research workflow often involves structured sequences of open, high, low, close, volume, and technical indicators. Thus the second paper broadens the domain from stylized return generation toward structured financial feature generation. The point of that broadening is not only realism. It is to make the generative problem closer to the objects that downstream researchers and systems actually use. A generator over structured OHLCV-style sequences is already closer to backtesting, imputation, scenario design, and model-development workflows than a generator over abstract returns alone.

The third paper then takes a further step that, in many ways, changes the meaning of the whole project. It argues that a truly advanced financial generative model cannot remain purely numerical. Market behavior is not expressed only through prices and indicators. It is also interpreted, narrated, and acted upon through language. A technical-analysis sentence, a regime description, a sentiment tag, or a structured commentary string is not external to market reality. It is one of the ways market reality is represented and processed. Once one accepts this, the target of generation expands again. The goal is no longer merely to generate a numerical sequence, but to generate a pair: a financial feature sequence together with a matching textual interpretation. The third paper therefore introduces a dual-head diffusion Transformer in which sequence and text are generated jointly through a shared backbone with cross-modal fusion.

Read in order, the three papers describe a movement. The movement begins with diffusion as a way to model synthetic financial paths under regime uncertainty. It proceeds toward more structured feature spaces and more operational financial sequence generation. It then extends into multimodal generation, where market sequences and language are treated as coupled views of the same event. This movement is the true subject of the wrapper. The book is not simply a collection of three papers about diffusion. It is a compact research arc about how financial generative modeling becomes richer, more realistic, and more useful when diffusion is allowed to move from unimodal path generation toward structured and multimodal financial representations.

There is another reason these papers belong together. All three resist the same inadequate view of generative modeling. That inadequate view says that generation is mainly about producing outputs that look plausible. In image generation, the weakness of that view is already visible. In finance, it becomes more serious. A synthetic path that looks plausible under a plot may still be useless for research. A generated OHLCV window may match some stylized facts but fail to support realistic scenario reasoning. A generated text description may sound like technical analysis while being poorly aligned with the underlying feature pattern. These papers therefore take a different position. A generative model in finance should be judged not only by sample realism, but by structural usefulness. Can it preserve regime information? Can it support diagnostics? Can it help impute missing windows? Can it generate paired data for downstream fine-tuning? Can it

expose cross-modal inconsistencies that can be used as quality filters? These are stronger questions than visual or statistical plausibility alone.

This is why the book repeatedly returns to the idea of **diagnostic value**. Diffusion models are attractive not only because they generate samples, but because they define a structured denoising trajectory. At each stage of the reverse process, the model is making an inference about how a corrupted object should be restored. Those inferences leave behind hidden representations, intermediate states, and geometry in latent space. For financial data, such geometry can be meaningful. It can reveal regime neighborhoods, structural ambiguity, or mismatch between modalities. In that sense, diffusion models can become more than generators. They can become instruments for analyzing the processes from which financial data arise. This diagnostic role appears already in the first paper and becomes even more explicit in the later multimodal work.

The collection also reflects a particular methodological preference: theory should remain close to implementation. These papers are inspired by working Colab notebooks and compact synthetic laboratories. That fact matters. It means the collection does not treat diffusion models as remote abstractions. The mathematical objects – schedules, reverse kernels, epsilon predictors, masked token losses, pooled cross-modal representations, feature-conditioned sentiment decoders – are presented as part of an executable research logic. This is important for two reasons. First, it keeps the work grounded. Second, it makes the collection usable by readers who want not only to understand the theory but to reproduce and extend the models.

There is also a pedagogical logic to the wrapper. Each paper contains technical machinery that can be read in isolation, but the broader intellectual gain comes from seeing how the same family of ideas reappears under different levels of complexity. The first paper clarifies why diffusion matters for uncertainty-rich financial generation and how regime structure enters the problem. The second reinforces the point in a richer state space that looks more like actual market features. The third asks what changes when language is added as a generative modality. By placing the papers together, the book allows the reader to experience these not as abrupt leaps but as controlled expansions of the same core idea.

That core idea can be stated plainly. Financial generative modeling should evolve from single-series imitation toward structured, regime-aware, and eventually multimodal generation. Diffusion models are a particularly strong candidate framework for that evolution because they combine probabilistic coherence, stable training, and architectural flexibility. The Transformer backbone strengthens this further because finance is a domain of non-local temporal dependencies, cross-feature relations, and contextual structure. Once multimodality is introduced, the same Transformer machinery becomes even more valuable because it can support shared latent spaces and cross-modal fusion between sequences and text.

Still, the wrapper is not meant to suggest that the research trajectory is complete. On the contrary, the three papers also make visible how much remains open. The first paper shows that regime-

conditioned diffusion is promising, but real financial regimes are more complex than synthetic Markov switching. The second paper shows that structured financial features can be generated, but real market state spaces are still richer. The third paper shows that sequence-and-text generation can be coupled, but its current tokenizer, template corpus, and greedy decoding remain limited relative to real financial discourse. The book therefore documents both achievement and frontier. It shows what has been built, but also what still needs to be solved.

This open-endedness is not a weakness. It is one of the reasons the wrapper is useful. A set of papers read separately may appear as finished products. A set of papers read together under one title can more easily be seen as steps in an ongoing project. The present book should be understood in that way. It is not the definitive statement on financial diffusion models. It is a compact map of one research path through the topic: from regime-aware synthetic generation, to structured financial sequence modeling, to multimodal financial generation with text.

Another reason for collecting the papers now is that the field itself is moving in this direction. Diffusion models have already escaped the boundaries of image synthesis. They are now being used for tabular data, language-like discrete structures, code, multimodal generation, and structured data synthesis. Finance is therefore not adopting diffusion as an isolated special case. It is entering a broader transformation in generative modeling. That transformation favors models that can represent uncertainty explicitly, operate over several modalities, and provide more than one kind of output. Financial applications are especially demanding because they require realism, interpretability, and usefulness under noisy and unstable conditions. In that sense, finance is not merely a beneficiary of diffusion research. It is an excellent stress test for what diffusion models can really do.

The wrapper is also motivated by a more practical concern. Researchers, students, and practitioners often work best with compact thematic books rather than scattered papers. A wrapper creates continuity. It provides front matter, a table of contents, interpretive framing, and a conclusion that the individual papers do not provide on their own. It allows the reader to approach the collection as one research object instead of reconstructing its unity mentally. This is particularly valuable when the goal is not only to preserve the papers, but to use them pedagogically, share them as a coherent volume, or situate them within a wider body of work.

What, then, should the reader expect from this book? Not a general introduction to all of diffusion modeling. Not a complete treatment of every financial generative architecture. Not a production manual for institutional deployment. Instead, the reader should expect a focused and cumulative argument. The argument is that diffusion models, especially when paired with Transformer backbones and extended toward multimodal outputs, provide a compelling framework for financial generation, interpretation, and synthetic-data construction. That argument is made mathematically, algorithmically, and experimentally across the three chapters.

At the deepest level, the wrapper invites the reader to think differently about what a financial generative model ought to be. It ought not merely to reproduce surface distributions. It ought to

organize latent regimes, support conditional reasoning, expose internal consistency, and potentially align numerical and linguistic views of the same event. Diffusion models do not solve all of these challenges automatically. But they provide an unusually rich foundation on which such systems can be built.

That is why these three papers belong in one volume. Together they tell a story that none of them tells fully alone. The story is the emergence of a richer conception of financial generative modeling – one that begins with synthetic regime-aware sequences and extends toward multimodal financial intelligence. This preface is offered as an invitation to read the collection in exactly that spirit.

How to Use This Book

This book is best used as a compact research volume rather than as a passive cover-to-cover narrative. Its three chapters are connected, but they operate at different levels of abstraction and with different kinds of target objects. The first is primarily about synthetic return generation and regime interpretation. The second broadens the numerical state space toward structured financial feature generation. The third extends the problem into multimodality by coupling market sequences with technical-analysis text. The most effective way to read the book is therefore to treat it as a progression.

A useful reading order is the chapter order itself. Begin with the first chapter even if your main interest is multimodal generation. The reason is conceptual. The first chapter establishes the core logic of diffusion in finance: forward corruption, reverse denoising, regime mixtures, and the idea that a generative model can also be diagnostically useful. Without that foundation, later chapters may look like architectural elaborations without a clear conceptual center. Once the reader has internalized why diffusion matters for financial path structure, the later expansions into richer features and text become much easier to understand.

Each chapter can be read in four passes. In the first pass, read only the abstract, introduction, and conclusion. This gives you the research claim of the chapter before you encounter the equations. In the second pass, read the mathematical framework carefully and rewrite key definitions in your own notation if needed. The important thing is not memorization. It is to understand what is being modeled, what is observed, what is latent, what is being corrupted, and what the reverse process is trying to reconstruct. In the third pass, read the algorithm and pipeline sections and connect them back to the mathematics. Ask where the formulas become implementation, where approximation enters, and which design choices are principled versus practical. In the fourth pass, read the applications section and ask what becomes possible because the model exists in this particular form.

It is also useful to keep a running map of the recurring objects in the book. Several terms reappear across chapters: forward diffusion, reverse process, epsilon prediction, cosine schedule, DDIM sampling, masked diffusion, shared backbone, cross-modal fusion, latent representation, regime structure, and diagnostic use. These repetitions are a strength. They are the threads that tie the chapters together. One effective reading practice is to keep a small glossary or margin note for each

of these terms and update its meaning as the chapters become more sophisticated.

Readers interested mainly in finance should pay special attention to the distinction between realism and usefulness. These chapters do not treat a generative model as good merely because it can produce plausible-looking samples. Again and again, the practical question is whether the model supports something meaningful: backtesting, stress scenario design, imputation, regime interpretation, paired synthetic dataset generation, or cross-modal consistency filtering. This is a good lens through which to read the applications sections. Ask not only whether the generated outputs look correct, but why they would matter in a research or analytical workflow.

Readers more interested in machine learning than in finance should focus on the architectural progression. The first chapter shows how diffusion adapts to financial sequence generation. The second shows how the problem changes when one moves to a richer structured feature space. The third shows what must change again when language is introduced as a modality. Seen this way, the book becomes a case study in how a generative framework scales from unimodal to multimodal settings under domain-specific constraints.

If you have the associated notebooks or code, the strongest way to use the book is to pair reading with implementation. Read the conceptual and mathematical sections first, then run the notebook, then return to the chapter and inspect the outputs in light of the formal framework. This is particularly useful for understanding what the denoiser is actually learning, how DDIM accelerates generation, what masked diffusion does in practice, and why shared latent spaces matter for multimodal consistency. The book is written in a way that assumes such back-and-forth is valuable.

Another good strategy is comparative reading. After finishing all three chapters, return to the introductions only and compare them. Then compare the mathematical frameworks. Then compare the application sections. This makes the internal research progression much more visible. You will see how regime awareness in the first chapter becomes structured feature generation in the second, and how both of those become prerequisites for multimodal financial generation in the third.

Finally, this book should be used not as a closed doctrine but as a research springboard. The chapters deliberately leave open many questions: how to scale beyond synthetic data, how to handle richer language corpora, how to implement full multimodal reverse processes, how to evaluate generated financial data more rigorously, and how to extend these models toward richer conditioning or real-world deployment. The best use of the book is therefore not only to understand what has been done, but to sharpen your sense of what remains open.

Read it slowly, comparatively, and with an eye for progression. The unity of the book lies in the movement from financial diffusion as synthetic sequence generation to financial diffusion as multimodal generative intelligence.

Contents

Executive Summary **i**

Preface **v**

How to Use This Book **x**

1 Diffusion for Synthetic Financial Data **1**

2 From Returns to Rich Features **2**

2.1 Introduction: Transformers and Diffusion Models as Generative Engines in Finance . 3

2.2 Mathematical Framework 13

2.3 Pipeline and Algorithm 22

2.4 Applications in Finance 33

2.5 Challenges and Conclusions 42

3 Dual-Head Multimodal Diffusion Transformers **51**

3.1 Introduction: Complex Generative Models as Multimodal Financial Engines 52

3.2 Mathematical Framework 61

3.3 Pipeline and Algorithm 70

3.4 Applications in Finance 79

3.5 Challenges and Conclusions 87

Conclusion **99**

Chapter 1

Diffusion Models as Generative Engines for Synthetic Financial Data: Regime Detection, Process Identification, and Diagnostic Applications

Chapter 2

From Returns to Rich Features: Diffusion Transformers for Structured Financial Sequence Generation

Abstract. This chapter studies diffusion modeling in a richer financial sequence setting where the target is a structured OHLCV-style feature process rather than returns alone. The central claim is that diffusion Transformers can generate high-dimensional market-state sequences while preserving temporal coherence, cross-feature structure, and practical utility for applied quantitative workflows. The chapter formalizes the continuous diffusion framework for multivariate financial features, details the training and DDIM-based sampling pipeline, and clarifies the design choices required to move from toy one-dimensional generation to realistic structured financial state spaces. It then evaluates practical uses, including synthetic dataset generation, missing-data imputation, and scenario design under heterogeneous market conditions. The chapter concludes that diffusion models are not only statistically expressive in finance, but also operationally useful when built around structured feature representations.

2.1 Introduction: Transformers and Diffusion Models as Generative Engines in Finance

2.1.1 From Discriminative to Generative Modeling in Quantitative Finance

Quantitative finance has traditionally been dominated by **discriminative modeling**. In its most standard form, a model receives an input vector, tensor, or sequence and is trained to predict a target quantity such as a return, a direction, a volatility class, a default event, or an allocation decision. If $\mathcal{X}$ denotes the input space and $\mathcal{Y}$ the target space, a discriminative system seeks a map

$$f_\phi : \mathcal{X} \to \mathcal{Y}, \tag{2.1}$$

where ϕ denotes trainable parameters. This formulation has obvious usefulness in forecasting and classification. It is the language of most supervised machine learning in finance.

Yet many of the practically important problems in institutional finance are not fundamentally prediction problems. They are **distributional** problems. A portfolio manager does not merely want a point estimate of tomorrow's return; a risk officer wants plausible stressed scenarios, a quantitative researcher wants synthetic samples for robustness analysis, a validator wants to know whether a candidate model reproduces stylized facts, and a strategist may need a full ensemble of possible future paths rather than a single estimate. These objectives require a model of the data distribution itself rather than solely a conditional decision rule. Formally, one wants to approximate

$$p_\theta(x) \approx p_{\text{data}}(x), \tag{2.2}$$

where x may represent a univariate return path, a multivariate panel of asset returns, a volatility trajectory, a table of financial records, or a sequence of latent market states.

This distinction is not superficial. A forecast can be locally accurate while remaining globally useless for simulation. A classifier can identify a regime correctly on average while offering no ability

to generate realistic paths within that regime. In finance, this difference matters because many downstream tasks are inherently path-based. Backtesting, scenario construction, Value-at-Risk estimation under alternative sampling assumptions, and stress propagation analysis all depend on the quality of the generated distribution, not only on a prediction score.

Historically, synthetic financial data have often been obtained through parametric models, bootstrapping, block bootstrapping, or hidden-state simulation. These approaches remain valuable, but each imposes limitations. Parametric models may be too rigid. Bootstrapping may preserve too much of the historical sample and too little structural variation. Regime-switching econometric models capture latent-state changes elegantly, but their functional forms can still be restrictive when one wants to learn flexible nonlinear dependence across many assets and time steps. Deep generative models emerged in part because they offer a way to learn complex distributions directly from data.

Among these, **generative adversarial networks** (GANs) were particularly influential. Their objective can be written as

$$\min_{G} \max_{D} \left\{ \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))] \right\}, \tag{2.3}$$

where G is a generator and D is a discriminator. In finance, GANs have been explored for return-series generation and related tasks, including the Quant GAN framework, which demonstrated that adversarial methods can in fact generate financial time series with nontrivial realism [23]. However, the adversarial game is often unstable, and the kinds of failures that can arise are particularly costly in financial contexts. If the model collapses to a narrow subset of modes, then precisely the states one most wants to preserve—stress, transition, and tail events—may disappear from the synthetic sample.

Variational autoencoders and related latent-variable models provide a different route. Their central logic is to maximize a variational lower bound on the log-likelihood,

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)), \tag{2.4}$$

but these models face a different difficulty. When the decoder is highly expressive, the latent variables may become weakly informative, and the resulting representations can fail to carry the full structural burden of the data. In financial sequence modeling, where localized shocks, pathwise asymmetries, and heterogeneous dependence structures matter, such degradation of latent expressiveness is not merely a technical nuisance; it can materially reduce the realism and diagnostic usefulness of the generated output.

Autoregressive models provide a third alternative. They decompose a joint distribution as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{1:t-1}). \tag{2.5}$$

This factorization is elegant and statistically natural. It is also sequential. For long horizons, large cross-sections, or repeated scenario generation, the left-to-right bottleneck becomes computationally costly. More importantly, it imposes a generative logic in which each step depends on all previous generated steps, increasing latency and compounding error in applications where thousands of paths may need to be sampled.

Diffusion models alter this landscape by reframing generative modeling as a **progressive denoising problem**. Instead of learning to generate from scratch or to win an adversarial contest, the model learns to reverse a known noising process. This reparameterization has profound consequences. First, the training objective becomes stable and local: remove the noise that was added. Second, the forward corruption process has a clear probabilistic interpretation. Third, the reverse process can be approximated with a flexible neural architecture while preserving a mathematically tractable connection to the original data distribution. The basic diffusion idea introduced in modern form by [6] and built on the nonequilibrium thermodynamics view of [13] has therefore become one of the dominant paradigms in generative modeling.

In finance, the attraction is especially strong. Financial data are not merely noisy. They are distributional objects shaped by latent states, cross-sectional interactions, stochastic volatility, temporal clustering, and structural breaks. A methodology that explicitly models the transition from signal to noise and learns the inverse map is therefore conceptually aligned with the domain itself. The question is not whether uncertainty exists, but how to model it. Diffusion models answer by making uncertainty the center of the architecture rather than a residual nuisance.

This paper adopts that perspective and places diffusion in a specifically financial setting: multivariate synthetic return series generated from a regime-switching process, coupled with a discrete diffusion model for regime tokens. The aim is not only to show that diffusion can generate plausible financial samples, but to argue that it provides a coherent framework for regime-sensitive generative modeling and downstream diagnostics.

2.1.2 Limitations of Earlier Generative Paradigms

The appeal of diffusion models becomes clearer when contrasted carefully with the main generative paradigms that preceded them. Although GANs, VAEs, and autoregressive models each contributed essential ideas to the development of deep generative modeling, each also encounters structural limitations that become especially salient in finance.

The first limitation concerns **optimization stability**. In adversarial training, the generator and discriminator define a two-player game rather than a single-objective learning problem. Even when the theoretical objective is well posed, the empirical training dynamics may oscillate, diverge, or converge to degenerate solutions. In financial time series this is particularly problematic because the true data-generating environment is already heterogeneous and heavy-tailed. The adversarial game adds another layer of instability on top of a domain that is structurally difficult. The experience

with Quant GANs showed that adversarial methods can generate meaningful financial series, but it also highlighted the domain-specific importance of preserving rare and stress-like events [23]. A model that generates only typical calm periods may appear statistically smooth while being financially uninformative.

The second limitation is **mode collapse**. In images, mode collapse is visually obvious. In finance, it can be far more dangerous because it may remain hidden under standard summary statistics. A generator may match means, variances, and some autocorrelation structure while underrepresenting abrupt jumps, volatility bursts, or transition episodes. Since many risk-management tasks care most about low-frequency but high-impact states, missing those modes is not a cosmetic failure but a substantive one. In this sense, the challenge is not merely to produce realistic average samples, but to preserve the diversity of economically meaningful states.

The third limitation concerns **latent compression**. Variational models impose a bottleneck in which the data must be encoded into a latent variable z from which the full sample is reconstructed. This can be useful for representation learning, but it also risks over-compressing the information needed to preserve detailed pathwise structure. Financial sequences are often driven by combinations of local shocks, persistent regimes, and cross-asset interactions whose joint expression may not be well captured by a compact latent bottleneck. Even when the aggregate reconstruction error is low, the fine structure relevant for stress testing or strategy validation may be weakened.

The fourth limitation is **sequential generation**. Autoregressive models have an exact probabilistic factorization, but exactness does not imply practicality. Suppose one wishes to sample M synthetic multivariate return paths of length T across A assets. In a purely autoregressive framework, the effective sampling cost scales with the product of the path length and the number of conditional evaluations:

$$\mathrm{Cost}_{\mathrm{AR}} \propto M \times T \times C_\theta, \tag{2.6}$$

where C_θ denotes the cost of a forward pass of the model. For modest values this may be acceptable. For institutional scenario libraries or repeated Monte Carlo-like synthetic generation, it becomes operationally restrictive. Moreover, the conditional ordering itself becomes an architectural commitment: one must decide how to linearize the joint sequence, and different orderings may emphasize different dependencies.

A fifth limitation, often underappreciated in finance, concerns **structural non-stationarity**. Many generative models are implicitly optimized to approximate a single data distribution. But financial time series are frequently better described as arising from mixtures, state-dependent laws, or evolving structural regimes. If calm periods, crash periods, and transition phases all coexist within the training set, then a model that effectively averages them into one smooth law may generate samples that are statistically plausible yet economically incoherent. This is where the language of regimes becomes essential.

Diffusion models do not solve all these difficulties automatically, but they change the geometry of

the learning problem in a favorable way. Instead of asking the model to generate a full path in one shot or through a fragile minimax game, diffusion asks the model to learn a family of local denoising tasks indexed by noise level. The forward process is fixed and analytically specified. The reverse process is learned. The simplified training objective in the continuous Gaussian setting typically takes the form

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right], \tag{2.7}$$

which is markedly more stable than adversarial alternatives [6]. The model is not trying to outwit a discriminator or compress everything into a single bottleneck. It is learning to infer the noise that was added at a known stage of corruption.

This has two consequences relevant to finance. First, the learning problem becomes naturally multiscale. High-noise states encode coarse structural information; low-noise states encode fine local details. Financial data, which combine global regimes with local fluctuations, are therefore well matched to the progressive-denoising logic. Second, the learned denoising trajectories can themselves be interpreted. That is important because a good financial model is not only a generator but a diagnostic instrument. One wants to know whether the model distinguishes calm from stressed states, transition regimes from persistent regimes, and typical paths from unlikely ones.

The recent literature on diffusion in non-image domains reinforces this generalization. Diffusion methods have moved from vision to tabular modeling, discrete sequence generation, and language-like settings. TabDDPM demonstrated that diffusion can be adapted to tabular data [8]. Masked diffusion language models showed that diffusion-style ideas can also operate effectively in discrete token spaces [20]. This broadening of scope matters for finance because financial data are rarely purely continuous vectors. They often combine returns, labels, states, text, and categorical descriptors. A generative framework that can accommodate both continuous and discrete components is therefore especially attractive.

The present paper builds on that insight. The point is not to treat finance as another benchmark domain onto which diffusion is mechanically applied. The point is to recognize that finance is a domain where mixtures, uncertainty, non-stationarity, and latent structural states are central. Diffusion is compelling precisely because it offers a way to learn such distributions without collapsing them into a single adversarial or autoregressive narrative.

2.1.3 Why Diffusion Models Are Well Suited to Financial Data

The theoretical attractiveness of diffusion models in finance begins with the structure of the forward process. Let x_0 denote a financial sequence, such as a matrix of standardized returns. The forward process defines a Markov chain that gradually corrupts the data by adding Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\Big). \tag{2.8}$$

This process is not learned; it is specified. As t increases, the data lose more structure until the terminal state is approximately isotropic Gaussian noise. The model then learns the reverse transitions that approximately invert this corruption. The crucial conceptual move is that generative modeling is no longer framed as direct synthesis but as recovery from structured uncertainty.

That perspective is highly natural for finance. Financial data are observed through noise, partial information, latent state changes, and nonlinear interactions. A model built on progressive corruption and recovery therefore aligns more closely with the epistemic structure of the domain than one that assumes static representation suffices. One may say that diffusion models do not merely tolerate uncertainty; they operationalize it.

The second reason diffusion suits finance is its **multiscale character**. At high noise levels, only coarse information about the sample remains. At intermediate levels, broader structural aspects survive while fine local details are suppressed. At low noise levels, high-resolution path features are restored. This hierarchy maps well onto the layered nature of financial information. Market regimes, volatility states, trend persistence, and cross-asset correlation patterns often reveal themselves more clearly at coarser scales than do micro-level fluctuations. Conversely, local return asymmetries and short-lived shocks emerge at finer scales. A diffusion model learns across this full continuum.

A third advantage is that diffusion handles **mixture distributions** naturally. Suppose that the true return-generating mechanism is regime dependent. Then the observed data distribution is not a single simple law but a mixture over latent-state-conditioned components. In finance this may arise from switching volatility levels, changes in drift, liquidity crises, policy shocks, or transitions between market narratives. Diffusion does not require that these modes be explicitly enumerated within the continuous model. It is trained on samples from the mixture, and its denoising network therefore learns a representation of the mixture geometry itself. This is especially valuable when the latent regimes are economically meaningful but not perfectly observed.

A fourth advantage is the relative **stability of optimization**. The simplified denoising loss is a supervised learning problem over synthetic noise targets. That does not make training trivial, but it does make it more regular than adversarial alternatives. In practical financial research settings—especially notebook-based experimentation, prototype generation, and controlled synthetic laboratories—stability is not merely convenient. It determines whether a methodology can be iterated, debugged, and extended by researchers who are not exclusively focused on tuning a fragile generative game.

A fifth advantage is **sampling flexibility**. Diffusion models support both stochastic and deterministic samplers. DDPM-style sampling follows the full reverse chain. DDIM-style sampling provides a faster deterministic approximation [21]. This matters because finance rarely requires only one sample. Scenario analysis often requires many. The ability to trade off fidelity and speed by adjusting the number of reverse steps makes diffusion operationally attractive in settings where compute budgets are finite but repeated simulation remains necessary.

A sixth reason is the compatibility of diffusion with **inverse problems**. Inpainting, imputation, conditional completion, and local editing arise naturally within the diffusion formalism because one can partially corrupt a sample and then denoise subject to constraints. For finance, this opens doors to missing-value reconstruction, scenario editing around observed crises, and conditional stress generation from partial histories. A generative model is most useful when it can be conditioned and manipulated, not only sampled unconditionally. Diffusion provides that flexibility in a principled probabilistic framework.

The emerging literature also suggests that diffusion is not confined to images. The original DDPM formulation established the probabilistic foundations and empirical effectiveness of denoising diffusion [6]. Later work improved schedules, likelihood bounds, and sampling quality [14]. Score-based formulations connected diffusion to stochastic differential equations and continuous-time score matching [15]. More recently, diffusion architectures have been extended to tabular data and sequence-like discrete domains [8, 20]. These developments jointly support the claim that diffusion is not an image-specific technique but a general language for generative modeling under uncertainty.

This generality is exactly what finance requires. Financial datasets are heterogeneous. One encounters continuous returns, volatility curves, categorical regime labels, textual indicators, balance-sheet tables, and event sequences. A generative framework that already spans continuous and discrete domains is therefore a stronger candidate for future finance-specific systems than one tied to a single data modality.

Finally, diffusion models are attractive because they possess **diagnostic interpretability** at the level of trajectories. The intermediate states x_t and the latent representations used to predict denoising directions are not just internal numerical artifacts. They are structured approximations to how the model organizes uncertainty at different scales. In finance, where the distinction between calm, stressed, and transitional environments is central, such representations may encode regime information even when the training objective does not explicitly require classification. This opens the door to a different view of generative modeling: not only as sample production, but as a tool for process identification and structural assessment.

2.1.4 Why Transformers Replace U-Nets for Financial Sequences

Although diffusion models were popularized in image generation, the architectural assumptions that proved successful in vision do not transfer mechanically to finance. The original denoising diffusion probabilistic models were commonly implemented with **U-Net** backbones, a natural choice for images because images are spatially local objects. Convolutional hierarchies exploit translation invariance, local neighborhood structure, and multiscale spatial aggregation. Financial time series, however, are not images disguised as matrices. Their geometry is different, and so the backbone should be different.

The key issue is **dependency structure**. In financial sequences, dependence is often non-local.

A shock at one time may influence volatility several periods later. Cross-asset effects may appear with lags. Correlation structures may reorganize abruptly during crises. A localized convolutional receptive field can capture some of this only indirectly or after many layers. In contrast, the Transformer uses self-attention to model relationships across the entire sequence in a single block. If $H \in \mathbb{R}^{T \times d}$ denotes a sequence of hidden states, self-attention computes

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2.9}$$

where (Q, K, V) are learned projections of H. This mechanism allows each time index to attend to all others, making long-range temporal dependence first-class rather than indirect.

For finance, this is a decisive advantage. Return sequences often contain dependencies that are not spatially local and not even purely temporal in the univariate sense. A multivariate financial tensor contains both temporal and cross-sectional structure. One wants an architecture that can treat a period of turbulence not merely as a local pattern but as an event that reorganizes relationships throughout the sequence. Self-attention is better suited to that task than a purely convolutional stack.

The argument becomes stronger once one considers **diffusion-time conditioning**. In a denoising model, the network does not solve the same problem at each step. At high noise levels it must infer broad structural features; at low noise levels it must reconstruct fine details. The conditioning variable t is therefore central. A naïve way to incorporate it is to append a sinusoidal embedding to the input. A more powerful way is to use **Adaptive Layer Normalization (AdaLN)**, in which the time embedding produces scale and shift parameters that modulate hidden activations throughout the network. If e_t is a time embedding and h a hidden representation, one can write

$$(\gamma_t, \beta_t) = \mathrm{MLP}(e_t), \tag{2.10}$$

followed by

$$\mathrm{AdaLN}(h, e_t) = \gamma_t \odot \mathrm{LN}(h) + \beta_t. \tag{2.11}$$

This gives the denoiser a flexible mechanism for changing its effective computation as a function of diffusion time.

The importance of this design was demonstrated in Diffusion Transformers, which showed that transformer backbones with adaptive normalization can scale effectively in diffusion settings [16]. While that work was conducted in visual generation, its architectural lesson is broader: if diffusion is fundamentally a sequence of context-dependent denoising problems, then an architecture built around global context and explicit conditioning is highly attractive. Financial sequences satisfy that description naturally.

There is also a representational argument. A U-Net assumes a strong prior over local composition and hierarchical aggregation. That is sensible when neighboring pixels belong to coherent local

objects. In finance, adjacency in time does matter, but it is not the sole organizing principle. A calm period followed by a crisis may be more strongly linked to another crisis episode far away in time than to its immediate neighbors. Similarly, cross-asset co-movement can realign under stress in ways that are not localized. A transformer backbone is therefore less committed to a spatial analogy that does not truly hold.

From an implementation perspective, the transformer also integrates more naturally with mixed-modality extensions. If one later wants to combine continuous returns with discrete regime tokens, textual descriptors, or categorical event markers, token-based attention mechanisms provide a flexible bridge. In that sense, choosing a transformer for financial diffusion is not only about current fit, but also about future extensibility.

For the purposes of this paper, the denoiser is therefore modeled as an AdaLN-conditioned transformer rather than a U-Net. This is not an aesthetic choice. It is a domain-specific architectural claim: financial sequences are better described by global and regime-dependent temporal relations than by image-like spatial locality. The transformer is superior because it treats those non-local dependencies directly, and AdaLN is superior because it lets the network adapt its computation continuously across denoising time.

2.1.5 Regime-Switching as the Central Financial Problem

The central financial difficulty addressed in this paper is **non-stationarity with regime switching**. Financial time series are rarely well described by a single stationary distribution over long horizons. Drifts change, volatilities shift, correlations reorganize, and transitions between calm and stressed periods are recurrent features of market behavior. Hamilton's seminal contribution formalized this idea by modeling time series as evolving under latent regimes governed by a Markov process [**?**]. That insight remains foundational because it captures a key fact about financial data: observed returns may look noisy, but the noise is often structured by hidden states.

Let $R_t \in \{1, \ldots, K\}$ denote a latent regime variable. Then the data-generating law is conditional on the regime, while the unconditional observed distribution is a mixture induced by the regime dynamics. In the simplest regime-switching view, one may write

$$p(x_t) = \sum_{k=1}^{K} \pi_k \, p(x_t \mid R_t = k), \tag{2.12}$$

where π_k are regime weights, often linked to the stationary distribution of the underlying Markov chain. The key implication is that the observed process is not generated by one law but by several, together with a transition mechanism.

This matters enormously for generative modeling. A model trained as though the data came from a single homogeneous distribution may blur the distinctions among regimes. It may generate samples

that average calm and stressed states into implausible hybrids. It may underrepresent transitions, which are often more informative than the steady states themselves. In finance, a generator that cannot distinguish persistent calm from pending stress is of limited value.

The regime perspective changes the interpretation of diffusion. If the training sample is drawn from a regime-switching process, then the forward diffusion process corrupts observations that themselves arise from a latent mixture. The denoiser is therefore trained not on a single-law dataset, but on samples from the full regime mixture. As a result, its learned reverse map approximates a distribution over all admissible regime-conditioned paths. This is precisely why diffusion becomes useful not only as a generator, but as a diagnostic tool. Intermediate denoising states may encode information about which regime family best explains the corrupted observation.

This idea is especially important because regime labels are not always directly observed in real finance. Often one has only returns, spreads, or prices, and the regimes must be inferred. In such a setting, a useful generative model is one that organizes the path space in a way that reflects the latent-state structure even without explicit state supervision in every step of the continuous model. That is one reason to pair continuous diffusion over returns with a separate masked diffusion model over regime tokens. The two components reinforce one another conceptually: one learns the continuous path mixture, the other learns the discrete latent-state grammar.

In the synthetic laboratory motivating this paper, the continuous returns are generated by a regime-switching geometric Brownian motion, while the discrete regime sequence is separately modeled through masked diffusion. This setup is pedagogically powerful because it creates a controlled environment where the link between latent state and observed path is explicit. It also mirrors real finance in a stylized way: latent regimes exist, returns depend on them, and the observed distribution is their mixture.

The regime-switching perspective therefore serves as the conceptual bridge between econometric structure and deep generative modeling. It explains why standard one-distribution generative assumptions are inadequate, why diffusion is promising, and why a joint continuous-discrete architecture is meaningful rather than arbitrary.

2.1.6 Contributions of the Paper

Against this background, the paper makes three contributions.

First, it develops a formal framework for applying a **cosine-schedule denoising diffusion probabilistic model** with a **transformer denoiser** to multivariate financial return sequences. The formulation is tailored to data represented as tensors of length T across A assets, and it emphasizes the suitability of transformer attention and AdaLN time conditioning for finance. The mathematical framework shows how a regime-switching synthetic return process can be embedded into a continuous diffusion pipeline in a way that preserves both pathwise realism and architectural

tractability.

Second, the paper integrates a **masked diffusion model for discrete regime token sequences** into the same generative workflow. This extends the modeling scope beyond continuous returns and acknowledges that financial systems often involve latent categorical structure. Recent work on masked diffusion language models and discrete diffusion by density-ratio estimation shows that diffusion-style methods can operate effectively in discrete token spaces [20, 12]. In the present setting, this makes it possible to model both the continuous financial sequence and the discrete latent-state process in a unified conceptual pipeline.

Third, the paper proposes a **diagnostic interpretation of diffusion models in finance**. The model is not treated merely as a source of synthetic samples. Its latent representations and reverse trajectories are viewed as economically informative objects. Intermediate denoising states can be used for regime identification; partial noising and denoising can be used for local stress scenario generation; reverse-step constraints can be used for missing-data imputation; and discrepancies between generated and target distributions can be used as a diagnostic signal for misspecification of the assumed data-generating process.

These contributions lead to a broader conclusion. Diffusion models are not simply another black-box generator transplanted from image synthesis into finance. When combined with transformers and a regime-aware design, they become a principled framework for learning uncertainty-rich financial distributions, generating synthetic data under latent-state dependence, and probing the structure of the processes from which those data arise. That is the methodological claim developed in the remainder of the paper.

2.2 Mathematical Framework

2.2.1 The Forward Diffusion Process

The mathematical foundation of diffusion modeling begins with the specification of a **forward noising process**. Let $x_0 \in \mathbb{R}^{T\times A}$ denote a clean financial sequence, where T is the number of time steps and A is the number of assets. In the present context, x_0 may be interpreted as a normalized multivariate return tensor constructed from a synthetic regime-switching financial process. The forward diffusion mechanism defines a Markov chain

$$q(x_{1:T_{\text{diff}}} \mid x_0) = \prod_{t=1}^{T_{\text{diff}}} q(x_t \mid x_{t-1}), \tag{2.13}$$

in which each transition gradually perturbs the signal by injecting Gaussian noise. The individual transition kernel is given by

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\Big), \tag{2.14}$$

where $\beta_t \in (0,1)$ is the variance schedule and I is the identity operator on the vectorized sequence space. This construction follows the diffusion formalism developed in [13, 6], where the progressive corruption of data is designed so that the terminal distribution approaches isotropic Gaussian noise.

The central object controlling this process is the cumulative signal-retention coefficient. Define

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s. \tag{2.15}$$

The quantity $\bar{\alpha}_t$ measures the fraction of the original signal that survives after t forward steps. Under this notation, repeated application of the Gaussian transition kernel yields the closed-form marginal

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t) I\Big). \tag{2.16}$$

This identity is one of the key reasons diffusion models are tractable: the state at any diffusion time t can be sampled directly from the clean sample x_0 without simulating the entire chain.

Using the reparameterization trick, one obtains the familiar sampling equation

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{2.17}$$

This expression is fundamental because it converts forward noising into a supervised learning setup. If the model is shown x_t and knows t, then learning to infer ϵ is equivalent to learning how the original data are embedded inside a partially noised observation.

A crucial design choice is the schedule $\{\beta_t\}_{t=1}^{T_{\text{diff}}}$. Early diffusion implementations often used a linear schedule, but later work showed that a cosine schedule provides better signal preservation over the middle range of diffusion times [14]. Let

$$f(\tau) = \cos^2\!\left(\frac{\tau / T_{\text{diff}} + s}{1 + s} \cdot \frac{\pi}{2}\right), \tag{2.18}$$

where $s > 0$ is a small offset introduced for numerical stability. The cumulative coefficient is then defined as

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}. \tag{2.19}$$

From this, the stepwise schedule is recovered by

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}. \tag{2.20}$$

The practical importance of the cosine schedule in finance lies in its slower destruction of signal over the mid-range of t. Financial sequences often contain medium-scale structure—such as volatility persistence, cross-asset co-movement, and regime-linked dependence—that should not vanish too early during training. A schedule that preserves these components longer provides more informative intermediate training examples.

The degree of corruption at each step can also be expressed via the **signal-to-noise ratio**:

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}. \tag{2.21}$$

This quantity is useful both conceptually and practically. Conceptually, it measures the relative weight of original structure and injected noise at time t. Practically, it helps interpret the denoising difficulty across the diffusion chain. When $\mathrm{SNR}(t)$ is high, much of the original financial structure remains and denoising is relatively local. When $\mathrm{SNR}(t)$ is low, the task becomes global and distributional: the model must infer the coarse geometry of the data manifold from a highly corrupted observation.

In financial applications, the forward process has a particularly natural interpretation. A return sequence may be seen as progressively obscured by uncertainty until all economically meaningful structure is erased. The reverse process is then the task of reconstructing a plausible financial sequence from a state of increasing indeterminacy. This is not merely a computational device. It mirrors the way finance itself treats partial information, uncertainty, and latent structure. Because the forward process is explicitly defined and analytically tractable, the entire learning problem becomes one of approximating the inverse transformation from noisy observation back toward structured financial reality.

2.2.2 The Reverse Process and Epsilon Prediction

The purpose of training is to learn a reverse Markov chain that approximately inverts the forward diffusion process. The true reverse posterior is intractable in general, but diffusion modeling replaces it with a parameterized family of Gaussian reverse transitions:

$$p_\theta(x_{0:T_{\mathrm{diff}}}) = p(x_{T_{\mathrm{diff}}}) \prod_{t=1}^{T_{\mathrm{diff}}} p_\theta(x_{t-1} \mid x_t), \tag{2.22}$$

where the learned reverse kernel is

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\Big(\mu_\theta(x_t, t), \sigma_t^2 I\Big). \tag{2.23}$$

Here, $\mu_\theta(x_t, t)$ is the learned reverse mean and σ_t^2 is a prescribed or learned variance schedule. The training objective is to make this reverse chain reproduce the data distribution when started from Gaussian noise.

A central result of the DDPM framework is that one need not predict μ_θ directly. Instead, the model can predict the noise term ϵ appearing in the forward equation. From

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \tag{2.24}$$

one can solve for the clean sample:

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon}{\sqrt{\bar{\alpha}_t}}. \tag{2.25}$$

Replacing the unknown ϵ by a learned estimate $\epsilon_\theta(x_t, t)$ yields the model-based reconstruction

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{2.26}$$

This is the basis of **epsilon prediction**. The denoiser is trained to infer the Gaussian noise that corrupted the original sample rather than the sample itself. Once that prediction is available, the clean sample and the reverse mean can be reconstructed algebraically.

The reverse mean can be written in epsilon-parameterized form as

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right). \tag{2.27}$$

This parameterization is not merely a re-expression of the same model. It changes the geometry of the learning problem. The network is asked to recover the injected perturbation, which is statistically standardized and naturally aligned with the forward corruption process. This leads to a stable objective,

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right], \tag{2.28}$$

that was shown in [6] to work effectively in practice. Because ϵ is sampled directly from a standard Gaussian, the target distribution is well behaved even when the original data distribution is complicated.

The interpretation of this loss in finance is important. The network is not being trained to memorize raw return paths. It is being trained to infer how much random perturbation has been applied to a structured financial object at a given diffusion step. This forces the model to learn the shape of the data distribution across many levels of corruption. It is therefore learning a denoising field across the entire path space rather than a direct one-step predictor.

The reverse process can also be connected to score-based modeling. If one views the denoiser as approximating the score of the perturbed data distribution, then the reverse chain approximates a time-reversed stochastic process whose drift is driven by the gradient of the log-density [15]. In continuous time, this leads to a stochastic differential equation interpretation in which denoising corresponds to following a learned vector field back toward the data manifold. In the discrete DDPM setting, epsilon prediction can be regarded as the operational form of this score recovery.

For financial sequences, this interpretation matters because the score field need not be simple. It reflects the geometry of a distribution shaped by regime mixtures, volatility clustering, and cross-sectional dependence. The denoiser is therefore learning not merely a smoothing operator, but an approximation to the local geometry of the entire financial data law. This is why the reverse

dynamics can later be used diagnostically: they encode more than sample generation alone.

2.2.3 Transformer Denoiser with AdaLN Conditioning

The denoiser $\epsilon_\theta(x_t, t)$ must solve a challenging task. It receives a partially corrupted multivariate financial sequence and a diffusion-time index and must infer the noise component that produced the corruption. In image diffusion, this role was originally played by a U-Net. In the present setting, the denoiser is modeled as a **Transformer** because financial sequences are characterized by long-range temporal dependencies and cross-asset interactions rather than spatial locality.

Let $x_t \in \mathbb{R}^{T \times A}$ be linearly projected into a sequence of hidden vectors

$$H^{(0)} = \mathrm{Proj}(x_t) + P, \tag{2.29}$$

where P is a positional encoding matrix. The denoiser then applies a stack of self-attention and feed-forward blocks. For hidden state $H^{(\ell)}$, the self-attention sublayer computes

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2.30}$$

where

$$Q = H^{(\ell)} W_Q, \qquad K = H^{(\ell)} W_K, \qquad V = H^{(\ell)} W_V. \tag{2.31}$$

This allows each time step to aggregate information from all other time steps. In a financial setting, that is essential because a local return pattern may be informative only in relation to distant volatility bursts, regime shifts, or cross-sectional alignment across assets.

The second essential ingredient is diffusion-time conditioning. Each reverse step corresponds to a different denoising task, so the network must adapt as a function of t. This is done through a sinusoidal time embedding e_t, analogous to positional encoding but indexed by diffusion time rather than by sequence position. The embedding is then passed through a multilayer perceptron to generate an adaptive scale and shift:

$$(\gamma_t, \beta_t) = \mathrm{MLP}(e_t). \tag{2.32}$$

These parameters are injected through **Adaptive Layer Normalization**:

$$\mathrm{AdaLN}(h, e_t) = \gamma_t \odot \mathrm{LN}(h) + \beta_t. \tag{2.33}$$

The role of AdaLN is profound. Instead of merely appending time information as an additional input, it modulates the internal representation of the network at every layer. When the diffusion step corresponds to a high-noise regime, the hidden activations are transformed differently than when the input is almost clean. This lets the model specialize its computation continuously across

denoising depth.

A generic residual block of the denoiser can therefore be written as

$$H^{(\ell+\frac{1}{2})} = H^{(\ell)} + \mathrm{MHA}\Big(\mathrm{AdaLN}(H^{(\ell)}, e_t)\Big), \tag{2.34}$$

followed by

$$H^{(\ell+1)} = H^{(\ell+\frac{1}{2})} + \mathrm{FFN}\Big(\mathrm{AdaLN}(H^{(\ell+\frac{1}{2})}, e_t)\Big). \tag{2.35}$$

After several such layers, the final hidden representation is projected back into the original signal space to obtain the estimated noise:

$$\epsilon_\theta(x_t, t) = \mathrm{OutProj}(H^{(L)}). \tag{2.36}$$

The importance of this design in finance is twofold. First, attention allows the model to represent non-local dependencies directly. A volatility spike at one point in the sequence may alter the relevance of observations many steps away. Second, AdaLN makes the network sensitive to the semantic meaning of diffusion time. At low noise levels, the model can focus on fine-scale return reconstruction. At high noise levels, it can emphasize broader structural and regime-level organization.

This architecture also supports the broader claim of the paper: denoising is not merely a low-level reconstruction task but a structured inference problem. The hidden states of the transformer become latent representations of partially denoised financial structure. Those representations can later be inspected for regime identification and related diagnostics. In that sense, the denoiser is not just a function approximator; it is a learned representation system for uncertainty across financial path space.

2.2.4 Regime-Switching Data-Generating Process

The continuous diffusion model is trained on data generated from a **regime-switching geometric Brownian motion** (GBM). This choice is not incidental. It provides a controlled synthetic setting in which the observed return distribution is explicitly generated by a latent Markov structure. The purpose is to study how diffusion models behave when the true data law is a regime mixture rather than a single stationary Gaussian process.

Let $\{R_t\}_{t\geq 0}$ be a finite-state Markov chain taking values in $\{1, \ldots, K\}$. Its transition matrix is

$$P_{ij} = \Pr(R_t = j \mid R_{t-1} = i), \tag{2.37}$$

with each row summing to one:

$$\sum_{j=1}^{K} P_{ij} = 1. \tag{2.38}$$

The latent regime controls the parameters of the return process. For each regime k, the one-step return over interval Δt follows

$$r_t \mid R_t = k \sim \mathcal{N}\left(\left(\mu_k - \frac{\sigma_k^2}{2}\right)\Delta t, \sigma_k^2 \Delta t\right). \tag{2.39}$$

This is the discrete-time return law implied by a GBM with regime-dependent drift μ_k and volatility σ_k.

If the Markov chain admits a stationary distribution $\pi = (\pi_1, \ldots, \pi_K)$ satisfying

$$\pi^\top P = \pi^\top, \qquad \sum_{k=1}^{K} \pi_k = 1, \tag{2.40}$$

then the unconditional one-step return distribution is a finite mixture:

$$p(r_t) = \sum_{k=1}^{K} \pi_k \, p(r_t \mid R_t = k). \tag{2.41}$$

Since each component is Gaussian, this becomes

$$p(r_t) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}\left(r_t; \left(\mu_k - \frac{\sigma_k^2}{2}\right)\Delta t, \sigma_k^2 \Delta t\right). \tag{2.42}$$

This mixture interpretation is central. Even if each regime individually follows a simple law, the aggregate distribution may exhibit heavier tails, clustered volatility, and state-dependent asymmetry because of the switching mechanism.

For pathwise data, the structure is richer still. A full return path is generated jointly with a latent regime path

$$R_{1:T} = (R_1, \ldots, R_T), \tag{2.43}$$

and the observed path density is obtained by summing over all possible latent-state trajectories:

$$p(x_{1:T}) = \sum_{R_{1:T}} p(x_{1:T} \mid R_{1:T}) \, p(R_{1:T}). \tag{2.44}$$

Thus, the continuous financial data presented to the diffusion model are sampled from a path mixture rather than a single homogeneous process.

This has an important implication for learning. The denoiser is trained on corrupted samples x_t obtained from draws x_0 of this regime-mixture law. Therefore, the learned reverse process approximates the geometry of the mixture itself. It does not simply learn one Gaussian-like attractor. It learns a denoising field shaped by all admissible regimes and the transitions among them. This is why the model can later be interpreted as carrying regime information even when the continuous diffusion objective does not explicitly classify regimes.

One may formalize this intuition as follows. Let $p_{\text{mix}}(x)$ denote the true mixture distribution over return paths induced by the latent Markov chain. Then the score field relevant to denoising is

$$\nabla_x \log p_{\text{mix}}(x), \tag{2.45}$$

which differs in general from the score of any individual regime component. The denoiser trained on the full dataset therefore approximates a score structure that already integrates over regime uncertainty. At intermediate noise levels, this structure may reflect which regime components remain locally plausible. This observation underlies the diagnostic use of latent denoising states in later sections.

The regime-switching GBM is therefore not merely a convenient simulator. It is the structural bridge between classical financial econometrics and modern generative diffusion. It provides a controlled environment in which the mixture nature of financial path distributions can be studied explicitly, and it clarifies why diffusion is suited to learning more than a single stationary return law.

2.2.5 Masked Diffusion for Discrete Regime Tokens

The continuous diffusion model captures the return process, but the latent regime sequence itself is discrete. To model it directly, one uses a **masked diffusion process** over tokens. Let

$$s = (R_1, \ldots, R_T) \in \{0, \ldots, K-1\}^T \tag{2.46}$$

denote the discrete regime sequence associated with a financial path. Since the state space is categorical, Gaussian diffusion is not the natural formalism. Instead, one constructs a forward process that progressively replaces tokens by a mask symbol.

Let [MASK] denote a special masking token. At masking step $\tau \in \{1, \ldots, T_{\text{mask}}\}$, each token is independently replaced by [MASK] with probability

$$m_\tau = \frac{\tau}{T_{\text{mask}}}. \tag{2.47}$$

If $\tilde{s}_\tau$ denotes the masked sequence and M_τ the set of masked positions, then the objective is to predict the original token values on the masked sites. The training loss is

$$\mathcal{L}_{\text{mask}} = -\mathbb{E}_{s,\tau} \sum_{j \in M_\tau} \log p_\phi(s_j \mid \tilde{s}_\tau, \tau). \tag{2.48}$$

This is a cross-entropy objective over masked positions. It parallels the diffusion philosophy in a discrete setting: information is gradually removed, and the model learns to reconstruct what was hidden.

The reverse process proceeds through iterative unmasking. At each reverse step, the model predicts

a categorical distribution over the masked sites and fills in a subset of them. One simple schedule is to unmask positions with probability

$$u_\tau = \frac{1}{T_{\text{mask}} - \tau + 1}, \tag{2.49}$$

so that the expected number of remaining masked tokens decreases smoothly toward zero. The final output is a fully generated regime sequence.

This masked-diffusion construction is important for two reasons. First, it makes explicit that the latent-state process can itself be modeled generatively rather than treated only as hidden metadata. Second, it aligns the continuous and discrete parts of the overall system. The return tensor is modeled through Gaussian noising and denoising; the regime sequence is modeled through masking and unmasking. Both are instances of the same conceptual idea: structured information is progressively destroyed and then reconstructed by a learned reverse operator.

The connection to recent discrete diffusion literature is immediate. Diffusion-style text and token models have shown that iterative masking can serve as a practical alternative to purely autoregressive generation [9, 20, 12]. In the present financial setting, the vocabulary is small, but the conceptual gain is large. It allows the latent regime sequence to be modeled as an object in its own right and later aligned with the continuous return-generation pipeline.

2.2.6 DDIM Sampling

Training defines the reverse process, but practical generation requires an efficient sampler. The basic DDPM sampler follows the full reverse chain through all diffusion steps. Although statistically faithful, this can be computationally costly when T_{diff} is large. **Denoising Diffusion Implicit Models** (DDIM) provide a deterministic alternative that preserves sample quality while greatly reducing the number of required denoising evaluations [21].

Starting from a noisy state x_t, the denoiser predicts $\epsilon_\theta(x_t, t)$ and then reconstructs the clean estimate

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\, \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{2.50}$$

The deterministic DDIM update is then

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\, \epsilon_\theta(x_t, t). \tag{2.51}$$

Unlike the stochastic DDPM update, this step does not add fresh Gaussian randomness. It traces a deterministic trajectory through latent space, controlled entirely by the denoiser.

This formulation has several advantages in finance. First, it sharply lowers the cost of scenario generation. If the full diffusion chain has 1000 steps and DDIM sampling uses 50, then the nominal

number of denoiser evaluations is reduced by a factor of

$$\frac{50}{1000} = 0.05. \tag{2.52}$$

When generating many synthetic financial paths, this reduction is operationally material.

Second, DDIM supports a useful continuum between fidelity and efficiency. More steps generally improve approximation quality, while fewer steps improve throughput. In a research notebook or model-development environment, a 50-step sampler is often attractive because it allows repeated experimentation without the cost of a full reverse chain.

Third, deterministic sampling is especially convenient for diagnostic and conditional tasks. If one partially noises an observed path to some intermediate step and then denoises deterministically, the resulting output can be interpreted as a structured transformation of that seed path under the learned model. This is particularly useful for stress testing, local scenario generation, and inpainting, where one wants reproducible behavior rather than purely stochastic variation.

The DDIM formulation therefore closes the mathematical framework by connecting the learned denoiser to a practical synthetic-data engine. The forward process defines structured corruption, the reverse process defines learned denoising, the transformer parameterizes the denoiser, the latent-state process is modeled through masking, and DDIM provides the tractable mechanism by which the model is actually used to generate financial samples.

2.3 Pipeline and Algorithm

2.3.1 Overview of the End-to-End Pipeline

The mathematical framework developed in the previous section specifies the probabilistic objects that define the model. The present section turns those objects into an executable pipeline. The central goal is to move from a synthetic financial environment with explicit regime structure to a trained generative system that can produce continuous return paths, discrete regime sequences, and latent representations useful for diagnosis. In this sense, the pipeline is not only a training recipe; it is the operational realization of the paper's broader thesis that diffusion models can serve simultaneously as **generative engines** and **diagnostic instruments** in finance.

The full workflow begins with the construction of a synthetic dataset. Each sample consists of two aligned objects. The first is a multivariate return tensor

$$X \in \mathbb{R}^{N \times T \times A}, \tag{2.53}$$

where N is the number of samples, T is the sequence length, and A is the number of assets. The

second is a regime-token tensor

$$S \in \{0, \ldots, K-1\}^{N \times T}, \tag{2.54}$$

where each entry records the latent regime associated with a time point in a return path. The data are generated from a regime-switching geometric Brownian motion, so the continuous returns and discrete regime states are structurally coupled from the start.

Before training, the return tensor is normalized. This step is important for both numerical stability and geometric regularity. Let μ_X and σ_X denote empirical mean and standard deviation statistics computed over the training set. A standardized sample is obtained as

$$\tilde{x}_0 = \frac{x_0 - \mu_X}{\sigma_X + \varepsilon_{\text{norm}}}, \tag{2.55}$$

where $\varepsilon_{\text{norm}} > 0$ is a small constant preventing division by zero. Standardization helps ensure that the Gaussian assumptions of the forward diffusion process are applied to a scale-compatible input space. Since the forward kernel injects isotropic noise, excessively uneven raw magnitudes across assets or time steps could distort the effective corruption schedule.

The pipeline then splits into two tightly related learning problems. The first is **continuous diffusion** over return tensors. A diffusion step t is sampled uniformly from $\{1, \ldots, T_{\text{diff}}\}$, Gaussian noise is drawn, and the noised state x_t is computed using the forward reparameterization. The AdaLN-conditioned Transformer denoiser receives (x_t, t) and predicts the injected noise. The mismatch between predicted and true noise defines the feature loss. The second problem is **masked diffusion** over regime tokens. A masking step t_{text} is sampled, each regime token is masked with probability $t_{\text{text}}/T_{\text{mask}}$, and a separate Transformer predicts the original tokens on the masked positions. The mismatch defines the token loss.

These two objectives are optimized jointly. If L_{feat} denotes the continuous diffusion loss and L_{sent} the discrete token loss, the combined objective is

$$L = L_{\text{feat}} + \lambda_{\text{sent}} L_{\text{sent}}, \tag{2.56}$$

where λ_{sent} controls the relative influence of the token component. This formulation keeps the return generator and regime generator conceptually linked while allowing each to retain an architecture appropriate to its own domain.

After training, the system can be used in several ways. For unconditional generation, one begins from Gaussian noise and runs a deterministic DDIM reverse process to obtain a synthetic return path. In parallel or subsequently, one begins from a fully masked regime-token sequence and iteratively unmasks it using the discrete diffusion model. For conditional tasks, one may partially noise a real path and denoise from that intermediate point. For diagnostic tasks, one may inspect latent states, intermediate denoiser representations, or discrepancies between real and generated statistics.

It is worth emphasizing that this pipeline is designed to be modular. Each component has a

clearly identifiable probabilistic role. The schedule precomputation belongs to the forward diffusion geometry. The continuous denoiser belongs to reverse estimation in Euclidean sequence space. The discrete regime model belongs to masked token reconstruction. The optimizer and scheduler belong to numerical implementation. Because the stages are explicit, the system is more interpretable and easier to audit than a monolithic black-box generator whose internal responsibilities are opaque.

A second important feature is reproducibility. The pipeline can be run with fixed random seeds for the synthetic regime generator, diffusion noise sampling, and optimizer initialization. This matters because in a financial research context, a useful generative system must be not only expressive but also inspectable. One wants to know whether a given output difference is due to model architecture, data generation, optimization randomness, or sampling configuration. A pipeline whose stages are clearly delineated is therefore preferable to one in which all uncertainty is entangled.

The end-to-end view can thus be summarized as follows. First, generate regime-aware synthetic financial data. Second, standardize the continuous component. Third, train a continuous diffusion model on return paths. Fourth, train a discrete masked diffusion model on regime tokens within the same optimization loop. Fifth, sample with DDIM for returns and iterative unmasking for regimes. Sixth, evaluate the results not only as generated samples but as evidence about regime structure and process adequacy. The remainder of this section formalizes that logic into algorithms and architectural detail.

2.3.2 Algorithm 1: Training Pipeline

The training algorithm is the computational heart of the framework. It translates the theoretical objectives of continuous denoising and discrete masked reconstruction into a single optimization loop. Although the procedure is conceptually straightforward, its organization is important because diffusion models depend critically on precomputed schedules, consistent timestep sampling, and a stable optimization regime.

The inputs are the normalized return tensor X and the regime-token tensor S. For each minibatch, the algorithm first draws a set of continuous diffusion steps. Let

$$t \sim \text{Uniform}\{1, \ldots, T_{\text{diff}}\}. \tag{2.57}$$

This uniform timestep sampling ensures that the denoiser sees the entire range of corruption levels throughout training. Because each step corresponds to a different signal-to-noise configuration, the denoiser must learn to operate across the full continuum rather than specialize only to easy low-noise examples.

Conditional on t, Gaussian noise is drawn:

$$\epsilon \sim \mathcal{N}(0, I). \tag{2.58}$$

The forward corruption step is then implemented through

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon. \tag{2.59}$$

The denoiser receives x_t together with the timestep embedding and returns a prediction

$$\hat{\epsilon} = \epsilon_\theta(x_t, t). \tag{2.60}$$

The continuous loss is the mean squared error

$$L_{\text{feat}} = \|\hat{\epsilon} - \epsilon\|_2^2\,, \tag{2.61}$$

or its minibatch average. This is the direct implementation of the DDPM epsilon-prediction objective.

In parallel, the algorithm samples a masking time for the discrete regime model:

$$t_{\text{text}} \sim \text{Uniform}\{1, \ldots, T_{\text{mask}}\}. \tag{2.62}$$

Each token in the regime sequence is independently masked with probability

$$m(t_{\text{text}}) = \frac{t_{\text{text}}}{T_{\text{mask}}}. \tag{2.63}$$

Let M denote the set of masked positions and $\tilde{s}$ the resulting partially masked sequence. The masked-token Transformer then produces logits over regime classes at each position. The discrete loss is

$$L_{\text{sent}} = -\sum_{j \in M} \log p_\phi(s_j \mid \tilde{s}, t_{\text{text}}). \tag{2.64}$$

This objective is a cross-entropy loss restricted to the masked positions, ensuring that the model is trained as a denoiser of missing symbolic information rather than as a trivial identity map.

The full parameter update is based on the joint loss

$$L = L_{\text{feat}} + \lambda_{\text{sent}} L_{\text{sent}}. \tag{2.65}$$

The parameter vectors of the continuous and discrete models may be distinct or partially coordinated, but the optimization loop treats them as parts of one regime-aware generative system. This is conceptually useful because the continuous and discrete processes arise from the same synthetic financial environment even if they live in different representational spaces.

The schedule buffers required for the continuous diffusion model are precomputed before training begins. These typically include

$$\{\beta_t, \alpha_t, \bar{\alpha}_t, \sqrt{\bar{\alpha}_t}, \sqrt{1 - \bar{\alpha}_t}, \sigma_t^2\}_{t=1}^{T_{\text{diff}}}. \tag{2.66}$$

Precomputing them avoids repeated recomputation and ensures exact consistency across batches. In diffusion implementations, these buffers are not merely optimization conveniences; they define the geometry of the learning problem.

The optimization step itself uses AdamW with cosine learning-rate scheduling and gradient clipping. If g_u denotes the gradient at update step u, clipping enforces

$$g_u \leftarrow g_u \cdot \min\left(1, \frac{c}{\|g_u\|_2}\right), \tag{2.67}$$

for some clipping threshold $c > 0$. This is important because diffusion losses can fluctuate across timesteps, particularly when a minibatch contains a concentration of difficult high-noise examples or highly volatile synthetic paths. Gradient clipping helps maintain stable training dynamics without changing the probabilistic interpretation of the model.

The training procedure can therefore be summarized as repeated stochastic estimation of two denoising tasks: one continuous, one discrete. The former learns the reverse geometry of multivariate return space; the latter learns the reconstruction grammar of latent regimes. Their combination is what makes the pipeline regime-aware in both path and token space.

[h] Training Pipeline [1] **Input:** normalized return tensor $X \in \mathbb{R}^{N\times T\times A}$, regime token tensor $S \in \{0,\dots,K-1\}^{N\times T}$ Precompute cosine schedule buffers $\{\beta_t, \alpha_t, \bar{\alpha}_t, \sqrt{\bar{\alpha}_t}, \sqrt{1-\bar{\alpha}_t}, \sigma_t^2\}_{t=1}^{T_{\text{diff}}}$ training not converged Sample minibatch (x_0, s) from (X, S) Sample $t \sim \text{Uniform}\{1,\dots,T_{\text{diff}}\}$ Sample $\epsilon \sim \mathcal{N}(0, I)$ Compute $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ Compute $\hat{\epsilon} = \epsilon_\theta(x_t, t)$ Compute $L_{\text{feat}} = \|\hat{\epsilon} - \epsilon\|_2^2$ Sample $t_{\text{text}} \sim \text{Uniform}\{1,\dots,T_{\text{mask}}\}$ Mask each token in s independently with probability $t_{\text{text}}/T_{\text{mask}}$ Compute token logits from the masked diffusion Transformer Compute $L_{\text{sent}} = \text{CE}(\text{logits}[M], s[M])$ Set total loss $L = L_{\text{feat}} + \lambda_{\text{sent}} L_{\text{sent}}$ Update parameters with AdamW, cosine learning-rate decay, and gradient clipping

A final point deserves emphasis. Training in this setting is not merely a matter of minimizing loss values. Because the data are synthetic and regime structured, the quality of the learned model can be interpreted relative to a known latent system. This creates a rare and useful environment in which generative training, representation learning, and model diagnosis can be studied together. The training algorithm is therefore central not only computationally, but epistemically: it is the mechanism by which the model internalizes the mixture geometry of regime-dependent financial data.

2.3.3 Algorithm 2: DDIM Sampling and Token Unmasking

Once training is complete, the model becomes a generative engine. The continuous component generates synthetic return paths through reverse diffusion; the discrete component generates regime sequences through iterative unmasking. The sampling pipeline is the point at which the model's learned representation of the financial data distribution becomes observable.

For the continuous model, the starting point is a Gaussian latent state

$$x_{T_{\text{diff}}} \sim \mathcal{N}(0, I). \tag{2.68}$$

If one uses a standard DDPM sampler, the reverse process follows all steps from T_{diff} down to 1, injecting stochasticity at each stage. In the present framework, however, sampling is performed with DDIM in order to reduce computational cost while preserving a meaningful approximation to the learned reverse path. At each chosen reverse timestep t, the model predicts

$$\hat{\epsilon}_t = \epsilon_\theta(x_t, t), \tag{2.69}$$

from which the clean estimate is reconstructed as

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\,\hat{\epsilon}_t}{\sqrt{\bar{\alpha}_t}}. \tag{2.70}$$

The deterministic update is then

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\,\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\,\hat{\epsilon}_t. \tag{2.71}$$

Repeating this over a 50-step schedule rather than the full 1000-step chain yields a synthetic return path at far lower computational cost.

The reason this matters in finance is straightforward. Practical use rarely involves a single synthetic sample. One often wants many scenarios, many simulated histories, or repeated stress variants of a seed path. If the cost of one denoiser evaluation is C_θ, then a full-step sampler costs on the order of

$$\text{Cost}_{1000} \approx 1000\, C_\theta, \tag{2.72}$$

whereas a 50-step DDIM sampler costs

$$\text{Cost}_{50} \approx 50\, C_\theta. \tag{2.73}$$

The ratio

$$\frac{\text{Cost}_{50}}{\text{Cost}_{1000}} = 0.05 \tag{2.74}$$

makes clear why deterministic reduced-step sampling is attractive for synthetic finance workflows.

For the discrete regime model, the starting point is different. One initializes a fully masked sequence

$$\tilde{s}^{(0)} = ([\text{MASK}], \ldots, [\text{MASK}]). \tag{2.75}$$

At each reverse iteration τ, the masked-token Transformer produces a categorical distribution over possible regime labels at each masked location. A subset of positions is then selected for unmasking according to a prechosen schedule. If U_τ denotes the set of positions unmasked at step τ, then for

each $j \in U_\tau$ one samples or selects

$$\hat{s}_j \sim p_\phi(s_j \mid \tilde{s}^{(\tau-1)}, \tau). \tag{2.76}$$

The partially reconstructed sequence is then fed back into the next step. After T_{mask} iterations, no masked tokens remain.

The combination of DDIM sampling and iterative unmasking is conceptually elegant because it mirrors the dual structure of the data. Continuous returns are generated by reversing Gaussian corruption; discrete regimes are generated by reversing masking corruption. The two objects may be sampled independently after training, but they belong to the same regime-conditioned synthetic world.

[h] DDIM Sampling with Iterative Regime Unmasking [1] **Input:** trained continuous denoiser ϵ_θ, trained token model p_ϕ, DDIM timestep set $\mathcal{T}$ Sample $x_{T_{\text{diff}}} \sim \mathcal{N}(0, I)$ $t \in \mathcal{T}$ in descending order Compute $\hat{\epsilon}_t = \epsilon_\theta(x_t, t)$ Compute $\hat{x}_0 = \left(x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_t\right) / \sqrt{\bar{\alpha}_t}$ Update $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\hat{\epsilon}_t$ Initialize fully masked regime sequence $\tilde{s}^{(0)}$ $\tau = 1$ to T_{mask} Predict token logits from $p_\phi(\cdot \mid \tilde{s}^{(\tau-1)}, \tau)$ Select positions to unmask Fill selected positions with sampled or argmax regime tokens **Return:** synthetic return path x_0 and regime-token sequence $\hat{s}$

The sampling algorithm is also the gateway to diagnostic applications. If one starts not from pure Gaussian noise but from a partially noised real path, the reverse trajectory becomes a controlled neighborhood generator around an observed event. If one inspects hidden denoiser states at intermediate steps, sampling becomes a source of regime-sensitive embeddings. If one repeatedly samples from the same model and compares generated to target statistics, sampling becomes a tool for process validation. Thus, Algorithm 2 is not merely a procedure for producing synthetic outputs; it is the operational expression of the paper's claim that diffusion can be both generative and diagnostic.

2.3.4 Architectural Details

The continuous denoiser is implemented as an **AdaLN Transformer** with model width $d = 128$, $L = 6$ Transformer blocks, and $H = 8$ attention heads. This configuration is intentionally moderate: expressive enough to capture non-local temporal and cross-sectional structure, yet compact enough to remain practical in a notebook-based experimental setting. The input return tensor $x_t \in \mathbb{R}^{T \times A}$ is first projected into model space:

$$H^{(0)} = \text{Proj}(x_t) + P, \tag{2.77}$$

where P denotes positional embeddings. Each Transformer block then applies multi-head self-attention followed by a feed-forward network, with both sublayers conditioned on diffusion time through AdaLN.

For head $h \in \{1, \ldots, H\}$, the attention computation is

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d/H}}\right) V_h, \tag{2.78}$$

and the multi-head output is

$$\text{MHA}(H) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H) W_O. \tag{2.79}$$

The use of 8 heads allows the model to represent several distinct relational patterns simultaneously. In a financial sequence, one head may focus on volatility clustering, another on cross-asset alignment, another on recovery dynamics after a shock, and so on. These are not preassigned interpretations, but the architecture has the capacity to represent such decompositions.

Diffusion-time information is encoded through a sinusoidal embedding

$$e_t = \text{SinusoidalEmb}(t), \tag{2.80}$$

which is passed through an MLP to generate adaptive normalization parameters for each block. If block ℓ receives hidden state $H^{(\ell)}$, then

$$(\gamma_t^{(\ell)}, \beta_t^{(\ell)}) = \text{MLP}^{(\ell)}(e_t), \tag{2.81}$$

and the normalized activations become

$$\widetilde{H}^{(\ell)} = \gamma_t^{(\ell)} \odot \text{LN}(H^{(\ell)}) + \beta_t^{(\ell)}. \tag{2.82}$$

This mechanism is what turns a generic Transformer into a diffusion-conditioned Transformer. The same return tensor corrupted at different timesteps should not be treated as the same kind of object, and AdaLN provides the necessary adaptive modulation.

The discrete regime model, referred to in the notebook logic as a separate **SentimentDiffTransformer** or regime-token Transformer, is architecturally analogous but operates in categorical token space. Its input is a sequence of regime-token embeddings with positional encodings and a masking-time embedding. Rather than predicting Gaussian noise, it produces logits over the K regime classes at each token position:

$$\text{logits}_j \in \mathbb{R}^K, \qquad p_\phi(s_j = k \mid \tilde{s}, t_{\text{text}}) = \frac{\exp(\text{logits}_{j,k})}{\sum_{k'=1}^{K} \exp(\text{logits}_{j,k'})}. \tag{2.83}$$

This categorical output layer makes the discrete model well matched to masked diffusion over a small latent-state vocabulary.

The architectural separation between continuous and discrete diffusion is deliberate. The return

tensor and the regime sequence live in fundamentally different domains. One is Euclidean and naturally paired with Gaussian corruption. The other is categorical and naturally paired with masking corruption. Forcing them into a single undifferentiated network would complicate both training and interpretation. By contrast, the dual-Transformer design is modular and easier to diagnose.

The chosen architecture also reflects an important practical principle: the goal is not to create the largest possible model, but the clearest possible mapping between probabilistic structure and computational structure. Model width 128, depth 6, and head count 8 are large enough to permit expressive denoising, yet small enough that latent behaviors can still be inspected meaningfully. In finance, where interpretability and validation matter, that balance is often preferable to brute-force scale.

2.3.5 Optimization and Hyperparameters

The optimization scheme is designed to stabilize joint training across continuous and discrete objectives. The optimizer is AdamW, which updates parameters according to adaptive first- and second-moment estimates together with decoupled weight decay. Abstractly, if θ_u denotes the parameter vector at update step u, then AdamW applies

$$\theta_{u+1} = \theta_u - \eta_u \frac{\hat{m}_u}{\sqrt{\hat{v}_u} + \varepsilon_{\text{opt}}} - \eta_u \lambda_w \theta_u, \tag{2.84}$$

where η_u is the learning rate, $\hat{m}_u$ and $\hat{v}_u$ are bias-corrected first and second moments, and λ_w is the weight-decay coefficient. Decoupled weight decay is useful because it regularizes the parameter norm without entangling regularization with the adaptive gradient scaling itself.

The learning rate follows a cosine decay schedule. If $\eta_{\max}$ is the initial learning rate and U the total number of optimization steps, a generic cosine schedule can be written as

$$\eta_u = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi u}{U}\right). \tag{2.85}$$

This choice is natural in a diffusion context because it allows large early steps during broad representation formation and smaller later steps during refinement. It also harmonizes with the broader cosine design already used in the noise schedule, although the two cosine constructions serve different purposes.

Gradient clipping is applied to maintain numerical stability:

$$g_u \leftarrow g_u \cdot \min\left(1, \frac{c}{\|g_u\|_2}\right). \tag{2.86}$$

The threshold c is a hyperparameter, typically chosen to keep rare gradient spikes from destabilizing the optimizer. This is particularly relevant in joint training because the magnitude of the continuous

loss and discrete loss may fluctuate differently across batches.

The core hyperparameters of the notebook-aligned system are as follows:

$$d = 128, \qquad L = 6, \qquad H = 8, \tag{2.87}$$

for the continuous denoiser, together with

$$T_{\text{diff}} = 1000, \qquad T_{\text{DDIM}} = 50, \qquad T_{\text{mask}} \text{ finite and modest}, \tag{2.88}$$

for diffusion and masked-generation horizons. The return model trains with epsilon prediction under a cosine noise schedule; the regime model trains with token cross-entropy under linear masking probability $t_{\text{text}}/T_{\text{mask}}$.

Additional hyperparameters include batch size, number of training epochs, token-embedding dimension, feed-forward expansion ratio, weight-decay coefficient, and the balancing parameter λ_{sent}. Although these may vary across experiments, their conceptual roles are fixed. Batch size controls stochastic gradient variance. Epoch count determines optimization exposure to the synthetic dataset. The expansion ratio controls the expressive power of the feed-forward sublayers. The balancing parameter controls whether the system prioritizes return denoising or token recovery.

In this framework, hyperparameters are not arbitrary implementation decorations. They shape the inductive bias of the model. A larger T_{diff} produces a finer denoising path but increases sampling cost. A smaller DDIM step count accelerates generation but may reduce fidelity. A larger model width increases expressive capacity but may complicate interpretation and require more data. Thus, hyperparameter selection is best viewed as a trade-off among fidelity, compute, and diagnostic clarity.

2.3.6 Implementation Logic and Reproducibility

The final component of the pipeline is implementation logic. In modern machine learning, it is possible to specify a mathematically elegant model and still obtain unreliable behavior if the data flow, initialization, randomness, or evaluation path are poorly controlled. This issue is especially acute in finance, where minor implementation differences can produce substantial differences in generated outputs and therefore in downstream conclusions.

The first reproducibility principle is **deterministic schedule construction**. Once T_{diff} and the cosine schedule offset are fixed, the arrays β_t, α_t, $\bar{\alpha}_t$, and their square roots should be computed once and stored. This guarantees that every batch uses exactly the same diffusion geometry. Similarly, the DDIM sampling schedule should be defined deterministically before generation begins. If one changes timestep subsets ad hoc across runs, then output differences become harder to attribute.

The second principle is **controlled randomness**. Synthetic data generation, minibatch shuffling,

Gaussian diffusion noise, and parameter initialization all involve random draws. Let r denote a master random seed. Then the full experiment may be regarded as conditionally defined by

$$\mathcal{E}(r) = \Big(X(r), S(r), \theta_0(r), \{\epsilon_u(r)\}, \{\text{batches}_u(r)\}\Big). \tag{2.89}$$

Fixing r does not remove stochasticity conceptually, but it does make the experimental instance reproducible. This is valuable when comparing architectural changes, schedule changes, or evaluation metrics.

The third principle is **clear separation of preprocessing, training, and sampling**. Normalization statistics must be computed on the training data and applied consistently at generation time. The denormalization map

$$x_{\text{raw}} = x_{\text{norm}} \cdot (\sigma_X + \varepsilon_{\text{norm}}) + \mu_X \tag{2.90}$$

should be explicit and stable. Otherwise, generated outputs may appear unrealistic merely because the inverse transformation was handled inconsistently.

The fourth principle is **traceable evaluation**. Since the purpose of the model extends beyond raw generation to diagnostic use, the evaluation pipeline should log both sample outputs and structural summaries. For example, one may store generated return moments, KS statistics, latent similarity matrices, or token confusion summaries. The point is to make the generative model inspectable as a financial object rather than as an opaque sample emitter.

A final principle is **modularity**. The synthetic data generator, continuous diffusion trainer, discrete masked-diffusion trainer, sampler, and evaluator should remain distinct components. This makes it possible to answer questions such as: Did a change in generated path realism come from the regime simulator, from denoiser capacity, from token masking behavior, or from DDIM step reduction? In a research setting, such questions are unavoidable. A modular pipeline makes them answerable.

For these reasons, the implementation logic of the present framework is part of the mathematical contribution rather than a purely engineering appendix. The pipeline embodies a view of financial generative modeling in which each stage has a clearly defined structural role: regime-aware data construction, schedule-aware corruption, denoising-based learning, efficient reverse generation, and statistics-aware evaluation. This clarity is what allows the model to serve not only as a generator of synthetic financial data, but also as a framework for analyzing the relationship between regime structure, learned representations, and the adequacy of assumed data-generating processes.

2.4 Applications in Finance

2.4.1 Synthetic Data Generation for Backtesting

One of the most immediate uses of diffusion models in finance is the generation of **synthetic return paths** for backtesting and simulation. The underlying problem is straightforward to state but difficult to solve well. Historical financial datasets are finite, regime dependent, and often dominated by only a few major market episodes. Yet many quantitative workflows require repeated exposure to alternative plausible realizations of return dynamics. If a strategy is tested only on the exact historical sample that happened to occur, then the evaluation may confound structural robustness with accidental path dependence. A useful synthetic-data generator should therefore produce return sequences that are *new* but still statistically aligned with the empirical or assumed financial process.

Formally, let $X_{\text{real}} = \{x^{(i)}\}_{i=1}^{N}$ denote a set of observed or simulated training paths, each $x^{(i)} \in \mathbb{R}^{T\times A}$. The task is to generate a new collection

$$X_{\text{gen}} = \{\tilde{x}^{(j)}\}_{j=1}^{M}, \qquad \tilde{x}^{(j)} \sim p_\theta, \tag{2.91}$$

such that the learned generative distribution p_θ approximates the target law sufficiently well for backtesting purposes. In practice, one does not need exact equality between p_θ and the true data-generating distribution. What is needed is preservation of the structural features that matter for strategy evaluation: marginal return behavior, dispersion, dependence, regime composition, and the temporal organization of calm and stressed periods.

The diffusion framework is well suited to this task because sampling from the model yields full path realizations rather than one-step predictions. Starting from Gaussian noise,

$$x_{T_{\text{diff}}} \sim \mathcal{N}(0, I), \tag{2.92}$$

the DDIM reverse process produces a synthetic clean sample $\hat{x}_0$ through repeated deterministic denoising. This means that generation is pathwise from the outset. The model does not need to simulate one return at a time in left-to-right fashion. Instead, it progressively reconstructs an entire trajectory whose global and local structure are shaped by the learned return manifold.

This matters in backtesting because many strategy properties are path dependent. Maximum drawdown, turnover, stop-loss activation, trend persistence, and volatility targeting all depend on the sequence structure, not merely on the empirical marginal distribution of returns. A model that generates points independently may reproduce histograms yet remain useless for strategic evaluation. Diffusion models, by contrast, generate objects that live in path space, making them more naturally aligned with financial simulation.

In the synthetic regime-switching environment considered here, the learned diffusion model approximates not just a single return law but a mixture over latent market states. That is especially valuable

for backtesting because a strategy often performs differently in calm, volatile, and transition periods. A backtesting generator that erases those distinctions may produce smooth but misleading results. A regime-aware diffusion model can, in principle, generate sequences whose state composition more closely reflects the training environment.

Validation of synthetic paths begins with the marginal distribution. Let F_{real} and F_{gen} denote the empirical cumulative distribution functions of real and generated returns. The Kolmogorov–Smirnov statistic is

$$D_{\text{KS}} = \sup_x |F_{\text{real}}(x) - F_{\text{gen}}(x)| \,. \tag{2.93}$$

A small D_{KS} indicates close agreement in the one-dimensional marginal distribution. This is a useful first diagnostic because it is interpretable and model agnostic. If the generated returns fail even to reproduce the observed marginals, then deeper structural fidelity is unlikely.

However, backtesting requires more than marginal alignment. Financial returns are known to exhibit stylized facts such as heavy tails, volatility clustering, and dependence heterogeneity [2]. It is therefore useful to supplement the KS statistic with additional summary discrepancies. Let $T(\cdot)$ denote a vector of stylized-fact statistics, such as sample skewness, excess kurtosis, lagged autocorrelation of squared returns, and cross-asset correlation moments. A broader validation metric is then

$$\Delta_{\text{stylized}} = \|T(X_{\text{real}}) - T(X_{\text{gen}})\|_2 \,. \tag{2.94}$$

This metric acknowledges that backtesting fidelity depends not only on univariate distribution matching but on structural properties of the generated ensemble.

A particularly useful feature of DDIM generation is that it allows many synthetic paths to be generated rapidly. If M scenarios are required, the effective cost scales approximately as

$$\text{Cost}_{\text{gen}} \propto M \times T_{\text{DDIM}} \times C_\theta, \tag{2.95}$$

where T_{DDIM} is the reduced reverse-step count and C_θ the cost of a denoiser call. This makes scenario libraries feasible without the full burden of a 1000-step reverse chain. For financial practitioners, this is not a minor computational convenience. It is what makes synthetic path generation usable in iterative research, model comparison, and repeated strategy evaluation.

There is also a methodological benefit. In traditional backtesting, one often faces a tension between realism and diversity. Historical replay is realistic but not diverse. Parametric simulation is diverse but often unrealistic. Diffusion-generated samples occupy a potentially valuable middle ground: they can be structurally new while still anchored to the learned distributional geometry of the training data. That is precisely why recent work on diffusion for tabular and structured data is relevant for finance [8]. The question is not merely whether one can generate samples, but whether one can generate financially useful alternative histories.

In a controlled synthetic setting, the answer can be assessed more clearly because the underlying

data-generating process is known. If the diffusion model is trained on regime-switching GBM data and the generated samples reproduce the expected stylized properties and regime composition, then the model has demonstrated an ability to learn a nontrivial financial path distribution. That does not prove perfect real-world fidelity, but it establishes the methodological viability of diffusion as a synthetic backtesting engine.

2.4.2 Regime Identification via Latent Space Analysis

A second application moves beyond sample generation and treats the trained denoiser as a source of **latent representations** that can reveal regime structure. This is one of the most interesting aspects of diffusion in finance because it shifts the interpretation of the model from pure generator to structural analyzer. If the training data arise from a regime-switching process, then the denoiser must learn how different states of the world are organized across diffusion time. It is therefore natural to ask whether intermediate hidden states carry information about which regime family best explains a given observation.

Let $h_t(x)$ denote an intermediate representation of sequence x extracted from the denoiser at diffusion step t. For example, it may be the pooled hidden state from the final Transformer block when processing the noised sequence x_t. If two sequences x and x' belong to similar latent market regimes, one may expect their intermediate representations at a suitable noise level to be close. A natural regime-proximity measure is the cosine similarity

$$\mathrm{Sim}_t(x, x') = \frac{\langle h_t(x), h_t(x') \rangle}{\|h_t(x)\|_2 \|h_t(x')\|_2}. \tag{2.96}$$

High similarity suggests that the denoiser interprets the two sequences as occupying nearby regions of the learned mixture geometry.

The choice of diffusion level matters. At very low noise levels, the representation may be dominated by fine idiosyncratic path details that distinguish one sample from another even within the same regime. At very high noise levels, too much structure may already have been destroyed, leaving only coarse distributional information. An intermediate step, such as $t = 500$ in a 1000-step schedule, is often conceptually attractive because it suppresses local noise while preserving broad structural signals. In regime-dependent finance, those broad signals are precisely what one wants.

This representation-based perspective can be compared to the classic filtering logic of regime-switching econometrics. In Hamilton's framework, one estimates filtered or smoothed probabilities of latent states using the likelihood implied by the observed sequence [3]. Diffusion does not explicitly compute a filtered probability in that sense. Instead, it learns a representation space in which regime-conditioned geometry may emerge implicitly. The comparison is therefore not one of method substitution but of methodological analogy. Both approaches seek information about latent states from observed sequences, but one does so through explicit state-space likelihood recursion, and the

other through denoising representations learned from a generative objective.

A practical regime-identification workflow may proceed as follows. First, collect hidden representations $h_t(x^{(i)})$ for a set of sequences. Second, cluster them in representation space or compare them to regime prototypes. Third, assign regime labels by nearest centroid, soft similarity, or another geometry-based rule. If true synthetic labels are available, one may quantify the quality of this recovery through a simple accuracy measure:

$$\mathrm{Acc}_{\mathrm{reg}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{\hat{R}_i = R_i\}, \tag{2.97}$$

where R_i is the true regime label and $\hat{R}_i$ is the label inferred from latent-space analysis.

An alternative validation route uses pairwise distances. Suppose one has pairs of sequences known to belong to the same regime and pairs known to belong to different regimes. Then one can compare the average similarity gap:

$$\Delta_{\mathrm{sim}} = \mathbb{E}\big[\mathrm{Sim}_t(x, x') \mid R = R'\big] - \mathbb{E}\big[\mathrm{Sim}_t(x, x') \mid R \neq R'\big]. \tag{2.98}$$

A positive and substantial gap indicates that the denoiser's latent space organizes regime structure meaningfully.

This application is important because it reveals a different type of utility. A generative model need not be evaluated only by how realistic its samples look or how closely its marginals align with the data. It may also be valuable if its hidden states form a useful representation of the structural organization of the market. In finance, where latent-state interpretation is often central to strategy design and risk monitoring, that is a significant gain.

It also suggests an explanation for why diffusion is particularly promising in regime-sensitive environments. The denoiser is trained across all noise levels, which means it must learn both local detail and coarse mixture geometry. Intermediate representations therefore arise from a task that is already multiscale and regime aware. This is quite different from a conventional predictive model trained to minimize only next-step forecast error, which may not need to form a rich representation of market-state geometry at all.

The latent-space perspective also creates a bridge between synthetic and real data. In the synthetic environment, regime labels are known, so the geometric behavior of hidden states can be evaluated directly. In real data, the labels may be latent or only weakly approximated, but the same representation techniques can still be used for clustering, anomaly detection, or candidate regime segmentation. In that sense, the synthetic experiment provides more than a standalone benchmark. It provides a controlled proof of concept for using diffusion representations as regime-sensitive financial embeddings.

2.4.3 Stress Scenario Generation

A third application is **stress scenario generation**. In risk management and strategic analysis, one often wants not merely random synthetic paths from the learned distribution, but paths that are related in a controlled way to a given observed history. The problem can be phrased as follows: given a seed path x_0^*, generate plausible neighboring scenarios under the learned distribution that preserve some structural resemblance to the seed while introducing stress-like perturbations.

Diffusion models are naturally suited to this because they permit **partial noising**. Rather than start the reverse process from pure Gaussian noise, one can begin from an observed or historical path, corrupt it to an intermediate diffusion level, and then denoise from there. Formally, if x_0^* is a seed path and t^* a chosen noising depth, define

$$x_{t^*}^* = \sqrt{\bar{\alpha}_{t^*}} x_0^* + \sqrt{1 - \bar{\alpha}_{t^*}} \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{2.99}$$

This state lies partway between the original path and the isotropic noise prior. Denoising from $x_{t^*}^*$ with the trained reverse model yields a new sample that remains in the neighborhood of the seed path under the learned distribution but is not identical to it.

The interpretation of t^* is central. Small values of t^* correspond to gentle perturbations. Much of the original information survives, so the denoised output will tend to stay close to the seed history. Large values of t^* erase more structure, so the reverse process is freer to reconstruct a different but still plausible path. Thus t^* acts as a stress-radius parameter:

$$t^* \uparrow \quad \Longrightarrow \quad \text{broader scenario neighborhood.} \tag{2.100}$$

This gives the model user a direct control over how aggressively the original path should be altered.

In finance, this is extremely useful. A stress scenario is rarely intended to be a wholly arbitrary path. It is often intended to be a plausible deformation of a known event, position, or macro-financial environment. For example, one may wish to ask: what are nearby but more adverse versions of a given crisis week? What are alternative rebound trajectories after a volatility shock? What are plausible stressed versions of a currently observed market configuration? Partial noising and denoising answer such questions in a generative yet structured way.

A basic validation metric for stress generation compares the stressed path to the original seed:

$$d_{\text{stress}}(\tilde{x}, x_0^*) = \frac{1}{TA} \|\tilde{x} - x_0^*\|_F^2. \tag{2.101}$$

This measures the average squared deviation. However, distance alone is not enough, because a path could be far from the seed in a way that is economically uninteresting. One therefore supplements it with stress-oriented diagnostics such as realized volatility, tail quantiles, drawdown,

or negative-return concentration. If $V(\cdot)$ denotes a stress statistic, then one can define

$$\Delta_V = V(\tilde{x}) - V(x_0^*). \tag{2.102}$$

A scenario may then be considered meaningfully stressed if Δ_V exceeds a chosen threshold while the overall path remains in a plausible neighborhood of the seed.

This application is closely related to the broader idea of editing in diffusion systems. In image generation, stochastic differential editing shows that partial corruption followed by denoising can transform an input in a controlled manner. The present financial use is conceptually similar, except the object being edited is a return sequence rather than an image. What changes is not the mathematical principle but the economic interpretation. A nearby generated image may be visually interesting; a nearby generated return path may be useful for risk estimation, scenario planning, or governance review.

There is also an important governance advantage. Traditional stress testing often relies on handcrafted shocks or historically replayed crisis windows. Those methods remain useful, but they can be inflexible. Handcrafted shocks may be too simplistic, while historical replay may over-anchor to one specific realization. Diffusion-based stress generation offers a middle ground: it preserves the structural realism learned from the training distribution while permitting controlled local variation.

In the regime-switching synthetic environment, the meaning of stress can also be linked explicitly to regime transitions. If the seed path belongs mostly to a calm regime, stronger noising and denoising may move it into the neighborhood of a higher-volatility regime under the learned mixture geometry. If the seed path is already stressed, partial diffusion may produce variants that explore nearby stress realizations without abandoning the same broad regime family. Thus, stress generation becomes not merely perturbation, but structured movement through a regime-aware path space.

2.4.4 Missing Data Imputation

A fourth application is **missing data imputation**. Financial datasets frequently contain incomplete observations. Missing values may arise from illiquid assets, asynchronous market closes, data vendor gaps, reporting lags, or the deliberate withholding of certain inputs in scenario analysis. The challenge is to reconstruct the missing entries in a way that respects both cross-sectional and temporal structure. A model that imputes each point independently will often fail because financial data are organized by dependence, not merely by marginal plausibility.

Formally, let $x_0 \in \mathbb{R}^{T\times A}$ be a partially observed path, and let Ω denote the set of observed entries. Its complement Ω^c contains the missing values to be imputed. One may represent the observation pattern with a binary mask

$$M_\Omega(i,j) = \begin{cases} 1, & (i,j) \in \Omega, \\ 0, & (i,j) \in \Omega^c. \end{cases} \tag{2.103}$$

The imputation problem is to infer $\hat{x}_{ij}$ for all $(i,j) \in \Omega^c$ while preserving compatibility with the observed entries.

Diffusion offers a principled way to do this through **inpainting-style denoising**. The general idea is to run the reverse process while reimposing the observed values at each step. Let $\Phi_\theta(x_t, t)$ denote one reverse denoising update. Then an inpainting step can be written as

$$x_{t-1} = M_\Omega \odot x_{t-1}^{\text{obs}} + (1 - M_\Omega) \odot \Phi_\theta(x_t, t), \tag{2.104}$$

where $\odot$ denotes elementwise multiplication and x_{t-1}^{obs} is the appropriately noised or denoised version of the observed entries at step $t-1$. The key principle is that observed data are never abandoned. They are reinserted at every reverse stage, while missing entries are filled in by the model's learned conditional structure.

This mechanism is attractive in finance because it naturally leverages both temporal and cross-asset context. Suppose one is missing returns for one asset over a block of time. A diffusion model can reconstruct them using information from the observed history of that asset, the behavior of related assets, and the learned geometry of plausible regime-conditioned trajectories. This is much richer than simple forward fill, interpolation, or univariate mean imputation.

The quality of imputation can be evaluated by artificial masking experiments. One hides a subset of entries whose true values are known, imputes them, and then computes the error over the held-out set. A standard metric is the root mean squared error

$$\text{RMSE}_{\Omega^c} = \left(\frac{1}{|\Omega^c|} \sum_{(i,j)\in\Omega^c} (\hat{x}_{ij} - x_{ij})^2 \right)^{1/2}. \tag{2.105}$$

For financial data, it is also useful to examine whether the imputed values preserve downstream properties such as covariance structure, factor exposure, and volatility patterns. Thus, one may supplement pointwise RMSE with a structural discrepancy metric

$$\Delta_\Sigma = \|\hat{\Sigma}_{\text{imputed}} - \Sigma_{\text{true}}\|_F, \tag{2.106}$$

where Σ denotes an empirical covariance estimate. A low RMSE but a large covariance distortion would indicate that the imputation is numerically accurate yet financially misleading.

This application highlights a deep advantage of diffusion models: they are not merely samplers from an unconditional distribution. They are flexible conditional reconstructors. Inpainting works because the reverse process can be constrained while still following the learned distributional geometry. That same principle underlies scenario editing, partial-path completion, and conditional synthetic generation more broadly.

In a regime-aware context, missing-data imputation has an additional layer of meaning. Suppose

missing values occur during a transition episode. A simple imputer may smooth over the transition and pull the series toward an average state. A diffusion imputer, by contrast, can condition on the learned mixture distribution and infer values that are consistent with transition dynamics. This matters because transition periods are often precisely the times when data are most informative for risk and strategy.

There is also a conceptual symmetry with the discrete regime model. Just as the masked diffusion Transformer reconstructs missing tokens in a regime sequence, the continuous diffusion model reconstructs missing numerical values in a return tensor. The two are not identical mathematically, but they instantiate the same guiding principle: information is partially hidden, and the model learns to restore it by exploiting the joint structure of the data-generating environment.

2.4.5 Diagnostic Testing of the Data-Generating Process

The fifth application is perhaps the most conceptually distinctive: using the trained diffusion model as a tool for **diagnostic testing of the assumed data-generating process**. This moves beyond the idea of a generator as a simulator and treats it instead as a specification-testing device. The motivating question is simple: if the model is trained under an assumed structural law, does the resulting generative behavior match the target financial data closely enough to regard the assumed law as adequate?

In the present paper, the assumed structural law is a regime-switching GBM. That law is used to generate the training sample. In a broader methodological setting, however, one could imagine training a diffusion model on synthetic data produced by any candidate financial process and then comparing the resulting generated samples with real observed data. If the generated samples fail systematically to match the real ones, the structural process used to generate the synthetic training environment is likely misspecified. In this way, the diffusion model functions as an amplifier of structural adequacy or inadequacy.

Let X_{real} denote the target dataset and X_{gen} the sample generated by the diffusion model trained under a candidate DGP. A simple diagnostic rule is based on the KS statistic:

$$\text{Reject candidate DGP if } D_{\text{KS}} > \tau_{\text{KS}}, \tag{2.107}$$

for some threshold τ_{KS}. This is intentionally simple. It says that if the generated marginals deviate too much from the target marginals, then the assumed DGP cannot be regarded as adequate.

A more realistic diagnostic should examine multiple stylized properties simultaneously. Let

$$T(X) = \big(\text{mean}(X), \text{vol}(X), \text{kurt}(X), \text{acf}(|X|), \text{corr}(X), \dots\big) \tag{2.108}$$

denote a vector of summary statistics or stylized-fact descriptors. Then define

$$\Delta_{\text{diag}} = \|T(X_{\text{real}}) - T(X_{\text{gen}})\|_2. \tag{2.109}$$

If Δ_{diag} is persistently large, the candidate process is failing to reproduce essential features of the target distribution. This does not identify exactly *why* the DGP is wrong, but it indicates that the process family is inadequate relative to the statistics one cares about.

Why use a diffusion model for this instead of comparing the candidate DGP directly to the data? The answer is that the diffusion model acts as a flexible learned representation of the synthetic process family. If the candidate DGP is internally coherent and sufficiently expressive, the diffusion model trained on its outputs should be able to learn and reproduce the DGP's distributional geometry. If that learned geometry still fails against the target data, the problem is less likely to be the limited expressiveness of the generative neural model and more likely to be the inadequacy of the assumed structural law itself. In other words, diffusion serves as a powerful intermediary between structural simulation and empirical comparison.

This application connects naturally to the finance literature on stylized facts and model adequacy. A useful financial DGP is not one that merely produces Gaussian-looking returns, but one that captures volatility clustering, heavy tails, dependence asymmetries, and regime behavior [2]. If a trained diffusion model faithfully reproduces the synthetic regime-switching GBM law yet still cannot match the empirical financial dataset on these dimensions, the implication is that the underlying process family must be enriched.

It also connects to the broader idea of generative-model-based goodness-of-fit testing. In classical econometrics, specification testing often proceeds by examining residual properties or moment conditions. Here, one instead trains a generative surrogate for the candidate law and compares its output to reality. This is particularly attractive in complex or high-dimensional settings where closed-form likelihood comparison is impractical. The learned diffusion model becomes a tractable empirical proxy for the candidate distribution.

A practical testing workflow may therefore be described as follows. First, specify a candidate financial DGP. Second, generate a large synthetic dataset under that DGP. Third, train the diffusion pipeline on the synthetic data. Fourth, sample generated paths from the trained model. Fifth, compare those samples to real data using marginal, multivariate, and stylized-fact diagnostics. Sixth, reject or revise the candidate DGP when the discrepancies are too large. This process treats the diffusion model not as the hypothesis itself, but as an instrument for rendering the implications of the hypothesis observable and testable in a flexible generative space.

This is a particularly powerful idea for financial research because many financial models are best understood not as exact truths but as controlled approximations. A regime-switching GBM is not reality; it is a stylized candidate law. The value of the diagnostic diffusion approach is that it creates a systematic way to ask whether such a law is good enough for the generative, stress-testing, or

synthetic-data task at hand. If it is not, the discrepancy can motivate a richer process, such as state-dependent correlations, jumps, stochastic volatility, or hybrid continuous-discrete architectures.

For that reason, this fifth application is in some sense the culmination of the paper's argument. Diffusion is not useful only because it can generate synthetic financial paths. It is useful because the process of learning and sampling from a diffusion model reveals whether a candidate structural understanding of the market is capable of reproducing the distributional features one actually observes. That is what turns diffusion from a generator into a diagnostic framework.

2.5 Challenges and Conclusions

2.5.1 Current Limitations

The framework developed in this paper is intentionally ambitious: it combines continuous diffusion for multivariate return paths, masked diffusion for discrete regime tokens, Transformer denoising, regime-switching synthetic data generation, and diagnostic applications that extend beyond raw sampling. Precisely because the framework is broad, it is essential to state its limitations clearly. A generative system in finance is useful only if its power is matched by a careful understanding of what it does *not* yet solve.

The first and most important limitation concerns **non-stationarity in real markets**. The synthetic laboratory used in this paper is built around a regime-switching geometric Brownian motion. That construction is useful because it introduces latent states, changing drift, and changing volatility into the data-generating process. It therefore creates a meaningful bridge between classical financial econometrics and modern generative learning. Yet the regime-switching GBM remains a stylized world. Real financial time series are not simply governed by a finite Markov chain over a small number of Gaussian regimes. They are affected by institutional adaptation, policy interventions, changing market microstructure, endogenous reflexivity, structural breaks, evolving participant composition, and shocks that may alter the system itself rather than merely perturb a fixed law.

Formally, the synthetic setup assumes that returns are sampled from a mixture distribution with stationary transition structure:

$$p(r_t) = \sum_{k=1}^{K} \pi_k \, p(r_t \mid R_t = k), \tag{2.110}$$

where the regime weights are anchored by a stationary distribution π. In real markets, however, even the regime dynamics may drift over calendar time. One may therefore need a time-indexed law

$$p_u(r_t) = \sum_{k=1}^{K} \pi_{k,u} \, p_u(r_t \mid R_t = k), \tag{2.111}$$

where both the regime probabilities and the conditional regime laws vary with calendar time u. The current framework does not resolve this deeper non-stationarity. It models a regime mixture; it does not fully model an evolving financial ecology.

The second limitation concerns the **discrete regime model**. In this paper, the masked diffusion component operates over a small vocabulary of regime tokens. This is appropriate for latent-state sequences generated by a synthetic Markov system. It is not, however, a general financial language model. The token space is intentionally small:

$$|\mathcal{V}| = K + 1, \tag{2.112}$$

where the additional symbol is the masking token. This makes the masked diffusion problem clear and tractable, but it also restricts semantic scope. The discrete model here captures state labels, not the richness of analyst text, macro commentary, earnings narratives, or qualitative market discourse. It should therefore be interpreted as a regime-sequence generator, not as a full text-conditioned financial reasoning system.

A third limitation is **computational cost**. Although DDIM sampling reduces inference cost substantially, diffusion remains expensive relative to many simpler generators. If one uses a full reverse chain of length T_{diff}, the cost scales approximately as

$$\text{Cost}_{\text{DDPM}} \propto T_{\text{diff}}\, C_\theta, \tag{2.113}$$

where C_θ is the cost of one denoiser evaluation. Even with DDIM reduction to T_{DDIM} steps, one still has

$$\text{Cost}_{\text{DDIM}} \propto T_{\text{DDIM}}\, C_\theta, \qquad T_{\text{DDIM}} \ll T_{\text{diff}}, \tag{2.114}$$

but not zero cost. For small notebook-scale experiments this is manageable. For large-scale institutional scenario generation across many assets, horizons, and conditioning settings, compute remains a meaningful constraint. This affects not only runtime but also model design choices, sampling budgets, and the feasibility of repeated validation loops.

A fourth limitation concerns **evaluation**. In image generation, one may at least appeal to perceptual plausibility or benchmark metrics. In finance, there is no single accepted ground truth for synthetic data quality. A generator can match the marginal distribution and still fail to preserve dependency structure. It can preserve covariance and still miss regime transitions. It can generate plausible paths and still be strategically misleading for a specific downstream task. This difficulty can be written abstractly as follows. Let $Q(\cdot)$ denote a quality functional. In practice,

$$Q(X_{\text{gen}}, X_{\text{real}}) \neq \text{single scalar with universal meaning.} \tag{2.115}$$

Instead, quality is multi-objective: it depends on marginals, tails, temporal structure, cross-sectional dependence, regime coherence, and application-specific utility. The current framework uses KS

distance, stylized-fact discrepancies, latent similarities, and imputation errors, but these do not collapse into one definitive score.

A fifth limitation concerns **model risk in the synthetic environment itself**. Because the data are generated from a regime-switching GBM, the diffusion model is being trained in a world whose structure is partly known and partly simplified. This is valuable pedagogically and analytically, but it also creates a danger: success in the synthetic world may be mistaken for adequacy in the real one. A model may learn the synthetic regime mixture extremely well and still fail to handle jumps, discontinuous liquidity events, or changing factor structures in real markets. Thus, the synthetic laboratory is best understood as a controlled proving ground rather than a claim of immediate real-market completeness.

A sixth limitation is **conditioning scope**. The current model conditions on diffusion time and, in the discrete branch, on masked token context. It does not yet include richer conditional interfaces such as macro descriptors, asset metadata, textual prompts, or portfolio constraints. This limits controllability. A future financial user may wish to request scenarios with specific stress signatures, policy environments, or sector compositions. The current implementation does not yet provide that level of structured conditional generation.

These limitations are not reasons to reject the framework. They are reasons to interpret it correctly. The present system should be seen as a principled and extensible foundation for generative financial modeling under regime dependence, not as the final answer to non-stationary market simulation.

2.5.2 Open Research Problems

If the limitations define the boundaries of the present framework, the open research problems define its most promising directions for extension. Several questions stand out as especially important.

The first concerns **classifier-free guidance and controlled scenario generation**. In modern diffusion systems, classifier-free guidance provides a way to interpolate between unconditional and conditional denoising fields [7, 18]. In abstract form, a guided score or denoising field may be written as

$$\epsilon_{\text{guided}} = (1 + w)\epsilon_\theta(x_t, c, t) - w\,\epsilon_\theta(x_t, \varnothing, t), \tag{2.116}$$

where c is a conditioning signal, $\varnothing$ denotes the unconditional branch, and w is a guidance weight. In finance, this opens the possibility of text-to-scenario or metadata-to-scenario generation: one could condition on regime descriptions, macro states, portfolio exposures, or stress tags. A user might request, for example, a high-volatility recessionary scenario with specific cross-asset behavior, or a liquidity stress regime affecting a subset of sectors. The mathematical machinery for guided denoising exists in adjacent domains; the financial adaptation remains largely open.

The second open problem is the extension of masked diffusion from small regime vocabularies to **real financial text corpora**. The present paper uses discrete diffusion over latent regime

tokens, which is conceptually clean and computationally modest. A natural next step is to model genuine financial language: earnings call text, analyst commentary, research notes, macro releases, or risk narratives. Recent advances in discrete diffusion and masked diffusion language modeling suggest that this is plausible [9, 20, 12]. However, real financial text raises new challenges: larger vocabularies, domain-specific terminology, evolving semantics, long-context dependence, and the problem of connecting textual content to quantitative financial trajectories. One would need a joint model in which the continuous return manifold and the textual information manifold interact coherently rather than being trained as separate unrelated objects.

A third open problem is **online or streaming diffusion**. The present framework is batch based. One trains on a synthetic dataset and then samples from the learned model. Real finance, by contrast, evolves continuously. New data arrive every day, sometimes every second. If one wants diffusion to support real-time regime detection or live scenario monitoring, then the model must update or adapt without full retraining. Abstractly, one would like a family of time-indexed generative models $\{p_{\theta_u}\}$ such that

$$p_{\theta_u}(x) \approx p_{\mathrm{data},u}(x), \tag{2.117}$$

with controlled lag as calendar time u advances. This raises several difficulties: catastrophic forgetting, instability in online fine-tuning, changing normalization statistics, and the challenge of deciding whether drift reflects transient noise or structural change. A streaming diffusion system would be highly valuable for finance, but it requires both algorithmic innovation and rigorous governance controls.

A fourth open problem concerns **theoretical guarantees under non-stationary financial processes**. Diffusion models are well motivated probabilistically, but many of their strongest guarantees are tied to relatively stable or well-behaved data distributions. Finance is not such a setting. The underlying process may be non-ergodic, regime shifting, and structurally evolving. One would therefore like results of the following broad form: under what conditions on a time-varying process $p_u(x)$ does training on a rolling or weighted sample yield a denoiser whose reverse process approximates a meaningful local distribution? Put differently, when does the learned generative model converge to something economically interpretable under non-stationarity? This is not yet well understood.

A fifth open problem is **multimodal regime conditioning**. Financial reality is not represented by returns alone. It includes prices, volumes, options surfaces, macro indicators, textual commentary, event calendars, and accounting signals. A richer diffusion framework would allow all of these to enter a common conditioning structure. One could imagine a model in which a return path is generated conditional on a bundle

$$c = (c^{\mathrm{text}}, c^{\mathrm{macro}}, c^{\mathrm{portfolio}}, c^{\mathrm{event}}), \tag{2.118}$$

and the denoiser learns how these modalities jointly influence the reverse trajectory. Such systems

would move closer to practical decision support, but they would also introduce serious governance questions around provenance, leakage, and overconditioning.

A sixth research direction is **evaluation theory**. At present, synthetic financial data are judged through families of diagnostics rather than a unified benchmark. This is sensible, but it leaves the field methodologically fragmented. One important goal would be to define a structured evaluation protocol in which different uses of synthetic data correspond to different metric bundles. For example, backtesting utility may require one set of statistics, stress testing another, and imputation a third. A formal evaluation map could be written as

$$\mathcal{M} : \text{Application class} \to \text{relevant metric family}. \tag{2.119}$$

This would not eliminate judgment, but it would make diffusion-based financial generation more auditable and more comparable across studies.

Finally, there is an open question about **structural interpretability**. Diffusion models are more diagnostically rich than many alternative generators because they expose intermediate denoising states. Yet it remains unclear how best to interpret these states in financial terms. Are they best viewed as approximate regime embeddings, local score estimates, transition indicators, or something more complex? Developing a rigorous financial semantics for latent denoising representations would significantly strengthen the methodological case for diffusion in applied finance.

These research directions collectively suggest that the most important future of diffusion in finance is not simply better unconditional sample realism. It is controllability, multimodality, online adaptation, theory under non-stationarity, and application-specific evaluation. The current framework opens these doors, but it does not yet walk through them fully.

2.5.3 Conclusions

This paper has argued that **diffusion models** provide a principled foundation for synthetic financial data generation under regime-dependent dynamics, and that their value extends beyond raw sample synthesis into process identification and diagnostic use. The central contribution is not the claim that diffusion is universally superior to every other generative approach in every financial setting. Rather, the contribution is the demonstration that diffusion offers an especially coherent methodological language for financial data because it is built around uncertainty, structured corruption, and learned recovery.

The first contribution of the paper was a formal framework for applying a **cosine-schedule denoising diffusion probabilistic model** with a **Transformer denoiser** to multivariate financial return sequences. The forward process was defined over return tensors, the reverse process was parameterized through epsilon prediction, and the Transformer backbone with AdaLN conditioning was motivated as the correct architectural response to non-local temporal and cross-sectional

dependencies in financial data. This moved the diffusion framework from image-oriented intuition toward a sequence-oriented financial interpretation.

The second contribution was the integration of **masked diffusion for discrete regime-token sequences**. Financial systems are not purely continuous. They often involve latent states, labels, or symbolic structures that are economically meaningful. By modeling the regime sequence through iterative masking and unmasking, the framework accommodates the discrete side of regime-dependent financial dynamics and links it conceptually to the continuous return generator. This makes the overall system more faithful to the hybrid nature of financial data.

The third contribution was the development of a **diagnostic perspective** on diffusion. The trained model was treated not only as a source of synthetic return paths, but also as a representation system whose latent states and reverse trajectories can be used for regime identification, stress scenario generation, missing-data imputation, and diagnostic testing of assumed data-generating processes. This is arguably the most important conceptual move in the paper. It suggests that diffusion models in finance should not be judged only by whether they generate plausible paths, but by whether they help reveal the structural logic of the distributions from which those paths arise.

From this perspective, the combination of diffusion models, Transformer denoisers, and regime-aware synthetic data is attractive for several reasons. It is **principled** because it is grounded in a well-defined forward and reverse stochastic process. It is **flexible** because it can accommodate both continuous and discrete data modalities. It is **computationally tractable** because accelerated samplers such as DDIM make practical scenario generation feasible. And it is **financially meaningful** because regime dependence, uncertainty, and latent mixture structure are not incidental features of markets but central ones.

The broader implication is that generative modeling in finance should be viewed less as a race for visually impressive samples and more as a disciplined effort to learn, probe, and stress-test complex financial distributions. Diffusion is promising precisely because it gives access to the geometry of those distributions through denoising trajectories. When the training environment is regime structured, the model learns not only how to sample, but how to organize market states in latent space. That creates an opportunity for a new class of financial tools: generators that are simultaneously simulators, imputers, scenario editors, and diagnostic instruments.

At the same time, the paper has emphasized caution. Real markets remain more complex than the synthetic environment considered here. Evaluation remains difficult. Conditional control remains limited. Online adaptation remains unresolved. Yet these are productive limitations, not fatal ones. They define a research agenda rather than a dead end.

The final conclusion is therefore precise. A **Transformer-based diffusion pipeline for regime-conditioned synthetic financial data** is a credible and promising methodological foundation for future financial machine learning. It supports realistic synthetic path generation, enables latent-state analysis, and offers a new route for testing the adequacy of candidate data-generating processes. In

a domain where uncertainty is structural rather than accidental, that is a substantial advantage. The value of diffusion in finance lies not only in what it can generate, but in what it can help us understand about the changing probabilistic architecture of markets.

Bibliography

[1] Rama Cont. Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues. *Quantitative Finance*, 1(2):223–236, 2001.

[2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[3] James D. Hamilton. A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle. *Econometrica*, 57(2):357–384, 1989.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[5] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[6] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling Tabular Data with Diffusion Models. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, 2023.

[7] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-LM Improves Controllable Text Generation. *arXiv preprint arXiv:2205.14217*, 2022.

[8] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. *arXiv preprint arXiv:2310.16834*, 2024.

[9] Alexander Q. Nichol and Prafulla Dhariwal. Improved Denoising Diffusion Probabilistic Models. In *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 2021.

[10] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.

[11] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical Text-Conditional Image Generation with CLIP Latents. *arXiv preprint arXiv:2204.06125*, 2022.

[12] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models. *arXiv preprint arXiv:2406.07524*, 2024.

[13] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, PMLR 37, 2015.

[14] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*, 2021.

[15] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. In *International Conference on Learning Representations*, 2021.

[16] Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance*, 20(9):1419–1440, 2020.

Chapter 3

Dual-Head Diffusion Transformers for Joint Financial Sequence and Text Generation: Architecture, Training, and Multimodal Applications

Abstract. This chapter studies a multimodal generative framework for finance in which continuous market features and natural-language technical-analysis descriptions are learned jointly rather than in isolation. The core claim is that realistic financial generation is inherently multimodal: prices, volume structure, technical indicators, and descriptive narratives are different views of the same underlying market state, and a model that captures only one of these views is necessarily incomplete. We develop a dual-head diffusion Transformer with a shared AdaLN-conditioned backbone, a continuous feature denoising head, a discrete text-generation head, and explicit cross-modal fusion pathways that enforce consistency between the two modalities during training and sampling. The chapter formalizes the joint diffusion objective, presents the full training and co-refinement sampling pipeline, and examines financial applications including paired synthetic dataset generation, automated technical-analysis reporting, sentiment-conditioned scenario construction, multimodal imputation, and cross-modal quality filtering.

3.1 Introduction: Complex Generative Models as Multimodal Financial Engines

3.1.1 Financial Markets as Inherently Multimodal Systems

Financial markets are often modeled as if they were purely numerical systems. In many classical and modern quantitative pipelines, the central object is a time series of prices, returns, realized volatility, or factor exposures. This representation is convenient, mathematically clean, and often highly informative. Yet it is also incomplete. A market state is rarely exhausted by a price vector alone. A single day of trading activity simultaneously produces open, high, low, close, and volume observations; generates momentum and oscillation indicators; reveals local volatility and liquidity structure; and gives rise to human-readable descriptions such as "bullish continuation with strong momentum," "range-bound consolidation," or "sell-off on elevated volume." These are not unrelated artifacts. They are different manifestations of the same underlying financial event. The central premise of this paper is that **financial data are inherently multimodal**, and therefore a generative model that operates on only one modality captures only a projection of the true data-generating phenomenon.

Let x_0 denote a continuous market-feature sequence and w_0 a paired natural-language sequence describing the same financial episode. The object of interest is not merely the marginal law of one or the other, but the joint law

$$p(x_0, w_0), \tag{3.1}$$

which captures how price structure and linguistic interpretation co-occur. If the modalities were independent, one would have

$$p(x_0, w_0) = p(x_0)p(w_0). \tag{3.2}$$

In financial settings this factorization is implausible. A text sequence that describes a breakout,

a pullback, a bearish reversal, or a consolidation regime is constrained by the shape, direction, and volatility of the underlying market sequence. Conversely, the feature sequence is often more interpretable when contextualized by human language, because language compresses multivariate technical configurations into economically meaningful summaries. The relevant relation is therefore one of dependence, not independence, and that dependence can be quantified through mutual information:

$$I(X;W) = \left[\log \frac{p(x_0, w_0)}{p(x_0)p(w_0)}\right]. \tag{3.3}$$

Whenever $I(X;W) > 0$, a model that ignores one modality necessarily discards information carried by the other.

This point is not merely philosophical. It has concrete consequences for generative modeling. Suppose one trains a model only on price sequences. That model may become very good at reproducing path statistics, volatility clustering, or cross-feature dependence. Yet it remains silent about the semantic interpretation of those paths. It does not know whether the generated path corresponds to an overbought rally, a mean-reverting channel, or a capitulation move on heavy volume. Now suppose instead that one trains a text-only generator on technical-analysis sentences or regime labels. That model may produce linguistically plausible descriptions, but it has no guarantee that its output corresponds to any coherent feature sequence. Neither model alone enforces **cross-modal consistency**. This is the key deficiency of unimodal generative design in finance.

The problem becomes even more pressing when one considers actual financial workflows. A synthetic dataset intended for downstream fine-tuning of a financial language model is more valuable if it contains both the market sequence and a matching textual interpretation. A scenario generator for analysts is more useful if it can produce not only synthetic OHLCV windows but also the technical commentary that would plausibly accompany them. A missing-data imputation system is more informative if it can impute an OHLCV segment while simultaneously generating a textual description of the imputed regime. In each case, the practical use of the model depends on whether the generated modalities remain aligned.

This motivates a stronger generative target. Rather than learning only a price distribution $p(x_0)$ or only a language distribution $p(w_0)$, one seeks to learn the full joint distribution over continuous feature sequences and discrete text descriptions. Such a target is not just richer; it is fundamentally different. If one learns the joint law, then the marginals can be recovered by integration or summation:

$$p(x_0) = \sum_{w_0} p(x_0, w_0), \qquad p(w_0) = \int p(x_0, w_0)\, dx_0. \tag{3.4}$$

The converse is not true. Learning the marginals does not recover the dependence structure between them. In other words, the joint problem strictly contains the unimodal problems.

This observation parallels developments in other areas of multimodal machine learning. Vision-language models such as CLIP showed that the alignment between visual and linguistic representa-

tions can be highly informative, precisely because the modalities describe the same latent content from different perspectives [17]. Recent work in multimodal diffusion has further demonstrated that a single Transformer can be trained across heterogeneous distributions while preserving modality-specific structure and shared semantic information [1]. The financial setting is different in data type and downstream use, but the core logic remains. Prices and descriptions are not two unrelated datasets; they are two coordinated views of one market state.

In this paper, the financial modality is represented as a twelve-dimensional feature sequence over sixty time steps, including OHLCV variables and technical indicators, while the language modality is represented as a twenty-four-token technical-analysis text. These choices are intentionally modest. They do not claim to exhaust the richness of real financial communication. Instead, they create a controlled multimodal environment in which the architecture can be studied clearly. Even in this simplified setting, however, the point is already evident: a market episode is more faithfully represented as a pair (x_0, w_0) than as either object alone.

Thus the introduction begins from a methodological claim. A truly complex financial generative model should not be judged solely by whether it can produce plausible price paths. It should also be judged by whether it can produce the language that belongs with those paths, and whether the two outputs are mutually coherent. The correct objective is therefore multimodal generation under shared market structure. The remainder of this section explains why earlier financial generative models do not achieve this, how a dual-head diffusion Transformer addresses the limitation, and what contributions follow from that design.

3.1.2 Limits of Unimodal Financial Generative Models

The rise of deep generative modeling in finance has produced a diverse landscape of approaches, but most of that landscape remains fundamentally **unimodal**. Models are usually built to generate numerical sequences, or else to process text, but rarely to generate both jointly in a way that enforces consistency. This section reviews the main families of unimodal financial generative systems and identifies their shared limitation.

A first important line of work is the use of **generative adversarial networks** for financial time series. The Quant GAN framework demonstrated that adversarial training can produce realistic synthetic financial series and helped establish that deep generative models could be relevant for quantitative finance [23]. In a GAN-style setting, a generator G maps latent noise z to a synthetic sequence, while a discriminator D attempts to distinguish real from generated samples. The objective is the usual minimax game

$$\min_G \max_D \left[{}_{x\sim p_{\text{data}}} \log D(x) +_{z\sim p(z)} \log(1 - D(G(z))) \right]. \tag{3.5}$$

For financial sequences, this framework can indeed capture nontrivial temporal structure. Yet the

object of generation remains a single modality: typically a return path or price sequence. Even when the generated outputs look realistic statistically, the model does not tell us what language, sentiment, or narrative meaning belongs to those outputs. The adversarial criterion rewards realism in the target data space, not cross-modal alignment.

A second family consists of **variational autoencoder** and latent-variable approaches for return generation. In these systems, a financial sequence is compressed into a latent representation and reconstructed through a decoder. The training objective is a variational lower bound,

$$\log p_\theta(x) \geq_{q_\phi(z|x)} [\log p_\theta(x \mid z)] - (q_\phi(z \mid x) \,\|\, p(z)). \tag{3.6}$$

Such models can be useful for sampling, denoising, or representation learning. But again, the generated object is usually numerical. The latent variable may capture hidden market structure, yet nothing in the architecture forces it to align with a corresponding technical-analysis description. In practice, one could separately train a text generator on descriptions, but unless the two systems are explicitly coupled, there is no guarantee that the generated text corresponds to the generated market sequence.

A third relevant family is **autoregressive sequence modeling**, including transformer-based forecasting architectures. Temporal Fusion Transformers and related sequence models have become influential in structured forecasting because they capture long-range dependencies, variable importance, and heterogeneous temporal covariates. Their joint law factorizes as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{1:t-1}), \tag{3.7}$$

or, in the multimodal case, one may imagine a similar factorization over text tokens. These architectures are powerful for prediction and sometimes for generation. But in practice they are often trained in a forecasting regime, not a fully multimodal generative regime. A model trained to predict future OHLCV or indicators may produce accurate trajectories while remaining completely agnostic about the language that would describe them. Conversely, a language model trained on technical-analysis text may generate fluent sentences without any access to the underlying market path.

The shared limitation across these paradigms is therefore structural. Let x_0 denote the financial-feature sequence and w_0 the technical-analysis text. A unimodal modeling strategy effectively replaces the joint problem by separate marginals:

$$p_\theta^{\text{uni}}(x_0, w_0) = p_{\theta_x}(x_0)\, p_{\theta_w}(w_0). \tag{3.8}$$

This decomposition ignores the dependence structure between the modalities. Even if each marginal is individually well modeled, the pair sampled from (3.8) need not be coherent. The generated feature path may suggest a bearish breakdown, while the generated sentence may describe a bullish

rebound. There is no mechanism that forces agreement.

This limitation is especially problematic in finance because the relationship between numbers and language is not ornamental. Technical-analysis text is often a compressed summary of jointly observed price, volume, momentum, and volatility structure. A sentence such as "price consolidates above support with weakening momentum" encodes relations among multiple variables and timescales. If text and features are generated independently, this latent coordination is lost. One may measure the resulting inconsistency by comparing learned embeddings e_x and e_w of the two modalities:

$$\Delta_{\mathrm{cm}} = \|e_x - e_w\|_2. \tag{3.9}$$

In a unimodal pipeline that pairs independently generated outputs after the fact, Δ_{cm} is unconstrained by the training objective. Any observed alignment is accidental rather than enforced.

A common engineering workaround is **late fusion**. One separately processes prices and text, and combines them only at the output or decision stage. For instance, one may generate a financial sequence, then feed it to a second model that produces a description, or vice versa. Although this is sometimes effective for downstream tasks, it remains unsatisfactory as a generative principle. The modalities are not co-evolved during the generative process. Instead, one modality is conditioned on an already finished realization of the other. This design can improve coherence, but it still treats multimodality as an add-on rather than a core property of the learned data distribution.

The diffusion literature highlights why this is a limitation. Diffusion models excel because they progressively refine a noisy sample through a sequence of denoising operations [6]. If the sample is multimodal, then consistency should ideally be enforced throughout the denoising process, not only at the end. Similarly, Diffusion Transformers showed that a Transformer backbone can support highly expressive denoising across complex distributions [16]. Recent multimodal diffusion work went further, showing that one Transformer can fit heterogeneous modalities within a shared generative framework [1]. These developments suggest that the right solution to the unimodal limitation is not better post hoc pairing, but a genuinely multimodal denoising architecture.

The financial literature has not yet fully caught up to this idea. Time-series GANs, VAE return generators, and autoregressive forecasting systems remain useful and important, but they operate on a single representational channel at a time. They are not designed to learn the joint geometry of price behavior and language description. That is the gap this paper addresses.

3.1.3 Dual-Head Diffusion as a Multimodal Alternative

The central architectural proposal of this paper is **dual-head diffusion**: a multimodal generative system in which a single shared Transformer backbone is trained simultaneously to denoise continuous financial-feature sequences and to generate discrete technical-analysis tokens. The model receives noisy or partially masked versions of both modalities, processes them through a shared representation

stack, and then branches into two output heads: a **feature head** that predicts Gaussian noise for the continuous sequence and a **text head** that predicts masked tokens for the discrete sequence. The key distinction from unimodal or late-fusion systems is that the two modalities are refined together inside one shared denoising process.

At a high level, let x_t denote the noised continuous feature sequence and w_t the partially masked text sequence at diffusion or masking step t. The dual-head model computes

$$(\hat{\epsilon}, \text{logits}) = f_\theta^{\text{dual}}(x_t, w_t, t), \tag{3.10}$$

where $\hat{\epsilon}$ is the predicted continuous noise and logits are the token scores for the text head. This compact expression hides the main innovation, namely that the function f_θ^{dual} is not simply two unrelated subnetworks bundled together. It is a shared multimodal denoiser with explicit cross-modal fusion pathways.

The motivation for this design is straightforward. If prices and language are two views of one market event, then a generative model should let each view inform the other during generation. The numerical sequence should constrain the text, because certain technical descriptions are only plausible for certain price behaviors. The text should also influence the numerical denoising path, because a partially recovered linguistic regime label or descriptive phrase can supply coarse semantic information that helps organize the feature trajectory. In this sense, the two modalities are not merely co-generated; they are **co-refined**.

This is analogous in spirit to multimodal representation learning in other domains, but with a specifically generative objective. CLIP-style models learn alignment between image and text embeddings through contrastive supervision [17]. The present framework instead learns a joint generative representation by denoising continuous features and reconstructing masked tokens under a shared backbone. The objective is not just alignment, but alignment within a full probabilistic generation process.

The dual-head formulation also changes the interpretation of the Transformer backbone. In a purely continuous diffusion model, the Transformer learns to map noisy feature tensors toward cleaner ones. In a text-only masked diffusion model, a Transformer learns to reconstruct missing tokens from context. In the dual-head design, the shared backbone learns a representation space in which both tasks coexist. It must therefore internalize not only intra-modal regularities, such as temporal dependence among OHLCV features or local syntax among text tokens, but also **inter-modal regularities**, such as which textual descriptors belong to which price regimes.

This can be written abstractly in terms of a shared latent representation:

$$h_{\text{shared}}^{x,w} = \text{Backbone}_\theta(h_t^x, h_t^w, t), \tag{3.11}$$

where h_t^x and h_t^w are modality-specific input embeddings. The feature head and text head then

read from this common state. Since the shared state is trained under both objectives, it becomes a carrier of cross-modal consistency.

An important consequence is that the generated outputs are not independent conditional on the backbone; rather, they are coupled through it. This differs sharply from a system in which a price generator and a text generator are trained separately and then combined. In the separate case, the best one can hope for is conditional pairing after generation. In the dual-head case, the outputs are jointly shaped at every layer. This is why the model is better viewed as a multimodal financial engine than as two coordinated unimodal engines.

The diffusion aspect of the design is also crucial. Continuous feature denoising is performed through the DDPM framework of [6], while text generation is handled through masked token prediction, inspired by recent masked diffusion language models. The feature modality therefore uses Gaussian noise, whereas the text modality uses masking noise. These cannot be naively unified into one corruption process because the spaces are fundamentally different. Yet they can be jointly processed by the same Transformer backbone, which is precisely what enables multimodal diffusion.

The result is a model that fits naturally into a broader trend toward unified multimodal generative systems [1], but tailored to finance. Here the modalities are not image and caption, but financial sequence and technical text. The pairing is domain-specific, but the underlying generative logic is the same: learn a shared latent structure that supports coherent generation across modalities.

3.1.4 Cross-Modal Fusion versus Late Fusion

The novelty of the proposed design lies not only in having two heads, but in how those heads communicate. A multimodal architecture can combine modalities in many ways. The simplest is **late fusion**, in which each modality is processed largely independently and only merged near the output. Late fusion is easy to implement and often adequate for downstream prediction. It is considerably weaker, however, as a generative principle.

To see why, consider a late-fusion system in which one network processes the feature sequence and another processes the text sequence. Let their outputs be $g_x(x_t, t)$ and $g_w(w_t, t)$, respectively. A fused output might then be

$$y = \text{Combine}(g_x(x_t, t), g_w(w_t, t)). \tag{3.12}$$

This design allows the final decision or output to depend on both modalities, but it does not require the internal representations of one modality to shape the denoising dynamics of the other. The feature generator can proceed almost entirely without regard to the text generator, and vice versa. The resulting model is multimodal only at the surface.

The present paper instead advocates **cross-modal fusion inside the shared backbone**. In this design, intermediate feature representations are directly fused with pooled text information, and intermediate text representations receive a residual injection of pooled feature information. At the

conceptual level, the feature pathway is updated not only by the noisy numerical sequence itself, but also by a compressed summary of the current textual context. Symmetrically, the text pathway is updated not only by the partially masked token context, but also by a softened summary of the shared feature state.

The distinction is important because it changes what the model can learn. With internal fusion, the shared representation can encode coupled structures such as "high volume plus downward momentum corresponds to bearish language" or "range-bound oscillation corresponds to consolidation terminology." These are not learned as post hoc associations between finished outputs; they are learned as denoising relationships embedded inside the hidden-state geometry. This is precisely the kind of structure one wants in a multimodal financial generator.

One may write the abstract fusion operations as

$$h^x_{\text{fused}} = \text{Fuse}_x(h^x, h^w), \qquad h^w_{\text{fused}} = \text{Fuse}_w(h^w, h^x). \tag{3.13}$$

This bidirectionality matters. If only text were conditioned on features, the system would reduce to a description generator attached to a feature model. If only features were conditioned on text, it would reduce to a text-guided feature generator. The dual-head approach is more symmetric: both modalities participate in the construction of the shared state.

There is also a statistical argument. If the true data-generating law is joint, then the optimal hidden representation should, in principle, preserve dependence information between the modalities. Late fusion tends to preserve such dependence only at the point of decision, not throughout the generative process. Internal fusion offers a richer route because it allows dependence structure to accumulate across layers. This is closer in spirit to the shared semantic alignment achieved by multimodal representation models such as CLIP [17], but now embedded inside a diffusion-style generative process.

In financial applications, internal fusion also supports a valuable practical property: **coherence under partial information**. During sampling, the feature sequence is refined gradually from noise, and the text tokens are refined gradually from masked or uncertain states. If the modalities are fused throughout the backbone, then partially recovered structure in one modality can help stabilize and guide the other. This produces a co-refinement mechanism rather than a one-way conditioning mechanism.

For these reasons, cross-modal fusion is not a minor architectural embellishment. It is the core reason the model can claim to be truly multimodal. Without it, one would merely have a pair of coordinated generators. With it, one has a shared denoising system in which market features and language descriptions shape one another during generation.

3.1.5 Contributions of the Paper

Against this background, the paper makes three contributions.

The first contribution is the formal development of a **dual-head cross-modal fusion architecture** for multimodal financial generation. The model uses a shared AdaLN-conditioned Transformer backbone with two outputs: a feature denoising head for continuous OHLCV-derived sequences and a text generation head for paired technical-analysis descriptions. The novelty lies in the bidirectional information pathways that connect the two modalities inside the backbone, rather than combining them only after separate processing.

The second contribution is a **joint training objective** that combines continuous epsilon-prediction mean squared error and masked-token cross-entropy under a principled weighting parameter α. Because both losses backpropagate through the same shared fusion layers, the model is encouraged to learn representations that are useful for both numerical fidelity and linguistic coherence. This is important conceptually: the objective is not merely multitask learning, but consistency-enforcing joint denoising.

The third contribution is the introduction of a **feature-conditioned SentimentDiffTransformer** for day-by-day sentiment token generation conditioned on observed OHLCV features. This extends the multimodal philosophy beyond paired sentence generation and toward structured sequence annotation, in which a market path can be accompanied by an aligned discrete sentiment trajectory. The result is a more expressive and financially relevant multimodal system.

Taken together, these contributions support the paper's central claim: complex generative models for finance should learn joint distributions over prices, indicators, and language simultaneously. A model that produces only one of these is incomplete relative to the data-generating structure of actual market interpretation.

3.1.6 Experimental Context and Notebook Motivation

The theoretical development in this paper is grounded in a deliberately controlled experimental setting. The feature modality consists of synthetic financial sequences of length $T = 60$ with $F = 12$ dimensions, representing OHLCV variables together with seven technical indicators. The text modality consists of short technical-analysis descriptions generated from a library of twenty template sentences per four regime types. Text is tokenized with a simple whitespace tokenizer and padded or truncated to a fixed length of $L = 24$ tokens. The sentiment model operates on daily sentiment tokens aligned to the feature sequence.

These design choices are intentionally modest. They are not meant to represent the full richness of real-world technical analysis, equity research, or trading commentary. Rather, they create a controlled multimodal laboratory in which the architectural ideas can be isolated and studied. A larger corpus, richer tokenizer, or more realistic market simulator would add scale and domain

richness, but could also obscure the conceptual contribution of the dual-head design.

The implementation target is practical accessibility. The full training procedure is designed to run on a standard T4 GPU in under fifteen minutes, making the model suitable for educational Colab notebooks and rapid prototyping. This matters because one of the goals of the paper is not only to introduce a new multimodal architecture, but to do so in a way that is reproducible and auditable. The open-source notebook context therefore shapes the paper: the mathematics are formal, but the implementation remains close enough to an executable laboratory that each component can be inspected directly.

This experimental framing also clarifies what the paper does and does not claim. It does claim that joint feature-text generation is a more faithful generative target than unimodal generation. It does claim that cross-modal fusion inside a shared diffusion Transformer is a principled way to pursue that target. It does not claim that the current template-based language environment is already equivalent to real financial discourse. The value of the notebook is therefore methodological: it demonstrates the architecture in a setting simple enough to understand fully, yet rich enough to reveal why multimodal generation matters in finance.

The rest of the paper proceeds accordingly. Section 2 formalizes the multimodal data space, the dual forward processes, the shared backbone, the joint loss, and the feature-conditioned sentiment model. Section 3 describes the complete training and sampling pipeline. Section 4 examines financial applications, including synthetic paired dataset generation and multimodal imputation. Section 5 discusses limitations, open problems, and connections to frontier multimodal diffusion research.

3.2 Mathematical Framework

3.2.1 Multimodal Data Representation

The objective of this paper is to model a **joint financial data distribution** over two coupled modalities: a continuous market-feature sequence and a discrete natural-language description. The first modality captures the numerical structure of a market episode; the second captures the linguistic interpretation of that same episode. The mathematical point of departure is therefore not a single data space but a product space with heterogeneous geometry.

Let

$$x_0 \in^{T \times F}, \tag{3.14}$$

denote the continuous feature sequence, where $T = 60$ is the sequence length and $F = 12$ is the number of features. In the motivating implementation, these features consist of OHLCV variables together with seven technical indicators. Let

$$w_0 \in \{1, \ldots, V\}^L, \tag{3.15}$$

denote the discrete text-token sequence, where $L = 24$ is the maximum text length and V is the vocabulary size induced by the tokenizer. The full data point is therefore the pair (x_0, w_0).

The generative target is the joint law

$$p(x_0, w_0), \tag{3.16}$$

not merely the marginals

$$p(x_0) \qquad \text{and} \qquad p(w_0). \tag{3.17}$$

This distinction is fundamental. If one learns the joint law (3.16), the marginals can be recovered via

$$p(x_0) = \sum_{w_0} p(x_0, w_0), \qquad p(w_0) = \int p(x_0, w_0)\, dx_0. \tag{3.18}$$

By contrast, learning only the marginals leaves the dependence structure unspecified. In particular, there are infinitely many joint laws consistent with a given pair of marginals. The dependence information is measured by the mutual information

$$I(X; W) = \left[\log \frac{p(x_0, w_0)}{p(x_0)p(w_0)}\right]. \tag{3.19}$$

Whenever $I(X; W) > 0$, the two modalities share information that cannot be recovered from the marginals alone.

If $I(X; W) > 0$, then learning the joint distribution $p(x_0, w_0)$ is strictly more informative than learning only the marginals $p(x_0)$ and $p(w_0)$ separately.

The marginals determine no unique coupling between X and W. Mutual information is zero if and only if the joint factorizes as $p(x_0, w_0) = p(x_0)p(w_0)$. Therefore, when $I(X; W) > 0$, a model that learns only the marginals omits dependence structure. Since the joint law determines both marginals and dependence, it is strictly more informative.

The same conclusion can be expressed through the **data processing inequality**. Let $g(X)$ be any compressed or transformed representation of the continuous feature modality. Then

$$I(X; W) \geq I(g(X); W). \tag{3.20}$$

Similarly, for any representation $h(W)$ of the text modality,

$$I(X; W) \geq I(X; h(W)). \tag{3.21}$$

These inequalities imply that if one projects the multimodal problem into only one modality, one cannot increase the recoverable information about the other modality. In practical terms, a price-only generator necessarily loses part of the information carried by text, and a text-only generator necessarily loses part of the information carried by prices.

This argument is especially important in finance because the two modalities are not weakly related

annotations. The text description often summarizes patterns that are distributed across several numerical features and over multiple time steps. Conversely, the numerical path may be more useful when paired with its textual interpretation. Therefore, the relevant statistical object is the joint distribution over market sequences and market narratives. The role of the model is to learn an approximation

$$p_\theta(x_0, w_0) \approx p(x_0, w_0), \tag{3.22}$$

in which both within-modality and cross-modality structure are retained.

A final point concerns sequence alignment. The current paper treats x_0 and w_0 as paired but not necessarily token-time aligned one-to-one. The feature sequence has length $T = 60$, whereas the text has length $L = 24$. The shared relation is semantic rather than positional. This justifies the use of pooled cross-modal information rather than strict tokenwise alignment in the fusion design introduced below. The two modalities are views of the same event, but they are not isomorphic sequences. The mathematics must therefore respect both their dependence and their structural asymmetry.

3.2.2 The Dual Diffusion Forward Process

Because the two modalities live in different spaces, they require different corruption processes. The continuous feature sequence is naturally modeled with Gaussian diffusion, whereas the discrete text sequence is naturally modeled with masking diffusion. The forward process is therefore **dual**: one corruption mechanism for continuous features and another for discrete tokens. This is not an arbitrary design choice. It reflects the fact that the Euclidean structure of $^{T\times F}$ and the categorical structure of $\{1, \ldots, V\}^L$ are fundamentally different.

For the continuous feature modality, the forward process follows the DDPM construction [6, 14]. Let

$$q_x(x_{1:T_{\text{diff}}} \mid x_0) = \prod_{t=1}^{T_{\text{diff}}} q_x(x_t \mid x_{t-1}), \tag{3.23}$$

with Gaussian transitions

$$q_x(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t^{(x)}}\, x_{t-1}, \beta_t^{(x)} I\right). \tag{3.24}$$

Define

$$\alpha_t^{(x)} = 1 - \beta_t^{(x)}, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s^{(x)}. \tag{3.25}$$

Using the cosine schedule of [14], one obtains a cumulative signal coefficient $\bar{\alpha}_t$ that preserves information more smoothly across diffusion time than a linear schedule. The resulting closed-form marginal is

$$q_x(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I\right), \tag{3.26}$$

and the corresponding reparameterization is

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{3.27}$$

For the discrete text modality, Gaussian perturbation is inappropriate because token identities do not form a Euclidean vector space. Instead, we define a masked forward process with an absorbing mask state. Let [MASK] denote a special token outside the ordinary vocabulary. At masking horizon T_{mask}, define the linear masking schedule

$$\beta_t^{(w)} = \frac{t}{T_{\text{mask}}}. \tag{3.28}$$

At step t, each token position j is independently replaced by [MASK] with probability $\beta_t^{(w)}$, unless it is a padding token, which is excluded from masking. The forward process factorizes across positions:

$$q_w(w_t \mid w_0) = \prod_{j=1}^{L} q_w(w_{t,j} \mid w_{0,j}), \tag{3.29}$$

with

$$q_w(w_{t,j} = w_{0,j} \mid w_{0,j}) = 1 - \beta_t^{(w)}, \qquad q_w(w_{t,j} = [\text{MASK}] \mid w_{0,j}) = \beta_t^{(w)}. \tag{3.30}$$

Once a token is masked, the corruption is absorbing at that step: the state records only that the original token is hidden. The reverse model must infer it from context.

A natural question is whether one could force both modalities to share a single noise schedule. The answer is no in any principled sense, because the geometry of the corruption process differs. The continuous modality admits a signal-to-noise trade-off expressed through Gaussian variance and cumulative coefficients such as $\bar{\alpha}_t$. The discrete modality admits instead a visibility-to-masking trade-off determined by the probability of token erasure. There is no canonical continuous-to-discrete isomorphism that would make the same schedule meaningful in both domains. Formally, the feature corruption lives in a Gaussian family over ${}^{T\times F}$, while the text corruption lives in a categorical family over sequences with an absorbing state:

$$q_x(\cdot \mid x_0) \in \mathcal{P}({}^{T\times F}), \qquad q_w(\cdot \mid w_0) \in \mathcal{P}(\{1, \ldots, V, [\text{MASK}]\}^L), \tag{3.31}$$

and these families obey different algebraic and geometric constraints.

This separation is not a weakness. It is what allows the model to respect the distinct nature of both modalities while still processing them jointly through a shared backbone. The dual forward process therefore defines the multimodal denoising problem precisely: recover a clean financial-feature sequence from Gaussian corruption and recover a clean technical-analysis text from masking corruption, doing so in a way that allows each modality to inform the other.

3.2.3 The Shared Backbone and Cross-Modal Fusion

The two corrupted inputs x_t and w_t are passed into a **shared Transformer backbone** that learns multimodal hidden representations. The central design principle is that each modality should preserve its own local structure while also receiving contextual information from the other. This is achieved through modality-specific input projections followed by bidirectional cross-modal fusion.

For the feature modality, let

$$h^x = \mathrm{Proj}_x(x_t) \in^{T \times d}, \tag{3.32}$$

where Proj_x is a learned linear map from feature dimension F into model dimension d. Positional encodings are added across the T time steps. For the text modality, let

$$h^w = \mathrm{Embed}(w_t) \in^{L \times d}, \tag{3.33}$$

where Embed is a learned token embedding map and positional encodings are added across the L token positions. The shared backbone then processes both representations jointly, but not symmetrically in the same tensor. Instead, each modality maintains its own sequence while receiving pooled context from the other.

The feature-to-text fusion route is defined through pooled text context. Let

$$\mathrm{Pool}(h^w) \in^{d} \tag{3.34}$$

denote a pooling operator over text positions, such as mean pooling or the hidden state of a special summary token. Broadcast this pooled vector across the T feature positions and concatenate it with h^x:

$$[h^x, \mathrm{Broadcast}(\mathrm{Pool}(h^w))] \in^{T \times 2d} . \tag{3.35}$$

The fused feature representation is then

$$h^x_{\mathrm{fused}} = \mathrm{CrossFuse}([h^x, \mathrm{Broadcast}(\mathrm{Pool}(h^w))]) , \tag{3.36}$$

where CrossFuse is a learned linear projection from $2d$ back to d. This operation allows every feature-time position to access a global summary of the current text state.

The reverse pathway, text-to-feature conditioning, is deliberately softer. Let h^x_{shared} denote the feature representation after some shared processing blocks. Pool it into a global feature context and broadcast it across the L text positions. The final text representation is then

$$h^w_{\mathrm{final}} = h^w + 0.3 \cdot \mathrm{Broadcast}(\mathrm{Pool}(h^x_{\mathrm{shared}})) . \tag{3.37}$$

The scale factor 0.3 is crucial. It makes the conditioning **soft** rather than dominant. The text head receives informative feature context, but the original stochastic and contextual variation in the text

stream is not overwhelmed by feature information.

This can be made precise. Suppose instead that text conditioning were hard, meaning the final text representation were replaced by a deterministic function of the feature stream:

$$h^w_{\text{hard}} = \Psi(h^x_{\text{shared}}), \tag{3.38}$$

with negligible dependence on the original text state h^w. Then the text head would predict tokens through

$$p_\theta(w_0^{\text{mask}} \mid w_t, x_t) = p_\theta(w_0^{\text{mask}} \mid x_t), \tag{3.39}$$

which implies that, conditional on the features, the masked token distribution loses residual stochasticity from partial text context. In the extreme, the text output becomes a deterministic function of the features.

If the text head is conditioned through a fully dominant deterministic map from feature representations and the original masked text state contributes no residual information, then the text generation process collapses to a deterministic function of the feature modality.

Assume $h^w_{\text{final}} = \Psi(h^x_{\text{shared}})$ with no dependence on h^w. Then the logits of the text head are a deterministic function of x_t, say $\ell = \Gamma(\Psi(h^x_{\text{shared}}))$. Therefore

$$p_\theta(w_0^{\text{mask}} \mid w_t, x_t) = p_\theta(w_0^{\text{mask}} \mid x_t).$$

The masked tokens no longer depend on residual token uncertainty or masked-text context except through x_t. Hence the reverse text process cannot express multiple plausible linguistic realizations for the same feature context beyond what is encoded deterministically. This is the stated collapse.

The scale factor 0.3 in (3.37) prevents this collapse by ensuring that

$$h^w_{\text{final}} \neq \Psi(h^x_{\text{shared}}) \quad \text{alone}, \tag{3.40}$$

and therefore the conditional token distribution remains influenced by both masked text context and feature context. Intuitively, the text head is nudged toward feature consistency, not forced into a single canonical description.

The shared backbone itself is built from attention blocks in the standard Transformer sense [22]. If H denotes a hidden sequence, self-attention computes

$$\text{Attn}(Q, K, V) = \left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{3.41}$$

with learned projections for queries, keys, and values. The multimodal novelty lies not in changing attention mechanics, but in how the hidden states entering those mechanics are fused across modalities. The backbone is therefore shared at the representation level, while the modalities remain

distinct enough to preserve their own structure.

3.2.4 The Joint Training Objective

The dual-head architecture is trained under a **joint objective** that combines a continuous denoising loss for features and a masked-token prediction loss for text. The design principle is simple: if the backbone is shared, then both objectives should update it, forcing it to encode representations that are simultaneously useful for numerical fidelity and linguistic coherence.

The continuous feature loss is the DDPM epsilon-prediction objective:

$$L_{\text{feat}} = \left[\|\epsilon - \epsilon_\theta(x_t, t, w_t)\|_2^2\right], \tag{3.42}$$

where ϵ is the Gaussian noise used in the forward process and ϵ_θ is the feature head's prediction. The dependence on w_t indicates that the denoiser has access to the text stream through the shared backbone.

The text loss is masked cross-entropy over the positions that were hidden in the forward masking process:

$$L_{\text{text}} = \left[-\log p_\theta(w_0^{\text{mask}} \mid w_t, x_t)\right]. \tag{3.43}$$

Equivalently, if M is the masked-position set,

$$L_{\text{text}} = \left[-\sum_{j \in M} \log p_\theta(w_{0,j} \mid w_t, x_t)\right]. \tag{3.44}$$

The full joint objective is

$$L_{\text{joint}} = \alpha L_{\text{feat}} + (1-\alpha) L_{\text{text}}, \tag{3.45}$$

where $\alpha \in [0, 1]$ controls the relative weighting. In the implementation motivating this paper, $\alpha = 0.6$, so the continuous feature denoising task receives slightly greater weight than the text task:

$$L_{\text{joint}} = 0.6 L_{\text{feat}} + 0.4 L_{\text{text}}. \tag{3.46}$$

This choice reflects the practical fact that feature fidelity is slightly more sensitive to training instability, while the text branch remains simpler due to its small vocabulary and template-based corpus.

To see why joint training enforces cross-modal consistency, let θ_{shared} denote the parameters of the shared backbone and fusion layers. Then

$$\nabla_{\theta_{\text{shared}}} L_{\text{joint}} = \alpha \nabla_{\theta_{\text{shared}}} L_{\text{feat}} + (1-\alpha) \nabla_{\theta_{\text{shared}}} L_{\text{text}}. \tag{3.47}$$

Because both losses depend on the hidden states produced after fusion, their gradients pass through the cross-modal pathways. Writing the fusion layers abstractly as F_x and F_w, one has schematically

$$\nabla_{\theta_{\text{shared}}} L_{\text{feat}} = \frac{\partial L_{\text{feat}}}{\partial h^x_{\text{fused}}} \frac{\partial h^x_{\text{fused}}}{\partial \theta_{\text{shared}}}, \tag{3.48}$$

and

$$\nabla_{\theta_{\text{shared}}} L_{\text{text}} = \frac{\partial L_{\text{text}}}{\partial h^w_{\text{final}}} \frac{\partial h^w_{\text{final}}}{\partial \theta_{\text{shared}}}. \tag{3.49}$$

Since h^x_{fused} depends on pooled text context and h^w_{final} depends on pooled feature context, each branch indirectly pushes the backbone to represent the other modality in a useful way. This is the mechanism by which (3.45) does more than multitask optimization: it enforces pressure toward a shared representation that is cross-modally compatible.

The role of α can be interpreted geometrically. If α is too high, the backbone may overfit the continuous geometry and treat text as an auxiliary side task. If α is too low, feature denoising may degrade and the model may prioritize linguistic template recovery over numerical realism. Thus $\alpha = 0.6$ represents an empirical compromise between preserving the primary financial sequence and maintaining coherent paired language. In future work, one may replace fixed α with a dynamic schedule or uncertainty-weighted multitask scheme, but the present paper adopts a fixed value to keep the joint objective interpretable and reproducible.

3.2.5 The Feature-Conditioned Sentiment Model

In addition to the dual-head feature-text model, the framework includes a separate **feature-conditioned sentiment diffusion model**. The motivation is that a market path may naturally admit a day-by-day sentiment annotation sequence, and such a sequence is not the same object as the short technical-analysis text generated by the dual-head text head. The sentiment model therefore provides a complementary multimodal component: a conditional discrete sequence generator aligned to the full time horizon.

Let

$$s_0 \in \{0, \ldots, K-1\}^T \tag{3.50}$$

denote the discrete sentiment-token sequence, where K is the number of sentiment classes and the sequence length matches the feature horizon T. A masked forward process is defined analogously to the text branch, with masking schedule

$$\beta^{(s)}_t = \frac{t}{T_{\text{sent}}}. \tag{3.51}$$

At step t, each sentiment token is independently masked with probability $\beta^{(s)}_t$, producing a corrupted sequence s_t.

The reverse model is a bidirectional Transformer conditioned on a pooled feature-context vector extracted from the continuous feature sequence. First, a lightweight feature encoder g_ϕ maps the observed feature sequence into a contextual representation:

$$g_\phi(x_0) \in^{T \times d_c}, \tag{3.52}$$

where g_ϕ is implemented as a two-layer MLP or shallow temporal encoder. The pooled feature context is then

$$c_x = \mathrm{MeanPool}(g_\phi(x_0)) \in^{d_c} . \tag{3.53}$$

This vector is broadcast or injected into the sentiment Transformer so that token recovery is conditioned on the observed OHLCV-derived path. The reverse conditional model is therefore

$$p_\phi(s_0 \mid s_t, x_0), \tag{3.54}$$

and training minimizes masked cross-entropy:

$$L_{\text{sent}} = \left[-\log p_\phi(s_0^{\text{mask}} \mid s_t, x_0) \right]. \tag{3.55}$$

Equivalently, over masked positions M_s,

$$L_{\text{sent}} = \left[-\sum_{j \in M_s} \log p_\phi(s_{0,j} \mid s_t, x_0) \right]. \tag{3.56}$$

The conceptual role of this model is different from that of the text head in the main dual-head system. The text head produces a short descriptive sentence or phrase sequence summarizing the entire financial episode. The sentiment model instead produces a token at each day, conditioned on the path. One may think of the former as **global multimodal narration** and the latter as **local multimodal annotation**. Both are useful, but they address different aspects of multimodal financial generation.

This separate sentiment model also clarifies a broader methodological point. Multimodality in finance need not mean only one text channel attached to one numerical channel. It can mean several discrete or linguistic channels aligned to the same continuous process. The architecture introduced here is therefore best viewed not as a closed endpoint but as a modular foundation. The dual-head model captures the joint law of features and technical-analysis text, while the SentimentDiffTransformer captures the conditional law of daily sentiment given the feature path. Together they show how diffusion-style methods can be extended from unimodal financial generation toward a richer multimodal generative ecosystem.

3.3 Pipeline and Algorithm

3.3.1 Overview of the Dual-Head Training Pipeline

The mathematical framework in the previous section defined the joint data space, the dual corruption mechanisms, the shared backbone, the cross-modal fusion operators, the joint loss, and the auxiliary sentiment model. The present section turns those abstractions into an executable training and sampling system. The central practical question is how to operationalize a multimodal financial generator in a way that remains both mathematically coherent and computationally tractable.

The full pipeline begins with paired synthetic data. Each training instance consists of a continuous feature tensor and a discrete text description. Formally, the feature dataset is

$$X \in^{N \times T \times F}, \tag{3.57}$$

and the text dataset is

$$W \in \{1, \ldots, V\}^{N \times L}, \tag{3.58}$$

where N is the number of paired examples, $T = 60$ is the feature-sequence length, $F = 12$ is the feature dimension, $L = 24$ is the token length, and V is the vocabulary size. In the motivating notebook, the continuous features are synthetic OHLCV-derived sequences with technical indicators, and the text strings are short regime-aware technical-analysis templates tokenized by whitespace.

Before training, the continuous features are normalized. Let μ_X and σ_X be empirical statistics over the training set. A normalized sample is obtained by

$$\tilde{x}_0 = \frac{x_0 - \mu_X}{\sigma_X + \varepsilon_{\text{norm}}}, \tag{3.59}$$

where $\varepsilon_{\text{norm}} > 0$ is a small stabilizing constant. This normalization is important because the forward Gaussian diffusion process assumes isotropic noise injection in feature space. If raw feature magnitudes are highly imbalanced, the denoiser is forced to solve a scale-distorted reconstruction problem. Standardization makes the diffusion geometry more uniform.

The training loop then proceeds along two synchronized corruption streams. For the feature modality, one samples a continuous diffusion timestep

$$t_{\text{feat}} \sim \text{Uniform}\{1, \ldots, 1000\}, \tag{3.60}$$

draws Gaussian noise

$$\epsilon \sim \mathcal{N}(0, I), \tag{3.61}$$

and constructs the corrupted feature input

$$x_{t_{\text{feat}}} = \sqrt{\bar{\alpha}_{t_{\text{feat}}}}\, x_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{feat}}}}\, \epsilon. \tag{3.62}$$

For the text modality, one samples a masking timestep

$$t_{\text{text}} \sim \text{Uniform}\{1, \ldots, 500\}, \tag{3.63}$$

and masks each non-padding token with probability $t_{\text{text}}/500$. The result is a partially masked sequence $w_{t_{\text{text}}}$.

These two corrupted modalities are then passed jointly into the dual-head model, denoted `StockDiffTx`. The forward pass returns a predicted noise tensor for the feature branch and token logits for the text branch:

$$(\hat{\epsilon}, \ell) = \text{StockDiffTx}(x_{t_{\text{feat}}}, w_{t_{\text{text}}}, t_{\text{feat}}). \tag{3.64}$$

The feature loss is mean squared error,

$$L_{\text{feat}} = \|\hat{\epsilon} - \epsilon\|_2^2, \tag{3.65}$$

while the text loss is masked cross-entropy,

$$L_{\text{text}} = (\ell[\text{mask}], w_0[\text{mask}]). \tag{3.66}$$

The combined loss is

$$L = 0.6L_{\text{feat}} + 0.4L_{\text{text}}. \tag{3.67}$$

This weighting carries over directly from the mathematical framework and expresses the design choice that feature fidelity is slightly prioritized without allowing the text branch to become incidental.

The optimization step is performed with mixed precision, gradient clipping, AdamW, and cosine learning-rate decay. Mixed precision is especially valuable because it reduces memory usage and improves throughput on commodity GPUs such as the T4 while preserving model quality under appropriate loss scaling [13]. Gradient clipping at norm 1.0 controls instability across heterogeneous minibatches, and AdamW provides robust optimization with decoupled regularization [11]. The use of cosine learning-rate decay complements the cosine feature diffusion schedule and supports a stable decay in effective update scale over training.

After the dual-head model is trained, the `SentimentDiffTransformer` is trained separately. This separation is conceptually useful. The main model learns the joint law of feature sequences and global technical-analysis text. The sentiment model learns a conditional law for day-by-day sentiment tokens given the feature sequence. Training them separately keeps the interpretation of each component clean and allows one to study their contributions independently.

Sampling mirrors this modularity. The feature branch uses DDIM-style reverse denoising over a reduced set of 50 timesteps. The text branch uses greedy co-refinement: at each reverse feature step, the text logits are decoded and the current token sequence is updated. The result is a pair (x_0, w) refined jointly over the reverse process. This is an intentionally simple but effective way to

produce cross-modally aligned outputs without requiring a full masked diffusion reverse chain over the text modality at every stage.

Thus, the pipeline can be summarized as follows. First, prepare paired multimodal financial data. Second, normalize the continuous features. Third, jointly train the dual-head model under Gaussian feature corruption and masked text corruption. Fourth, train the sentiment model as a separate conditional discrete diffusion system. Fifth, sample with co-refinement DDIM for the main model and masked conditional generation for the sentiment model. The next subsections formalize this into explicit algorithms and architectural detail.

3.3.2 Algorithm 1: Dual-Head Joint Training

The dual-head training algorithm is the primary computational engine of the framework. It must simultaneously solve two denoising problems of different types: Gaussian denoising for the continuous feature modality and masked-token reconstruction for the text modality. Unlike a standard multitask setup, however, these objectives are not merely co-located. They are coupled through the shared backbone and the fusion operators, which means the algorithm must be interpreted as learning a joint multimodal denoising geometry.

The first step is schedule preparation. For the continuous feature modality, the model precomputes the cosine diffusion buffers

$$\left\{\beta_t, \alpha_t, \bar{\alpha}_t, \sqrt{\bar{\alpha}_t}, \sqrt{1-\bar{\alpha}_t}\right\}_{t=1}^{1000}. \tag{3.68}$$

These quantities define the full geometry of the feature forward process. Precomputation is important because it ensures exact consistency across batches and avoids repeated floating-point reconstruction during training. In diffusion models, schedules are not hyperparameters in the loose sense; they are structural components of the stochastic process.

At each minibatch iteration, the algorithm samples a set of feature diffusion times and text masking times. For simplicity, and as in the motivating notebook, one may use one timestep per sample or a broadcasted timestep across the batch. The feature timestep is sampled uniformly:

$$t_{\text{feat}} \sim \text{Uniform}\{1, \ldots, 1000\}, \tag{3.69}$$

and the text timestep is sampled uniformly from the masking horizon:

$$t_{\text{text}} \sim \text{Uniform}\{1, \ldots, 500\}. \tag{3.70}$$

This choice ensures that the model experiences the full range of denoising difficulty across both modalities.

The continuous corruption step follows the standard reparameterized DDPM form. If x_0 is a clean

feature sample and ϵ is sampled Gaussian noise, then

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon. \tag{3.71}$$

The text corruption step differs categorically. Let M denote the set of non-padding positions selected for masking. Then the corrupted text sequence w_t is defined by

$$w_{t,j} = \begin{cases} [\text{MASK}], & j \in M, \\ w_{0,j}, & j \notin M. \end{cases} \tag{3.72}$$

The masking probability is $t_{\text{text}}/500$, and padding tokens are excluded to avoid wasting probability mass on positions that carry no content.

The forward pass then computes

$$(\hat{\epsilon}, \text{text_logits}) = \text{StockDiffTx}(x_t, w_t, t_{\text{feat}}). \tag{3.73}$$

The feature head produces a tensor of the same shape as the input feature sequence and is trained against the known Gaussian perturbation:

$$L_{\text{feat}} = (\hat{\epsilon}, \epsilon). \tag{3.74}$$

The text head produces a vocabulary-sized logit vector at each text position, and the loss is computed only on masked locations:

$$L_{\text{text}} = (\text{text_logits}[M], w_0[M]). \tag{3.75}$$

The total loss is

$$L = 0.6 L_{\text{feat}} + 0.4 L_{\text{text}}. \tag{3.76}$$

Optimization uses automatic mixed precision (AMP) with a gradient scaler. If L is the raw loss, the scaled loss is

$$L_{\text{scaled}} = \lambda_{\text{scale}} L, \tag{3.77}$$

for a dynamic scaling factor λ_{scale} chosen by the scaler. The purpose is to preserve small gradients in half precision while still benefiting from reduced memory and faster tensor operations [13]. After backpropagation, the gradients are unscaled and clipped:

$$g \leftarrow g \cdot \min\left(1, \frac{1.0}{\|g\|_2}\right). \tag{3.78}$$

The parameters are then updated with AdamW and a cosine learning-rate schedule.

The following formal algorithm summarizes the procedure.

[h] Dual-Head Joint Training [1] **Input:** feature tensor $X \in^{N \times T \times F}$, text token matrix $W \in$

$\{1, \ldots, V\}^{N \times L}$ Precompute cosine schedule buffers for the feature diffusion process each minibatch (x_0, w_0) Sample $t_{\text{feat}} \sim \text{Uniform}\{1, \ldots, 1000\}$ Sample $\epsilon \sim \mathcal{N}(0, I)$ Compute $x_t = \sqrt{\bar{\alpha}_{t_{\text{feat}}}} x_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{feat}}}} \epsilon$ Sample $t_{\text{text}} \sim \text{Uniform}\{1, \ldots, 500\}$ Mask w_0 to w_t with probability $t_{\text{text}}/500$, skipping PAD tokens Forward pass: $(\hat{\epsilon}, \text{text_logits}) = \text{StockDiffTx}(x_t, w_t, t_{\text{feat}})$ Compute $L_{\text{feat}} = (\hat{\epsilon}, \epsilon)$ Compute $L_{\text{text}} = (\text{text_logits}[M], w_0[M])$ Compute $L = 0.6 L_{\text{feat}} + 0.4 L_{\text{text}}$ Backward pass with AMP and GradScaler Clip gradient norm at 1.0 AdamW update and cosine learning-rate schedule

It is important to note that Algorithm 1 is not simply parallel training of two heads. Because the backbone is shared and the losses propagate through the fusion pathways, the algorithm trains a single multimodal latent geometry. This is what differentiates the method from a system that trains a feature diffusion model and a text masked model independently.

3.3.3 Algorithm 2: Co-Refinement DDIM Sampling with Text Decoding

Training defines the model, but generation reveals what the model has actually learned. The main challenge in sampling is that the two modalities evolve under different reverse semantics. The feature modality is denoised continuously through DDIM, while the text modality is updated through token prediction. The sampling design adopted here is **co-refinement**: the feature sequence and text sequence are iteratively updated together, each influencing the other through the shared backbone.

The reverse process begins by initializing the feature modality from Gaussian noise:

$$x_T \sim \mathcal{N}(0, I), \tag{3.79}$$

where T denotes the terminal DDIM timestep. The text modality is initialized from a simple categorical prior, which in the notebook implementation is a uniform draw over vocabulary entries:

$$w \sim \text{Uniform}(\{1, \ldots, V\})^L. \tag{3.80}$$

This is intentionally simple. It does not claim to be the optimal text reverse prior, but it provides a practical starting point for joint co-refinement.

At each timestep t in a reduced 50-step DDIM schedule, the model computes

$$(\hat{\epsilon}_t, \ell_t) = \text{StockDiffTx}(x_t, w_t, t), \tag{3.81}$$

where $\hat{\epsilon}_t$ is the predicted feature noise and ℓ_t are the current text logits. The clean feature estimate is reconstructed as

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_t}{\sqrt{\bar{\alpha}_t}}, \tag{3.82}$$

and the DDIM update is

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\epsilon}_t. \tag{3.83}$$

Simultaneously, the text state is updated greedily:

$$w_{t-1} = (\ell_t). \tag{3.84}$$

Thus the next iteration uses both a denoised feature state and an updated token state.

This sampling procedure differs from full masked diffusion reversal over text. In a proper MDLM-style reverse process, one would gradually unmask text positions according to a designed reverse schedule. Here, the text state is updated greedily at every feature reverse step. This simplification sacrifices some text diversity, but it greatly reduces complexity and aligns well with the practical notebook setting. More importantly, it still captures the central idea of multimodal co-refinement: the evolving feature state and evolving text state remain coupled throughout sampling.

The full algorithm is as follows.

[h] Co-Refinement DDIM Sampling with Text Decoding [1] Initialize $x_T \sim \mathcal{N}(0, I)$ Initialize $w \sim \text{Uniform}(\{1, \ldots, V\})^L$ t in reverse 50-step DDIM schedule Compute $(\hat{\epsilon}_t, \text{text_logits}_t) = \text{StockDiffTx}(x_t, w_t, t)$ Compute $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_t)/\sqrt{\bar{\alpha}_t}$ Update $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\hat{\epsilon}_t$ Update $w_{t-1} = (\text{text_logits}_t)$ Return $(x_0, w, \text{decoded_text})$

The practical value of this procedure lies in its iterative consistency. Because the text tokens are refreshed at each DDIM step, the text branch can react to the progressively denoised feature sequence. Conversely, because the updated text state is fed back into the shared backbone, the feature denoiser is conditioned not on a static placeholder text sequence, but on a progressively more coherent narrative state. This is what makes the process a co-refinement system rather than a one-shot attached decoder.

3.3.4 Co-Refinement Mechanism and Shared Representation Flow

The co-refinement mechanism is the most distinctive practical feature of the model. Standard unimodal sampling would denoise the feature sequence independently and then generate text from the final feature output, or else generate text independently and pair the outputs post hoc. The present method instead updates both modalities during the reverse trajectory itself. This produces a coupled denoising process in which each modality is repeatedly exposed to the evolving state of the other.

Let z_t^x and z_t^w denote pooled feature and text representations extracted from the shared backbone at reverse step t. Then the sampling state may be viewed as the triple

$$(x_t, w_t, z_t), \qquad z_t = \text{Backbone}_\theta(x_t, w_t, t). \tag{3.85}$$

The output heads read from this common representation:

$$\hat{\epsilon}_t = H_x(z_t), \qquad \ell_t = H_w(z_t), \tag{3.86}$$

where H_x and H_w are the feature and text heads. Since both outputs are functions of the same z_t, any change in one modality that affects z_t can influence the refinement of the other.

This is especially important because the two modalities carry different kinds of information. The feature modality is high dimensional and temporally structured. The text modality is compressed and semantic. Early in reverse sampling, the text branch may provide only crude signals, but even a crude token pattern such as a dominant regime word can influence the backbone's interpretation of the feature trajectory. Later in the reverse process, as the features become clearer, the text branch can lock onto a more coherent description. Thus, the information flow is asymmetric across time even though the fusion itself is bidirectional.

The co-refinement perspective can be contrasted with conditional generation. In a conditional model, one modality is fixed and the other is generated from it:

$$p_\theta(w \mid x) \qquad \text{or} \qquad p_\theta(x \mid w). \tag{3.87}$$

In co-refinement, by contrast, neither modality is final until the reverse trajectory completes. The process is closer to iterative fixed-point search for a mutually consistent pair. One may write the stepwise update abstractly as

$$(x_{t-1}, w_{t-1}) = \Phi_\theta(x_t, w_t, t), \tag{3.88}$$

where Φ_θ encapsulates both DDIM denoising and token update. The final sample is then the result of repeated application of Φ_θ over the reduced timestep schedule.

This viewpoint is useful conceptually because it connects the method to broader multimodal diffusion research. In image-text systems, iterative interaction between modalities has been explored through shared latent spaces, joint denoisers, or guidance mechanisms. The present framework translates that idea into finance, where the modalities are continuous OHLCV-style sequences and discrete technical text. The core principle remains the same: a jointly generated pair should be consistent not only at the end, but throughout the reverse construction process.

3.3.5 Three-Head Text Architecture and Hyperparameters

The text branch of the main model is not a monolithic shallow decoder. It is better understood as a three-stage structure embedded around the shared backbone. The first stage consists of **two pre-blocks** that process the masked text sequence into a modality-specific representation before heavy cross-modal interaction. The second stage consists of **six shared blocks** in which feature and text pathways exchange information through the shared Transformer stack. The third stage consists

of **two post-blocks** in which the text branch receives final feature injection before projecting to token logits.

This organization is useful because it gives the text modality both autonomy and coupling. The pre-blocks let the text tokens establish local syntactic and positional coherence before interacting heavily with the features. The shared blocks carry the main burden of multimodal integration. The post-blocks let the text branch reconcile the fused information into final token predictions. Thus, the architecture avoids the two extremes of total separation and immediate overfusion.

Let the model width be d, the number of shared blocks be $L_s = 6$, the number of pre-blocks $L_{\text{pre}} = 2$, and the number of post-blocks $L_{\text{post}} = 2$. In the motivating notebook-scale implementation, the width is modest and chosen to fit commodity hardware, while still allowing meaningful cross-modal capacity. If H denotes the number of attention heads, each head processes subspaces of size d/H:

$$d = H \cdot d_h. \tag{3.89}$$

The exact numerical choices may vary across runs, but the architectural structure remains fixed.

The core hyperparameters of the main dual-head model are as follows. The feature diffusion process uses 1000 forward steps and a 50-step DDIM reverse sampler. The text masking horizon is 500. The loss weighting is fixed at

$$\alpha = 0.6, \tag{3.90}$$

so the joint loss remains

$$L = 0.6L_{\text{feat}} + 0.4L_{\text{text}}. \tag{3.91}$$

Optimization uses AdamW, mixed precision, and gradient clipping at norm 1.0. The learning rate follows a cosine schedule:

$$\eta_u = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi u}{U}\right), \tag{3.92}$$

where u is the update step and U the total number of training steps.

The notebook's practical design target is a T4 GPU with training under approximately fifteen minutes. This constraint influences the selected width, depth, batch size, and tokenizer simplicity. It is therefore important not to interpret the present hyperparameters as universal optima. They are part of a deliberately compact research laboratory, chosen to make the multimodal architecture inspectable and reproducible.

3.3.6 Separate SentimentDiffTransformer Pipeline

The `SentimentDiffTransformer` is trained separately because it solves a distinct problem: conditional generation of a day-by-day discrete sentiment sequence given an observed feature path.

Whereas the main dual-head text branch produces a short global description, the sentiment model produces a token sequence of length T aligned with the daily feature sequence.

Its input dataset is a pair (x_0, s_0), where

$$x_0 \in^{T \times F}, \qquad s_0 \in \{0, \ldots, K-1\}^T. \tag{3.93}$$

The sentiment forward process masks positions in s_0 with linearly increasing probability. A masking timestep is sampled:

$$t_{\text{sent}} \sim \text{Uniform}\{1, \ldots, T_{\text{sent}}\}, \tag{3.94}$$

and each sentiment token is replaced by a mask symbol with probability $t_{\text{sent}}/T_{\text{sent}}$.

The conditioning signal is derived from the feature path through a small encoder g_ϕ. If

$$c_x = \text{MeanPool}(g_\phi(x_0)), \tag{3.95}$$

then the sentiment Transformer conditions on both the masked sentiment sequence s_t and the pooled feature context c_x. The output is a categorical distribution over sentiment classes at each masked position:

$$p_\phi(s_{0,j} \mid s_t, x_0), \qquad j \in M_s. \tag{3.96}$$

The loss is masked cross-entropy,

$$L_{\text{sent}} = -\sum_{j \in M_s} \log p_\phi(s_{0,j} \mid s_t, x_0). \tag{3.97}$$

The separate training of this model has two advantages. First, it keeps the main dual-head architecture focused on feature-text generation rather than overloading it with an additional day-level output head. Second, it yields a modular sentiment annotator that can be applied to observed or generated OHLCV paths. This is useful in finance because sentiment annotations are often needed as structured metadata even when one does not require a full descriptive sentence.

Taken together, the main dual-head training algorithm, the co-refinement DDIM sampler, the shared-representation flow, the three-stage text branch, and the separate sentiment pipeline define a complete multimodal generative system. The next section shows how these components can be used in finance for paired synthetic dataset generation, automated technical analysis, sentiment-conditioned scenarios, multimodal imputation, and cross-modal consistency filtering.

3.4 Applications in Finance

3.4.1 Paired Synthetic Dataset Generation for LLM Fine-Tuning

One of the most immediate and practically significant uses of the proposed dual-head architecture is the generation of **paired synthetic datasets** of the form

$$\mathcal{D}_{\text{syn}} = \{(\tilde{x}^{(i)}, \tilde{w}^{(i)})\}_{i=1}^{M}, \tag{3.98}$$

where each $\tilde{x}^{(i)} \in^{T\times F}$ is a synthetic financial-feature sequence and each $\tilde{w}^{(i)} \in \{1, \ldots, V\}^L$ is a paired technical-analysis text generated jointly with that sequence. The value of such a dataset is clear in the current financial AI landscape. Fine-tuning financial language models, alignment models, or multimodal assistants often requires substantial volumes of domain-relevant training data. Real datasets, however, may be restricted by licensing, confidentiality, privacy, or proprietary research constraints. A generative model that can produce aligned *price-plus-language* pairs therefore creates a useful synthetic alternative.

The central point is that the paired structure matters. A language model trained only on free-floating technical-analysis phrases may learn domain vocabulary but not the relation between those phrases and actual market configurations. Conversely, a model trained only on feature windows learns market structure but not how human-readable explanations relate to that structure. The dual-head model addresses this by sampling both modalities from an approximate joint law:

$$(\tilde{x}, \tilde{w}) \sim p_\theta(x_0, w_0). \tag{3.99}$$

This is more useful for multimodal downstream training than drawing

$$\tilde{x} \sim p_{\theta_x}(x_0), \qquad \tilde{w} \sim p_{\theta_w}(w_0), \tag{3.100}$$

independently and pairing the outputs arbitrarily.

There are several reasons why such synthetic data are attractive for fine-tuning. First, they scale. Once the trained model is available, one can generate arbitrarily large synthetic corpora at relatively low marginal cost. Second, they preserve controllable structure. By varying seeds, sampling schedules, or acceptance filters, one can generate datasets with different degrees of stylistic variety and regime balance. Third, they reduce legal and governance friction. Even when real financial commentary cannot be redistributed or stored broadly, a synthetic paired dataset may be easier to govern, especially in educational, experimental, or pre-production contexts.

In the present framework, paired sample generation proceeds through the co-refinement DDIM sampler described earlier. Starting from Gaussian noise in feature space and a random or weakly initialized token sequence in text space, the model iteratively refines both until it returns a coherent

pair $(\tilde{x}, \tilde{w})$. Since both modalities are shaped by the shared backbone during reverse-time generation, the resulting pairs are not simply price sequences with post hoc captions. They are generated through a process that encourages semantic consistency between the two.

A natural validation metric for synthetic paired corpora is **text diversity**. Because the training text corpus in the motivating notebook is based on templates, one must guard against trivial memorization or narrow interpolation. A simple and interpretable metric is bigram diversity:

$$\text{BigramDiv} = \frac{\#\{\text{unique bigrams in generated corpus}\}}{\#\{\text{all generated bigrams in corpus}\}}. \tag{3.101}$$

This statistic measures how varied the generated token transitions are relative to the size of the output corpus. Let $\mathcal{B}_{\text{gen}}$ be the multiset of generated bigrams. Then

$$\text{BigramDiv}_{\text{gen}} = \frac{|\text{Unique}(\mathcal{B}_{\text{gen}})|}{|\mathcal{B}_{\text{gen}}|}. \tag{3.102}$$

Comparing this to the template corpus baseline provides a basic sense of whether the model is merely reproducing a narrow subset of template transitions or generating a broader set of plausible textual patterns.

One can also define a relative diversity ratio:

$$\rho_{\text{div}} = \frac{\text{BigramDiv}_{\text{gen}}}{\text{BigramDiv}_{\text{tmpl}}}, \tag{3.103}$$

where $\text{BigramDiv}_{\text{tmpl}}$ is the bigram diversity of the original template corpus. If ρ_{div} is too low, the text generator is collapsing to a small template neighborhood. If it is substantially above one, the generator may be recombining known patterns in a richer but still constrained manner.

For multimodal fine-tuning, however, diversity alone is not enough. The generated pair must also be aligned. A useful extension is to combine text diversity with a consistency score based on the shared embedding space. Let z_x and z_w denote pooled feature and text representations from the shared backbone. Their cosine similarity is

$$\text{CMCons}(\tilde{x}, \tilde{w}) = \frac{\langle z_x, z_w \rangle}{\|z_x\|_2 \|z_w\|_2}. \tag{3.104}$$

This provides a quality check on whether the generated text belongs to the generated feature path in the model's own learned geometry.

The application therefore has a clear workflow. Train the dual-head model, sample a large corpus of $(\tilde{x}, \tilde{w})$ pairs, compute text diversity and optional consistency filters, and retain the resulting synthetic dataset for multimodal fine-tuning. Such a dataset could support tasks such as financial captioning, sequence-to-analysis generation, structured report drafting, or alignment of small financial language models without exposing proprietary real-world corpora. This is precisely the kind of use case for

which synthetic multimodal financial generation is more valuable than unimodal generation.

3.4.2 Automated Technical Analysis Report Generation

A second application is **automated technical analysis report generation**. In many financial workflows, the purpose of a generative model is not merely to create plausible numerical scenarios, but to accompany those scenarios with a concise and readable interpretation. Analysts, students, and downstream language systems often need a textual description that summarizes the main features of a price window: trend direction, range behavior, momentum status, volume anomalies, or likely support-resistance structure. The text head of the dual-head diffusion model provides exactly such a mechanism.

The problem can be formalized as follows. Given a generated or observed feature path x, one seeks a descriptive token sequence w such that the pair (x, w) is jointly plausible under the learned multimodal distribution. In the simplest operational version of the present model, the text output is obtained via greedy decoding from the logits produced during co-refinement:

$$w_{t-1} = (\ell_t), \tag{3.105}$$

with the final output after reverse refinement denoted by

$$\hat{w} = \mathrm{Decode}(w_0). \tag{3.106}$$

The resulting sentence can be interpreted as a candidate technical-analysis report for the associated feature sequence.

This is useful in at least three settings. First, if the feature path itself is synthetic, the text head produces a narrative explanation that can make the synthetic sample easier to inspect and use. Second, if the feature path is real but partially corrupted or imputed during a generative workflow, the text head provides a description of the reconstructed episode. Third, if one is producing educational or analyst-facing tools, the model can act as a drafting engine that translates structured market data into short natural-language summaries.

The value of this application lies in its **cross-modal grounding**. A standard language model could be prompted with abstract regime labels or a few hand-engineered indicators, but it would not necessarily remain tightly linked to the full sequence geometry. The dual-head model, by contrast, generates text from a shared representation learned jointly with feature denoising. This means the text is not simply conditioned on a post hoc feature summary; it is produced as part of the same generative state that organizes the financial sequence itself.

Validation of automated report generation is a delicate matter. Human evaluation is often desirable, but in a notebook-scale synthetic environment one also wants quantitative measures. A natural choice is to compare the embedding of the generated text to the embeddings of regime-matched

template descriptions. Let e_{gen} denote an embedding of the generated text and e_{tmpl} an embedding of a template sentence corresponding to the same known or inferred regime. Then the cosine similarity is

$$\text{Sim}_{\text{text}} = \frac{\langle e_{\text{gen}}, e_{\text{tmpl}} \rangle}{\|e_{\text{gen}}\|_2 \|e_{\text{tmpl}}\|_2}. \tag{3.107}$$

A high similarity indicates that the generated text lies near the intended template family in semantic space.

If multiple regime-matched templates are available, one may take the maximum or average similarity:

$$\text{Sim}_{\max} = \max_{k \in \mathcal{T}(r)} \frac{\langle e_{\text{gen}}, e_k \rangle}{\|e_{\text{gen}}\|_2 \|e_k\|_2}, \tag{3.108}$$

where $\mathcal{T}(r)$ is the set of templates associated with regime r. This provides a regime-aware quantitative measure of whether the generated sentence is semantically compatible with what one would expect for that market state.

A second possible metric is lexical fidelity relative to regime-specific keywords. Suppose each regime is associated with a keyword set $\mathcal{K}(r)$. Then one can define a keyword coverage rate

$$\text{Cov}_{\mathcal{K}}(\hat{w}, r) = \frac{\#\{\text{tokens in } \hat{w} \cap \mathcal{K}(r)\}}{|\mathcal{K}(r)|}. \tag{3.109}$$

This is less semantically sophisticated than embedding similarity, but in a small-template environment it can still be informative.

The broader significance of this application is methodological. Technical analysis is often thought of as a textual overlay on top of numerical data. The present model treats it instead as a paired generative output. This means the system can be used not just to classify market regimes, but to *describe* them in a form that is directly consumable by humans or downstream language models. In a larger-scale future extension with richer text corpora and better tokenization, this could evolve from short sentence drafting into paragraph-level or report-level generative analysis. The current notebook-scale implementation is therefore a proof of concept for a broader direction: multimodal financial generation as joint sequence-and-report synthesis.

3.4.3 Sentiment-Conditioned Scenario Generation

A third application uses the separate `SentimentDiffTransformer` to produce **day-by-day sentiment sequences conditioned on an OHLCV path**. This differs from the previous text-generation application in an important way. The main dual-head text branch generates a short, global description of an entire market episode. The sentiment model generates a discrete token at each time step, thereby providing a structured annotation sequence aligned with the full path.

Formally, given an observed or generated feature sequence $x_0 \in^{T \times F}$, the task is to sample

$$s_0 \in \{0, \ldots, K-1\}^T \tag{3.110}$$

from the conditional law

$$p_\phi(s_0 \mid x_0). \tag{3.111}$$

In practice, the sentiment model uses masked discrete diffusion. It begins with a masked or partially masked sentiment sequence, conditions on a pooled feature-context vector derived from x_0, and iteratively reconstructs the missing sentiment tokens. The resulting output is a day-level sentiment trace that describes how the market path might be labeled across time.

This application is attractive because sentiment is often used in finance as an explanatory layer that complements price dynamics. In classical quantitative settings, sentiment is sometimes treated as an external covariate, for example from news or analyst text. Here, by contrast, the sentiment sequence is generated *from* the price-and-indicator path, creating an internally consistent annotation of the scenario. This is useful for synthetic data generation, because one may want each price scenario to come with a structured sentiment path that can be used by downstream classifiers, interpreters, or reinforcement-learning environments.

The model component used here is distinct from the main dual-head generator. First, the feature path is encoded into a pooled context:

$$c_x = \mathrm{MeanPool}(g_\phi(x_0)), \tag{3.112}$$

where g_ϕ is the feature encoder. Then the masked sentiment sequence s_t is processed by a bidirectional Transformer conditioned on c_x, yielding token probabilities for the masked positions:

$$p_\phi(s_{0,j} \mid s_t, x_0), \qquad j \in M_s. \tag{3.113}$$

This conditional setup is well suited to sentiment generation because sentiment is not being treated as an independent sequence; it is explicitly generated *in light of* the feature path.

A basic validation metric compares the distribution of generated sentiment tokens to that of real or training sentiment tokens. Let $F^{(s)}_{\text{real}}$ and $F^{(s)}_{\text{gen}}$ denote empirical cumulative distributions over sentiment-token classes. Then the Kolmogorov–Smirnov statistic is

$$D^{(s)}_{\text{KS}} = \sup_k \left| F^{(s)}_{\text{real}}(k) - F^{(s)}_{\text{gen}}(k) \right|. \tag{3.114}$$

Although sentiment classes are discrete rather than continuous, this KS-style comparison is still informative as a measure of mismatch in empirical token distributions.

A second useful metric is transition consistency. Because sentiment is a sequence, not merely a bag of labels, one may examine the empirical transition matrix of the generated sequences. Let $\hat{P}_{\text{gen}}$

and $\hat{P}_{\text{real}}$ be generated and real transition matrices over sentiment classes. Then define

$$\Delta_P = \|\hat{P}_{\text{gen}} - \hat{P}_{\text{real}}\|_F. \tag{3.115}$$

A small Δ_P indicates that the generated sentiment process reproduces not only token frequencies but also their temporal evolution.

This application can be used in several ways. One use is to annotate synthetic price scenarios with sentiment trajectories, thereby enriching synthetic datasets for downstream multimodal training. Another is to use sentiment as an interpretable auxiliary output for human inspection: a generated path accompanied by a daily sentiment trace is easier to analyze than a path alone. A third is to support conditional scenario design. For example, one may generate price paths and then retain only those whose sentiment traces satisfy certain structural constraints, such as prolonged pessimism, rapid reversal, or alternating optimism and caution.

In this sense, the sentiment model extends the dual-head philosophy. It shows that once a multimodal generative framework is in place, one can attach additional conditional discrete channels aligned to the same financial sequence. The result is not merely a price generator with one attached captioner, but the beginning of a broader multimodal financial generative ecosystem.

3.4.4 Missing OHLCV Window Imputation with Text Consistency

A fourth application is **multimodal imputation**: the model fills missing windows of OHLCV-style features while simultaneously generating a text description of the imputed period. This is an especially compelling use case because it illustrates the advantage of multimodal generation over unimodal reconstruction. If only the feature values are imputed, the result may be numerically plausible but semantically opaque. If a text description is generated alongside the imputation, the reconstructed segment becomes easier to interpret, validate, and use in downstream tasks.

Let $x_0 \in^{T\times F}$ be a feature sequence with missing entries on a subset Ω^c of the index set $\{1,\dots,T\} \times \{1,\dots,F\}$. Let M_Ω be the corresponding observation mask:

$$M_\Omega(i,j) = \begin{cases} 1, & (i,j) \in \Omega, \\ 0, & (i,j) \in \Omega^c. \end{cases} \tag{3.116}$$

The numerical imputation problem is to infer $\hat{x}_{ij}$ for all $(i,j) \in \Omega^c$ while preserving consistency with the observed entries. In the dual-head setting, one additionally wishes to generate a text sequence $\hat{w}$ that describes the completed or reconstructed period coherently.

The feature branch handles imputation through a diffusion-style inpainting procedure. Abstractly, if $\Phi_\theta(x_t, w_t, t)$ denotes one reverse feature update conditioned on the current text state, then the

inpainting step can be written as

$$x_{t-1} = M_\Omega \odot x_{t-1}^{\text{obs}} + (1 - M_\Omega) \odot \Phi_\theta(x_t, w_t, t). \tag{3.117}$$

The observed entries are reinserted at each reverse step, while the missing entries are filled in by the learned denoiser. This preserves the known information while allowing the model to reconstruct plausible values for the unknown region.

What is distinctive in the present architecture is that the text head runs alongside this reconstruction. As the feature path is imputed, the text logits are updated through the shared backbone, eventually yielding a description of the imputed period:

$$\hat{w} = \text{Decode}((\ell_0)) \,. \tag{3.118}$$

This description need not refer only to the observed fragment or only to the imputed fragment. Rather, it describes the reconstructed full episode in the model's learned language space.

The validation metric for the numerical side is standard mean squared error over held-out windows:

$$\text{MSE}_{\Omega^c} = \frac{1}{|\Omega^c|} \sum_{(i,j)\in\Omega^c} (\hat{x}_{ij} - x_{ij})^2. \tag{3.119}$$

In synthetic experiments, the true values on Ω^c are known, so this metric can be computed directly. One may also supplement it with covariance or regime-structure diagnostics if preservation of broader structure is important.

For the text side, one can reuse the embedding-based or template-based metrics from the previous subsection. For example, if the imputed window belongs to a known regime family, one may compare the generated text embedding to regime-matched templates:

$$\text{Sim}_{\text{imp}} = \frac{\langle e_{\text{imp}}, e_{\text{tmpl}} \rangle}{\|e_{\text{imp}}\|_2 \|e_{\text{tmpl}}\|_2}. \tag{3.120}$$

This gives a rough quantitative sense of whether the generated description is semantically compatible with the reconstructed numerical episode.

The practical value of this application is substantial. Missing OHLCV windows are common in real financial pipelines due to market closures, illiquidity, vendor gaps, or asynchronous instruments. A purely numerical imputer may fill the gap, but the result is often difficult to interpret qualitatively. A multimodal imputer instead returns both a completed feature path and a candidate technical explanation. This is useful in educational settings, analyst tools, model debugging, and human-in-the-loop workflows where one wants not only an imputed number but also an interpretable narrative about what kind of period has been reconstructed.

Conceptually, this application also demonstrates the added value of joint modeling. The same

shared representation that helps the model generate paired samples also helps it impute missing data in a semantically grounded way. Thus, multimodal generation is not only about producing richer outputs; it can also improve the interpretability of inverse problems such as reconstruction and imputation.

3.4.5 Cross-Modal Consistency as a Quality Filter

The fifth application turns the shared embedding geometry of the model into a **quality-control mechanism**. Because the dual-head model learns a joint representation of feature sequences and technical-analysis text, one can use the agreement between the two modality embeddings as a diagnostic of sample quality. The idea is simple: if a generated feature path and generated text are strongly inconsistent in the learned representation space, then the pair is more likely to be low quality, semantically mismatched, or financially unhelpful. Such pairs can be rejected.

Let z_x and z_w denote pooled feature and text embeddings extracted from the shared backbone after sampling. Define the cross-modal consistency score

$$\mathrm{CMCons}(x, w) = \frac{\langle z_x, z_w \rangle}{\|z_x\|_2 \|z_w\|_2}. \tag{3.121}$$

This score lies in $[-1, 1]$ and is highest when the two representations align closely in the model's latent geometry. Given a threshold τ, define the acceptance set

$$\mathcal{A}_\tau = \{(x, w) : \mathrm{CMCons}(x, w) \geq \tau\}. \tag{3.122}$$

Generated samples outside $\mathcal{A}_\tau$ are rejected.

The practical intuition is that the shared backbone has already learned what kinds of text belong with what kinds of feature paths. If a sampled pair lies far apart in the shared embedding space, then the generation process has likely produced a mismatch. Such mismatches can arise because sampling is approximate, because greedy text decoding limits diversity in a way that sometimes harms alignment, or because the model's learned manifold is imperfect. The consistency filter turns this latent knowledge into an operational post-processing rule.

Why would this improve sample quality? Suppose that low-consistency samples are disproportionately associated with poor feature realism. Then rejecting them should raise the distributional similarity of the retained set relative to the target data. Let $D_{\mathrm{KS}}^{\mathrm{all}}$ denote the KS statistic between all generated feature marginals and the target marginals, and let D_{KS}^{τ} denote the KS statistic computed only on the accepted set $\mathcal{A}_\tau$. If the filter removes samples that are both semantically inconsistent and numerically atypical, one expects

$$D_{\mathrm{KS}}^{\tau} \leq D_{\mathrm{KS}}^{\mathrm{all}}. \tag{3.123}$$

If samples with low cross-modal consistency are disproportionately associated with poor feature

realism, then thresholding by $\mathcal{A}_\tau$ improves the effective KS similarity of the accepted sample set.

Let the generated sample set be partitioned into accepted and rejected subsets according to $\mathcal{A}_\tau$. Assume that the rejected subset has feature marginals farther from the target distribution than the accepted subset in the KS sense. Then the empirical CDF of the accepted subset is, by assumption, closer to the target CDF than the empirical CDF of the full sample. Therefore the KS statistic computed on the accepted subset is no larger than that computed on the full sample.

This proposition is intentionally conditional rather than universal. The filter improves quality only if low consistency is genuinely correlated with poor feature realism. In practice, that is an empirical question. However, the multimodal architecture provides a strong reason to expect such a correlation, because the shared backbone is explicitly trained to align useful feature and text representations. Samples that violate that alignment are likely to be the ones in which the generator has strayed away from the intended joint manifold.

A simple way to operationalize the filter is to compute the acceptance rate

$$\mathrm{AccRate}(\tau) = \frac{1}{M}\sum_{i=1}^{M}\{(\tilde{x}^{(i)}, \tilde{w}^{(i)}) \in \mathcal{A}_\tau\}, \tag{3.124}$$

and then study the trade-off between acceptance rate and retained-sample quality. Larger τ yields stricter selection and potentially higher quality but fewer usable samples. Smaller τ preserves more data but may admit more inconsistent outputs.

This application is especially attractive because it uses the multimodal nature of the model not only for generation, but for **self-evaluation**. A unimodal generator has no internal cross-modal agreement signal to exploit. The dual-head model does. That turns the shared embedding space into a built-in quality filter, which can be valuable for synthetic dataset generation, report generation, or any downstream workflow where one wishes to retain only the most coherent samples.

More broadly, this application supports the paper's central thesis. A multimodal financial generative model is valuable not only because it can produce more kinds of outputs, but because the interaction of those outputs creates new forms of diagnostic control. Cross-modal consistency is one such control. It allows the model to judge, at least approximately, whether its own generated price sequence and generated language belong together. In a field where sample realism is hard to assess and downstream misuse is a genuine concern, such internal consistency signals are particularly useful.

3.5 Challenges and Conclusions

3.5.1 Limitations of the Current Architecture

The dual-head diffusion Transformer developed in this paper is intended as a principled step toward multimodal financial generation, not as a finished production architecture. Its strengths are clear:

it learns a joint representation of continuous OHLCV-style feature sequences and technical-analysis text, it uses explicit cross-modal fusion rather than post hoc pairing, and it can be extended with a separate feature-conditioned sentiment model. Precisely because the framework is ambitious, however, it is important to state its limitations carefully. A multimodal financial generator becomes genuinely useful only when its capabilities are interpreted alongside the boundaries of what it can currently express.

A first limitation concerns the **text decoding strategy**. In the current implementation, the text head is updated during sampling through greedy decoding:

$$w_{t-1} = (\ell_t), \tag{3.125}$$

where ℓ_t denotes the token logits at reverse step t. This choice is computationally simple and works well in a compact notebook setting, but it suppresses stochastic diversity. Once a token achieves the highest logit score, it is selected deterministically, and alternative plausible descriptions are discarded. In a domain like finance, where several technical-analysis descriptions may all be compatible with the same price configuration, greedy decoding creates an artificial narrowing of the output space.

The limitation becomes clearer if one compares the current co-refinement rule with a proper masked-diffusion reverse process for the text modality. In a full discrete diffusion treatment, the text sequence would itself undergo a structured reverse chain in which masked positions are gradually refined according to a learned reverse transition kernel. Abstractly, one would want a process of the form

$$p_\theta(w_{t-1} \mid w_t, x_t), \tag{3.126}$$

iterated over a schedule of text timesteps, rather than a deterministic map

$$w_{t-1} = p_\theta(w_0 \mid w_t, x_t). \tag{3.127}$$

The current architecture therefore underuses the full generative capacity of the text branch. It treats text generation as a sequence of greedy updates coupled to feature denoising rather than as a full stochastic reverse process in its own right.

This simplification was reasonable for the current paper because it keeps the model computationally light and transparent, but it comes at a price. The generated texts tend to be high-confidence interpolations of patterns already emphasized by training rather than samples from a rich posterior over plausible descriptions. In practical terms, the feature branch may remain expressive while the text branch becomes stylistically conservative. Thus the model can achieve cross-modal consistency at the expense of linguistic diversity.

A second limitation concerns the **tokenization and corpus scale**. The current system uses a simple whitespace tokenizer and a relatively small template-based corpus of technical-analysis

phrases. This was a deliberate design choice for reproducibility and pedagogical clarity, but it strongly constrains the expressive range of the language modality. Let V denote the vocabulary size induced by the tokenizer. In the present notebook-scale setting, V is small and the text distribution is correspondingly narrow:

$$w_0 \in \{1, \ldots, V\}^L, \qquad V \text{ modest.} \tag{3.128}$$

This means that the model's text output should be interpreted as structured technical-language interpolation rather than broad financial language generation.

There are two related consequences. First, token boundaries are coarse. A whitespace tokenizer does not capture subword regularities, morphological variants, or compositional phrasing with the efficiency of a modern subword system. Second, the training corpus is limited in semantic breadth. The model can recombine and condition existing patterns, but it does not learn the full diversity of financial discourse. As a result, the generated texts are best understood as useful summaries inside a bounded technical register, not as production-quality analyst prose or open-ended research commentary. In a richer setting, one would likely want a byte-pair, unigram, or WordPiece-style tokenizer and a much broader financial text corpus.

A third limitation concerns the **cross-modal fusion strength**. In the current design, feature information enters the text branch through a fixed-scale residual injection,

$$h^w_{\text{final}} = h^w + 0.3 \cdot \text{Broadcast}(\text{Pool}(h^x_{\text{shared}}))\,. \tag{3.129}$$

The value 0.3 was chosen as a soft-conditioning constant: large enough to matter, small enough not to collapse text generation into a deterministic reflection of the features. This works well empirically in a compact training setting, but it remains a hand-tuned hyperparameter. There is no guarantee that the same value is optimal across different datasets, noise levels, sequence lengths, or textual richness.

The deeper issue is that conditioning strength should probably not be constant across diffusion time. At high noise levels, the feature signal is weak and ambiguous, so overly strong conditioning could inject unreliable feature context into the text branch. At low noise levels, by contrast, the feature representation is much clearer, and stronger conditioning may be appropriate. The current design ignores this dynamic structure. A more flexible alternative would introduce a learned gate λ_t depending on diffusion time:

$$h^w_{\text{final}} = h^w + \lambda_t \cdot \text{Broadcast}(\text{Pool}(h^x_{\text{shared}}))\,, \tag{3.130}$$

with λ_t produced by a small neural module or conditioning network. Such a mechanism would allow the model to adapt the intensity of cross-modal influence as a function of denoising stage.

A fourth limitation is **sampling asymmetry**. The feature modality enjoys a principled DDIM sampler with a reduced 50-step reverse process. The text modality does not yet enjoy an equally

principled co-sampler. Instead, it piggybacks on the feature reverse loop through greedy updates. This asymmetry means that the continuous branch is backed by a stronger generative story than the text branch. In the current model, feature realism is therefore likely to be more robust than textual richness. This is acceptable in a finance-first architecture where the numerical modality is primary, but it remains a limitation if one aims for truly symmetric multimodal generation.

A fifth limitation concerns **synthetic data realism**. The experiments motivating the paper are built on synthetic OHLCV-style sequences and templated texts. This is a strength from the standpoint of controlled methodology, but a limitation from the standpoint of external validity. Real financial markets involve more complex structural noise, richer discourse, institution-specific terminology, and much more heterogeneous cross-modal relations than those represented in the notebook laboratory. Thus, success in the current setting should be interpreted as proof of architectural viability, not proof of immediate production readiness.

A sixth limitation is **evaluation ambiguity**. Multimodal financial generation has no single universally accepted benchmark. A sample may be numerically plausible but semantically dull. It may be linguistically fluent but weakly grounded in the feature path. It may align well in latent space yet still fail to reproduce certain stylized facts. Formally, if Q denotes an evaluation functional, then there is no unique scalar quality map of the form

$$Q\big((x,w)_{\text{gen}},(x,w)_{\text{real}}\big) = \text{single universally decisive score.} \tag{3.131}$$

Instead, multimodal generation must be evaluated under multiple criteria: feature realism, text diversity, cross-modal consistency, regime faithfulness, and downstream usefulness. The current paper proposes several such criteria, but this remains an open methodological frontier.

Taken together, these limitations do not undermine the architecture. Rather, they define its present scope. The model is strongest as a compact, interpretable, and extendable multimodal laboratory. It demonstrates that joint feature-text diffusion is possible and useful. It does not yet claim to solve the full problem of open-ended multimodal financial generation.

3.5.2 Theoretical Open Problems

Beyond engineering limitations, the architecture raises several genuinely theoretical questions. These concern what exactly the model learns, how the joint reverse process behaves, and how multimodal weighting influences the learned representation geometry. Such questions are important because the long-term value of multimodal financial diffusion depends not only on empirical effectiveness but also on conceptual clarity.

The first open problem concerns **joint distribution recovery**. The model is trained to approximate the joint law of features and text, but it is not obvious under what conditions joint training truly recovers a coupled law rather than a product of approximately aligned marginals. Formally, the

target is

$$p_\theta(x_0, w_0) \approx p_{\text{true}}(x_0, w_0), \tag{3.132}$$

not merely

$$p_\theta(x_0, w_0) \approx p_{\theta_x}(x_0) p_{\theta_w}(w_0). \tag{3.133}$$

Because the model uses a shared backbone and cross-modal fusion, one expects it to learn dependence structure. But expectation is not proof. The precise conditions under which the joint loss and fusion mechanism guarantee recovery of nontrivial cross-modal dependence remain unclear.

One way to formalize the issue is through mutual information in the learned latent representation. Let Z_x and Z_w denote pooled latent embeddings for the two modalities under the shared backbone. If the model learns the joint law meaningfully, one would hope that

$$I_\theta(Z_x; Z_w) > 0 \tag{3.134}$$

in a way that reflects the true coupling between the observed modalities. But positive latent mutual information alone is not sufficient. It may reflect trivial co-adaptation or shortcut learning rather than meaningful recovery of the true joint structure. Thus, one open problem is to identify assumptions under which the joint diffusion objective induces consistent estimation of the true multimodal distribution.

The second open problem concerns the **convergence of the co-refinement DDIM sampler**. The feature branch follows a deterministic DDIM trajectory, while the text branch is updated through repeated greedy decoding conditioned on the evolving feature state. This defines an iterative map over the pair (x_t, w_t):

$$(x_{t-1}, w_{t-1}) = \Phi_\theta(x_t, w_t, t). \tag{3.135}$$

The final output (x_0, w_0) is obtained by repeatedly applying this map over the reverse schedule. But what kind of object is this sequence of updates in theoretical terms? Is it a contraction under some metric? Does it approximate a stationary point of a joint denoising energy? Or is it simply a useful heuristic that lacks a principled convergence interpretation?

If one had a fully stochastic reverse model for both modalities, then one could at least attempt to reason in terms of a coupled Markov chain over continuous and discrete states. In the present greedy text update scheme, however, the reverse process is partly deterministic and partly learned. It would therefore be valuable to characterize when repeated updates converge to a mutually consistent pair in the sense that further refinement would not change either modality substantially. One possible notion of fixed-point consistency is

$$\Phi_\theta(x^*, w^*, 0) = (x^*, w^*), \tag{3.136}$$

but the relation between such fixed points and true samples from the target joint distribution is not yet established.

A third theoretical question concerns the **role of the loss-weighting parameter** α. The joint loss is

$$L_{\text{joint}} = \alpha L_{\text{feat}} + (1 - \alpha) L_{\text{text}}. \tag{3.137}$$

Empirically, $\alpha = 0.6$ offers a useful balance between feature fidelity and text quality in the motivating notebook. But the theoretical meaning of this balance remains open. One possible hypothesis is that α affects the amount of mutual information preserved between modalities in the learned representations. If Z_x and Z_w are learned latent summaries, then one may ask whether there exists a systematic relation of the form

$$\alpha \mapsto I_\theta(Z_x; Z_w). \tag{3.138}$$

If so, then α would not merely be a practical weighting constant; it would be a knob controlling the degree of cross-modal coupling in the latent geometry.

This raises deeper questions. Is there an optimal α as a function of modality entropy, noise level, or sample complexity? Does too large an α suppress cross-modal information because the backbone over-specializes to feature denoising? Does too small an α distort the geometry of continuous generation by over-prioritizing templated text? More generally, how should one balance losses in multimodal diffusion when the modalities differ sharply in dimension, entropy, and noise process?

A fourth open problem concerns **identifiability of the fusion pathways**. The current model uses specific pooling and broadcast mechanisms to exchange information between modalities. But are these the only or best ways to induce multimodal coherence? There may be several architectures that yield similar empirical behavior while representing different internal decompositions of the joint law. This suggests that the learned fusion representation may not be uniquely identifiable. Establishing which aspects of the shared latent space are structural and which are architecture-dependent remains an important theoretical task.

A fifth question concerns **the relationship between discrete and continuous reverse-time semantics**. The model combines Gaussian diffusion for features with masking diffusion for text. These are coordinated empirically through the shared backbone, but the exact joint reverse-time interpretation is not yet fully unified. In principle, one would like a mathematical account of multimodal reverse diffusion in which both branches arise from a common generative formalism rather than from two adjoining but different denoising schemes. Unified multimodal diffusion research is beginning to address analogous problems in other domains, but the financial sequence-text setting remains wide open.

These theoretical questions matter because they determine how robustly the present architecture can scale. If multimodal financial diffusion is to become more than a clever notebook demonstration, one will need to understand when joint training truly learns the coupling of interest, when co-refinement is well behaved, and how the trade-off between modalities should be formalized. The current paper therefore offers not only an architecture but a research agenda.

3.5.3 Connections to Frontier Research

Although the present architecture is designed for finance, it sits naturally within several broader currents in generative modeling. Understanding these connections helps position the work more clearly and clarifies why multimodal financial diffusion is not an isolated curiosity, but part of a wider methodological movement.

The first connection is to **unified multimodal diffusion**. Recent work such as UniDiffuser argued that a single Transformer can be used to model multiple heterogeneous data distributions jointly [1]. The key idea is that different modalities need not be handled by completely separate model families if one can construct a shared latent processing framework with modality-specific heads. The dual-head financial architecture developed here is conceptually aligned with that idea. The difference is domain and scale: instead of images and captions or generic multimodal content, the modalities here are financial-feature sequences and technical-analysis text. The shared Transformer backbone with two output heads therefore functions as a finance-specific instance of the broader unified multimodal diffusion paradigm.

A second connection is to **text-conditioned image generation**. Models such as DALL-E 2 and Stable Diffusion demonstrated the power of conditioning one generative modality on another, especially through learned joint embedding spaces or guided diffusion [18, 19]. The financial architecture here does not perform text-to-image synthesis, but it does inherit the deeper lesson that generative quality improves when semantic information from one modality shapes the reverse process of another. In DALL-E 2-like settings, text embeddings guide image synthesis. In the present setting, pooled text context informs feature denoising, while pooled feature context informs text generation. The modalities differ, but the underlying generative logic is similar.

A third connection concerns **classifier-free guidance and controlled multimodal generation**. Guidance methods showed that one can interpolate between unconditional and conditional denoising fields to control generative behavior. Although the current notebook architecture does not yet implement full classifier-free guidance, it can be viewed as a step in that direction. The cross-modal fusion layers function as soft conditioners, and the fixed coefficient 0.3 in the text branch acts as a simple guidance strength. A future extension could make this explicit by training both conditional and unconditional branches and then applying guidance weights during sampling. This would move the architecture closer to the frontier of controllable multimodal diffusion.

A fourth connection is to **discrete diffusion and structured generation**. The text branch and the separate sentiment model rely on masked or discrete diffusion ideas that are increasingly important outside classical image synthesis. Recent work on discrete diffusion by ratio estimation and masked diffusion language modeling shows that diffusion-style generative logic is not confined to continuous spaces [12, 20]. The present paper contributes to this line indirectly by showing how discrete diffusion concepts can be paired with continuous financial diffusion under one multimodal architecture.

A fifth connection is to the broader theory of **representation learning in deep networks**. Modern deep learning has repeatedly shown that shared latent spaces can unify tasks that look superficially different. The deep learning literature provides the general conceptual backdrop: hidden layers can act as learned coordinate systems in which structure from different observation domains becomes comparable [5]. The dual-head model in this paper operationalizes that idea in a financial context. Feature paths and technical text are different raw objects, but the shared backbone learns a latent space in which they can inform one another. That is precisely the type of representational unification that frontier multimodal research increasingly relies upon.

These connections matter because they show that the current architecture is not merely an ad hoc solution to a niche financial problem. It is one local instance of a much broader transformation in generative modeling: the move from unimodal sample generation toward shared latent systems capable of supporting several data types at once. Finance is a natural domain for this evolution, because financial reality is already multimodal. The dual-head diffusion Transformer therefore sits at a meaningful intersection between frontier multimodal generative research and practical financial modeling.

3.5.4 Conclusions

This paper has introduced a multimodal generative framework in which continuous financial-feature sequences and technical-analysis language are learned jointly rather than separately. The motivating premise was that financial events are inherently multimodal: OHLCV structure, technical indicators, sentiment labels, and narrative descriptions are different expressions of the same underlying market state. A model that learns only prices is therefore incomplete relative to the data-generating phenomenon, just as a model that learns only technical text is incomplete relative to the underlying market path.

The first major contribution of the paper was the formal development of a **dual-head diffusion Transformer** with explicit **cross-modal fusion**. The architecture uses a shared AdaLN-conditioned backbone with a feature denoising head and a text generation head, together with bidirectional information pathways that allow feature context and text context to shape one another. This design distinguishes the model from late-fusion or independently trained multimodal systems, because the modalities are coupled throughout the generative process rather than only at the output stage.

The second major contribution was the definition of a **joint training objective** that combines epsilon-prediction mean squared error for the continuous branch with masked cross-entropy for the discrete text branch:

$$L_{\text{joint}} = \alpha L_{\text{feat}} + (1 - \alpha) L_{\text{text}}. \tag{3.139}$$

Because both objectives propagate through the shared backbone and fusion layers, the model is encouraged to learn hidden representations that support both numerical realism and semantic consistency. In this sense, the objective does more than combine two losses: it operationalizes

multimodal coherence during learning.

The third contribution was the introduction of the **feature-conditioned SentimentDiffTransformer**. This model extends the multimodal vision of the paper by showing that a generated or observed OHLCV path can be accompanied not only by a global descriptive sentence, but also by a structured day-by-day sentiment sequence. This expands the framework from paired sequence-text generation toward richer conditional multimodal annotation.

The practical applications developed in the paper illustrate why this matters. The model can generate paired synthetic datasets for downstream multimodal fine-tuning, provide candidate technical-analysis reports for generated paths, produce sentiment trajectories conditioned on feature sequences, impute missing OHLCV windows while generating a textual explanation, and filter low-quality outputs through cross-modal consistency scores. Each of these uses depends on the fact that the model learns more than one modality at once. A unimodal generator could achieve some of these goals partially, but it could not achieve them with the same internal consistency and diagnostic structure.

At the same time, the paper has deliberately been candid about limitations. Greedy decoding restricts textual diversity. The tokenizer and corpus remain too small for production-quality language generation. The fusion scale is fixed rather than learned. The text reverse process is simplified relative to full masked diffusion. The experimental environment is synthetic and controlled. These are genuine constraints, but they do not diminish the architectural significance of the framework. Rather, they clarify that the present model should be understood as a transparent and extensible research prototype.

The final conclusion is therefore precise. The **dual-head diffusion Transformer** represents a principled step toward truly multimodal financial generative modeling. It demonstrates that prices, indicators, and language can be learned within one denoising framework; that cross-modal fusion is more powerful than late fusion for coherent generation; and that multimodal latent consistency can become a useful tool for both generation and quality control. In a field where structured numerical data and structured human interpretation are inseparable, this is a meaningful advance. The long-term importance of such systems lies not only in generating synthetic financial data, but in making multimodal financial reasoning itself a native object of generative modeling.

Bibliography

[1] Fan Bao, Shen Nie, Kai Xue, Chongxuan Li, Hang Su, and Jun Zhu. One Transformer Fits All Distributions in Multi-Modal Diffusion at Scale. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, 1692–1717, 2023.

[2] Rama Cont. Empirical Properties of Asset Returns: Stylised Facts and Statistical Issues. *Quantitative Finance*, 1(2):223–236, 2001.

[3] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory.* 2nd ed., Wiley-Interscience, 2006.

[4] Prafulla Dhariwal and Alexander Nichol. Diffusion Models Beat GANs on Image Synthesis. In *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, 2016.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[7] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2022.

[8] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling Tabular Data with Diffusion Models. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, 17564–17579, 2023.

[9] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-LM Improves Controllable Text Generation. In *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[10] Marcos Lopez de Prado. *Advances in Financial Machine Learning.* John Wiley & Sons, 2018.

[11] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019.

[12] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 2024.

[13] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed Precision Training. In *International Conference on Learning Representations*, 2018.

[14] Alexander Q. Nichol and Prafulla Dhariwal. Improved Denoising Diffusion Probabilistic Models. In *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 8162–8171, 2021.

[15] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Amanda Askell, Paul Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[16] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.

[17] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning Transferable Visual Models from Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 8748–8763, 2021.

[18] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical Text-Conditional Image Generation with CLIP Latents. *arXiv preprint arXiv:2204.06125*, 2022.

[19] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.

[20] Subham Sekhar Sahoo, Carlos Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models. In *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[21] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*, 2021.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[23] Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance*, 20(9):1419–1440, 2020.

Conclusion

This wrapper was built to turn three related papers on diffusion models in finance into a coherent research book. That coherence matters because the deepest contribution of the volume is not only the technical content of each chapter, but the cumulative argument that emerges from reading them together. Synthetic financial generation, regime-aware diffusion, structured feature modeling, and multimodal sequence-and-text generation are not separate topics. They are connected stages in one broader effort: the effort to build financial generative systems that are probabilistically principled, diagnostically useful, and increasingly faithful to the multimodal structure of financial reality.

The first enduring lesson of the collection is that diffusion models are particularly well suited to finance because they are built around structured uncertainty. Financial data are noisy, heterogeneous, and often regime dependent. A framework that explicitly models corruption and recovery therefore aligns naturally with the domain. The second lesson is that generation should not be treated as an end in itself. A financial generative model becomes truly useful when it supports interpretation, diagnostics, imputation, scenario design, or downstream dataset construction. The third lesson is that the problem grows richer as one moves from returns, to structured market features, to text and sentiment. The financial generative task is therefore not shrinking toward one universal output object. It is expanding toward coupled modalities.

The book thus leaves the reader with a clear research direction. Diffusion models deserve a serious place in the future of financial machine learning not only because they can generate samples, but because they can help organize, interpret, and evaluate the probabilistic structure of financial phenomena. When paired with Transformer backbones and extended toward multimodal outputs, they become more than generators. They become a foundation for financial generative intelligence.

As a three-chapter volume, this book is compact. As a research direction, however, it opens outward. The chapters collected here are best understood not as the final word, but as a rigorous and coherent beginning.